# Aprendizaje Guiado con Gemini Notebook

## Inteligencia Artificial Explicable (XAI) - SHAP




Prof. Ing. Miguel Camacaro Pérez

## Introducción

Este documento aborda la integración de la Inteligencia Artificial Explicable (XAI) en la valuación inmobiliaria moderna, centrándose en la diada compuesta por *XGBoost* y *SHAP* (*SHapley Additive exPlanations*).

Ante el desafío histórico de la «caja negra» en los modelos automatizados de valuación (AVM), la obra analiza cómo fundamentar y descomponer predicciones complejas en contribuciones justas e interpretables mediante la teoría de juegos cooperativos.

A través de un análisis estructurado y una serie pedagógica de 50 tarjetas didácticas temáticas, el texto profundiza en los fundamentos teóricos de los valores de *Shapley*, optimizaciones algorítmicas de vanguardia como *TreeSHAP* y *Q-TreeSHAP*, y herramientas visuales clave (*waterfall, summary* y *dependence plots*).

Asimismo, se examina la aplicación práctica de estos métodos en el mercado inmobiliario (evaluando variables críticas como superficie, calidad y localización GIS), concluyendo con una revisión crítica sobre estabilidad estadística, sesgos, riesgos éticos y el rol insustituible del juicio pericial. En conjunto, proporciona una guía técnica esencial para transformar algoritmos opacos en sistemas de decisión transparentes, auditables y confiables.

**Contenido**

**Inteligencia Artificial Explicable (XAI) en la Valuación Inmobiliaria** ..... 6

**1. Resumen ejecutivo** ..... 6

**2. El problema de la caja negra** ..... 6

**3. *XGBoost*: Potencia predictiva y eficiencia** ..... 7

**4. SHAP: El fundamento de la equidad en la explicación** ..... 7

**5. *TreeSHAP*: Optimización para modelos de árboles** ..... 8

**6. *SHAP* Global vs. Local** ..... 8

**7. Flujo de aplicación en valuación inmobiliaria** ..... 9

**8. Ejemplo hipotético de explicación individual** ..... 9

**9. Limitaciones y consideraciones técnicas** ..... 10

**10. Implicaciones para el valuador profesional** ..... 10

**12. Conclusión** ..... 11

**13. Referencias** ..... 11

**Tarjetas didácticas** ..... 12

**Tarjeta 1** ..... 12

**Tarjeta 2** ..... 14

**Tarjeta 3** ..... 15

**Tarjeta 4** ..... 18

**Tarjeta 5** ..... 19

**Tarjeta 6** ..... 21

**Tarjeta 7** ..... 22

**Tarjeta 8** ..... 24

**Tarjeta 9** ..... 26

**Tarjeta 10** ..... 28

**Tarjeta 11** ..... 30

**Tarjeta 12** ..... 32

Tarjeta 13 ..... 34
Tarjeta 14 ..... 35
Tarjeta 15 ..... 37
Tarjeta 16 ..... 39
Tarjeta 17 ..... 42
Tarjeta 18 ..... 44
Tarjeta 19 ..... 45
Tarjeta 20 ..... 47
Tarjeta 21 ..... 49
Tarjeta 22 ..... 52
Tarjeta 23 ..... 53
Tarjeta 24 ..... 56
Tarjeta 25 ..... 58
Tarjeta 26 ..... 60
Tarjeta 27 ..... 61
Tarjeta 28 ..... 63
Tarjeta 29 ..... 65
Tarjeta 30 ..... 67
Tarjeta 31 ..... 68
Tarjeta 32 ..... 70
Tarjeta 33 ..... 72
Tarjeta 34 ..... 74
Tarjeta 35 ..... 76
Tarjeta 36 ..... 78
Tarjeta 37 ..... 80
Tarjeta 38 ..... 82
Tarjeta 39 ..... 85

**Tarjeta 40** ..... **87**

**Tarjeta 41** ..... **90**

**Tarjeta 42** ..... **92**

**Tarjeta 43** ..... **95**

**Tarjeta 44** ..... **96**

**Tarjeta 45** ..... **99**

**Tarjeta 46** ..... **101**

**Tarjeta 47** ..... **103**

**Tarjeta 48** ..... **106**

**Tarjeta 49** ..... **108**

**Tarjeta 50** ..... **111**

**Referencia bibliográficas** ..... **114**

Inteligencia Artificial Explicable (XAI) en la Valuación Inmobiliaria

Un Análisis Integral de SHAP y *XGBoost*

## 1. Resumen ejecutivo

La valuación inmobiliaria ha evolucionado de métodos manuales subjetivos hacia Modelos de Valuación Automatizados (AVM) basados en aprendizaje automático. Sin embargo, la adopción de modelos de alta precisión como *XGBoost* se ha visto limitada por su naturaleza de "caja negra". Este documento analiza la integración de XGBoost, un algoritmo de ensamble de árboles de decisión altamente eficiente, con SHAP (*SHapley Additive exPlanations*), una técnica basada en la teoría de juegos cooperativos para explicar predicciones individuales.

La Inteligencia Artificial Explicable (XAI) permite descomponer una predicción de precio en las contribuciones específicas de cada característica (ubicación, superficie, antigüedad), transformando modelos complejos en herramientas transparentes. Esta combinación es fundamental para la industria inmobiliaria, ya que no solo mejora la precisión predictiva (alcanzando coeficientes $R^2$ superiores a 0,87 en diversos estudios), sino que también garantiza el cumplimiento regulatorio, la rendición de cuentas y la confianza de los interesados al proporcionar el "por qué" detrás de cada valuación.

## 2. El problema de la caja negra

En la valuación inmobiliaria existe una tensión histórica entre la precisión predictiva y la interpretabilidad. Los modelos lineales tradicionales son fáciles de entender, pero no logran capturar las interacciones complejas y no lineales del mercado inmobiliario. Por otro lado, los modelos avanzados de aprendizaje profundo o ensambles como *XGBoost* ofrecen una precisión superior, pero funcionan como "cajas negras" donde el proceso de toma de decisiones es opaco.

Esta falta de transparencia representa un riesgo sistémico y ético. Instituciones regulatorias, como las de la Unión Europea bajo el Reglamento General de Protección de Datos (GDPR), exigen una "divulgación lógica significativa" para las decisiones automatizadas. En el contexto de los AVM, si un modelo predice un valor inusualmente bajo para una propiedad, el valuador debe poder explicar si esto se debe a la ubicación, al estado de conservación o a una anomalía en los datos. Sin XAI, la confianza de los bancos, inversores y propietarios se debilita, impidiendo que los modelos más potentes se utilicen en entornos de alta responsabilidad.

3. *XGBoost*: Potencia predictiva y eficiencia

*Extreme Gradient Boosting* (XGBoost) se ha consolidado como uno de los algoritmos más robustos para datos tabulares estructurados, superando frecuentemente a la regresión lineal y a los árboles de decisión simples. Sus características principales incluyen:

- *Gradient Boosting* (Potenciación del Gradiente): Es una técnica de ensamble que combina múltiples modelos "débiles" (generalmente árboles de decisión) para crear un modelo predictivo fuerte.
- Aprendizaje Secuencial: A diferencia del *bagging*, XGBoost entrena árboles de forma secuencial, donde cada nuevo árbol intenta corregir los errores residuales de los árboles anteriores.
- Relaciones No Lineales e Interacciones: El modelo es capaz de identificar interacciones complejas entre variables (por ejemplo, cómo el impacto de la superficie habitable varía dependiendo del código postal) que los modelos lineales ignorarían.
- Regularización: Incluye técnicas para evitar el sobreajuste (*overfitting*), lo que garantiza que el modelo se generalice bien a nuevas propiedades.
- Manejo de Datos Faltantes: *XGBoost* tiene capacidades nativas para manejar valores nulos de forma automática, lo cual es común en bases de datos inmobiliarias heterogéneas.

4. SHAP: El fundamento de la equidad en la explicación

*SHAP* es un método agnóstico al modelo para explicar predicciones individuales, basado en los Valores de *Shapley* de la teoría de juegos cooperativos (propuestos originalmente por *Lloyd Shapley* en 1953). En este marco, la predicción del modelo se considera la "ganancia" de un juego, y las características del inmueble (superficie, baños, ubicación) son los "jugadores" que colaboran para obtener esa ganancia.

*SHAP* distribuye la ganancia de manera justa entre las características basándose en su contribución marginal promedio a través de todas las combinaciones posibles de características. El modelo de explicación de *SHAP* se define matemáticamente como:

$$f(x) = E[f(X)] + \sum_{j=1}^{p} \phi_j$$

Donde:

- $f(x)$ es la predicción final del modelo para un inmueble específico.

- $E[f(X)]$ es el valor base, que representa la predicción promedio de todas las propiedades en el conjunto de datos (lo que el modelo predeciría si no conociera ninguna característica específica).
- $\phi_j$ es el valor SHAP para la característica $j$, representando cuánto aumentó o disminuyó esa variable el precio con respecto al promedio.

*SHAP* garantiza tres propiedades críticas:

1. Exactitud Local: La suma de las contribuciones de las características más el valor base siempre es igual a la predicción final.
2. Ausencia (*Missingness*): Una característica que no está presente recibe una atribución de cero.
3. Consistencia: Si una característica aumenta su contribución marginal en un modelo diferente, su valor *SHAP* no disminuirá.

5. *TreeSHAP*: Optimización para modelos de árboles

Aunque calcular los valores de *Shapley* exactos es computacionalmente costoso (exponencial en relación con el número de variables), *Lundberg et al*. (2020) introdujeron *TreeSHAP*. Esta es una variante optimizada específicamente para modelos basados en árboles como *XGBoost* y *Random Forest*.

*TreeSHAP* aprovecha la estructura interna de los árboles de decisión para calcular las contribuciones en tiempo polinomial ($O(TLD^2)$), donde $T$ es el número de árboles y $D$ la profundidad. Esto permite generar explicaciones precisas para miles de propiedades en segundos, algo que sería inviable con métodos agnósticos como *KernelSHAP*. *TreeSHAP* puede calcular valores "intervencionistas" (basados en la distribución marginal) o "dependientes de la ruta" (basados en la distribución condicional).

6. *SHAP* Global vs. Local

La versatilidad de SHAP permite analizar el modelo en dos niveles fundamentales:

| Aspecto | SHAP Local | SHAP Global |
|---|---|---|
| Objetivo | Explicar una predicción individual. | Entender el comportamiento general del modelo. |
| Unidad de análisis | Una sola propiedad. | Todo el conjunto de datos de entrenamiento/prueba. |

| Aspecto | SHAP Local | SHAP Global |
|---|---|---|
| Pregunta clave | ¿Por qué esta casa cuesta $500,000? | ¿Qué factores influyen más en el mercado? |
| Visualización | *Waterfall plots*, *Force plots*. | *Summary plots, Dependence plots.* |
| Utilidad para valuación | Justificar un avalúo ante un cliente o banco. | Identificar determinantes del mercado y sesgos. |

7. Flujo de aplicación en valuación inmobiliaria

Un proceso profesional de valuación utilizando estas herramientas sigue una estructura lógica de ingeniería de datos y ciencia de datos:

1. Datos Inmobiliarios: Recopilación de variables estructurales (superficie, cuartos), temporales (año de construcción, renovación) y geográficas (latitud, longitud, código postal).
2. Preparación: Limpieza de datos, eliminación de valores atípicos (*outliers*) mediante métodos como el rango intercuartílico (IQR), y codificación de variables categóricas.
3. XGBoost: Entrenamiento del modelo y optimización de hiperparámetros (tasa de aprendizaje, profundidad máxima, número de estimadores).
4. Validación: Evaluación de métricas estadísticas de error como el $R^2$, *MAE* (Error Absoluto Medio) y *RMSE* (Raíz del Error Cuadrático Medio).
5. *SHAP*: Aplicación de *TreeSHAP* sobre el modelo validado.
6. Explicación Global: Identificación de las variables con mayor impacto promedio (generalmente la superficie habitable y la calidad general).
7. Explicación Individual: Desglose del precio de una propiedad específica para ver cómo cada atributo movió la aguja desde el promedio.
8. Interpretación Profesional: Validación de las explicaciones por parte de un experto para asegurar que el modelo no esté utilizando correlaciones espurias.

8. Ejemplo hipotético de explicación individual

Imaginemos un modelo entrenado para una ciudad donde el precio promedio de una casa (valor base) es de $200.000. Se analiza una propiedad específica cuya predicción final es de $225.000. La explicación de *SHAP* revelaría el siguiente desglose:

- Superficie Habitable (120 m²): Como es menor al promedio del área, resta -$15.000.
- Año de Construcción (2015): Al ser una construcción moderna, suma +$30.000.

- Calidad de Acabados (Nivel 8/10): Superior al promedio, suma +$12.000.
- Cercanía a Transporte: Ubicación estratégica, suma +$5.000.
- Otras variables menores: Restan un neto de -$7.000.

Resultado: $200.000 (Base) - $15.000 + $30.000 + $12.000 + $5.000 – 7.000 = $225.000.

Este nivel de detalle permite al valuador comunicar con precisión por qué la casa tiene ese valor, a pesar de tener una superficie reducida.

9. Limitaciones y consideraciones técnicas

A pesar de su potencia, *SHAP* y *XGBoost* presentan desafíos que el profesional debe conocer:

- Correlación entre variables: Si dos variables están altamente correlacionadas (ej. superficie total y superficie habitable), *SHAP* puede dividir la importancia entre ambas, dificultando la interpretación de cuál es el verdadero motor del precio.
- Predicción vs. Causalidad: *SHAP* explica cómo el modelo usa los datos para predecir, pero no prueba causalidad. Que una variable tenga un valor *SHAP* alto no significa necesariamente que sea la causa del precio, sino que es un predictor estadísticamente fuerte.
- Dependencia del *Baseline*: Las explicaciones son siempre relativas al valor promedio del conjunto de datos utilizado. Un cambio en los datos de referencia cambiará las explicaciones individuales.
- Sesgo de los Datos: Si los datos de entrenamiento contienen sesgos históricos (ej. discriminación en ciertas zonas), el modelo los aprenderá y *SHAP* simplemente los hará visibles, no los corregirá.
- Incertidumbre: *SHAP* proporciona una estimación puntual de la contribución, pero no cuantifica la incertidumbre inherente a esa estimación.

10. Implicaciones para el valuador profesional

La adopción de XAI no reemplaza al valuador, sino que potencia su juicio profesional. *SHAP* contribuye en áreas críticas:

- Auditoría y Transparencia: Permite revisar si el modelo está tomando decisiones basadas en factores lógicos y aceptables en la práctica valuatoria.

- Identificación de Anomalías: Si una variable irrelevante (como el color de la puerta) aparece con una contribución alta, el valuador puede identificar errores en los datos o en el modelo.
- Soporte de Comunicación: Proporciona gráficos claros (como los de "fuerza" o "cascada") para explicar resultados complejos a clientes no técnicos, bancos o tribunales.
- Análisis de Sensibilidad: El valuador puede entender cómo pequeñas mejoras en un inmueble (ej. añadir un baño) impactarían la predicción del modelo.

12. Conclusión

¿Puede *SHAP* convertir un modelo *XGBoost* en una herramienta verdaderamente explicable? La respuesta es un sí matizado. *SHAP* logra abrir la "caja negra" proporcionando una base teórica sólida y matemáticamente justa para la atribución de características, permitiendo una transparencia sin precedentes en modelos no lineales de alta precisión. Sin embargo, la explicación técnica generada por *SHAP* es solo una parte de la ecuación.

La verdadera "explicabilidad" surge cuando estas métricas son interpretadas a la luz del juicio profesional del valuador, quien debe discernir entre correlaciones estadísticas y realidades del mercado. En última instancia, la combinación de la potencia de *XGBoost* y la claridad de *SHAP* crea un sistema de apoyo a la decisión que es a la vez preciso, auditable y confiable para el sector inmobiliario moderno.

13. Referencias


- *Leni, E. S., Revathi, T., & Devarajan, S*. (2025). *A Robust Ensemble-Based Framework for House Price Estimation: Integrating XG-Boost with SHAP and Web Deployment*. *ITM Web of Conferences*, 79, 01023.
- *Lundberg, S. M., & Lee, S.-I*. (2017). *A Unified Approach to Interpreting Model Predictions. Proceedings of the 31st International Conference on Neural Information Processing Systems* (NIPS 2017).
- *Molnar, C*. (2025). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*.
- *Chan, C.-P., Tsai, C.-H., Tang, F.-K., & Yang, J.-H*. (2025). *A SHAP-Based Comparative Analysis of Machine Learning Model Interpretability in Financial Classification Tasks*. *Journal of Applied Economic Sciences*, XX(3).
- *Chugani, V.* (2025). *A Gentle Introduction to SHAP for Tree-Based Models*. MachineLearningMastery.com.

Tarjetas didácticas

Tarjeta 1

En el anverso pone: "¿En qué teoría matemática fundamenta SHAP la atribución de importancia a las características de un modelo?" La respuesta del anverso es: "Se basa en la teoría de juegos cooperativos, utilizando los valores de Shapley formulados por Lloyd Shapley en 1953."

¿Podrías explicarme este tema más detalladamente?

La fundamentación de SHAP (*SHapley Additive exPlanations*) en la teoría de juegos cooperativos es el pilar que le otorga su rigor y consistencia matemática frente a otros métodos de explicabilidad.

1. La analogía: De la economía al *Machine Learning*

El matemático y premio Nobel *Lloyd Shapley* desarrolló en 1953 un método para resolver un problema clásico de la teoría de juegos: ¿cómo repartir equitativamente las ganancias (o pérdidas) entre un grupo de jugadores que colaboran en un equipo, cuando unos contribuyen más que otros?

En 2017, *Scott Lundberg* y *Su-In Lee* adaptaron este principio al *Machine Learning* mediante el marco *SHAP*:

- El juego (*Game*): Es el proceso de realizar la predicción de una observación específica (por ejemplo, estimar el precio de una vivienda concreta en 250.000 €).
- Los jugadores (*Players*): Son las variables o características de entrada del modelo (superficie, antigüedad, ubicación, número de habitaciones, etc.).
- La ganancia o pago (*Payout*): Es la diferencia entre la predicción de esa observación concreta (250.000 €) y el valor base o promedio general que predice el modelo para todo el conjunto de datos (por ejemplo, 200.000 €).

2. ¿Cómo se calcula? El concepto de Contribución Marginal

Para medir la importancia real de una variable (por ejemplo, tener un *garaje*), no basta con medir la predicción con o sin garaje de forma aislada, porque las variables interactúan entre sí.

El valor de *Shapley* calcula la contribución marginal de la variable probándola en todas las combinaciones posibles de variables (llamadas *coaliciones*):

1. Se evalúa cómo predice el modelo con un subconjunto de variables $S$ que no incluye al *garaje*.
2. Se evalúa la predicción añadiendo el *garaje* a ese mismo subconjunto: $S \cup \{\text{garaje}\}$.

3. La diferencia entre ambas predicciones es la contribución marginal del garaje en esa combinación específica.
4. El valor *SHAP* final es el promedio ponderado de esas contribuciones marginales a lo largo de todas las coaliciones posibles de variables.

3. Los 4 Axiomas Matemáticos (Por qué es una solución única)

La razón fundamental por la que se utiliza la teoría de *Shapley* es que demostró matemáticamente ser la única regla de distribución que satisface simultáneamente cuatro propiedades éticas y matemáticas fundamentales:

1. Eficiencia (*Local Accuracy*): La suma de los valores *SHAP* de todas las variables para una observación es exactamente igual a la diferencia entre la predicción final y el valor promedio del modelo. *(No queda impacto sin atribuir)*.
2. Simetría (*Symmetry*): Si dos variables contribuyen exactamente de la misma manera en todas las coaliciones posibles, ambas reciben exactamente el mismo valor *SHAP*.
3. Jugador Nulo / Ausencia (*Dummy / Missingness*): Si una variable no altera la predicción en ninguna combinación (es decir, no aporta información), su valor *SHAP* asignado es estrictamente cero.
4. Aditividad (*Additivity / Linealidad*): Permite calcular las explicaciones de modelos compuestos por la suma de submodelos (como un ensamble de árboles en *XGBoost*) sumando las contribuciones calculadas en cada árbol individual.

4. Del Valor de *Shapley* clásico a *SHAP*

Aunque el valor de *Shapley* proporciona una garantía matemática sólida, calcularlo exactamente requiere evaluar $2^M$ coaliciones (donde $M$ es el número de variables), lo que resulta impracticable cuando hay decenas de variables.

La contribución de *Lundberg* y *Lee* al crear *SHAP* fue conectar los valores de *Shapley* con modelos de explicación aditiva local y desarrollar algoritmos altamente eficientes como *TreeSHAP*. *TreeSHAP* aprovecha la estructura jerárquica de los árboles de decisión (como los de *XGBoost*) para reducir esa complejidad exponencial a un cálculo polinomial rápido, haciendo viable su aplicación en entornos reales como la valuación inmobiliaria o el análisis financiero.

Tarjeta 2

En el anverso pone: "¿Quiénes introdujeron el método *SHAP* (*SHapley Additive exPlanations*) en el campo del aprendizaje automático en 2017?" La respuesta del anverso es: "*Scott Lundberg y Su-In Lee*."

¿Podrías explicarme este tema más detalladamente?

La introducción del método *SHAP* (*SHapley Additive exPlanations*) por parte de Scott Lundberg y Su-In Lee en 2017 representó un hito fundamental en el campo de la Inteligencia Artificial Explicable (XAI).

1. El contexto histórico y la publicación en 2017

Antes de 2017, el concepto de los valores de *Shapley* ya existía en la teoría de juegos (desarrollado por *Lloyd Shapley* en 1953) y había tenido algunas adaptaciones iniciales para *Machine Learning* por autores como *Štrumbelj y Kononenko* (2011, 2014). Sin embargo, no se utilizaban ampliamente debido a su elevado costo computacional y a la falta de una teoría unificada.

En 2017, *Scott Lundberg y Su-In Lee* (de la Universidad de Washington) presentaron su célebre trabajo titulado *"A Unified Approach to Interpreting Model Predictions"* en la conferencia NIPS (actual NeurIPS). Este artículo transformó por completo el panorama de la interpretabilidad post-hoc.

2. La gran contribución: Unificación teórica

En aquel momento, existían múltiples métodos locales para intentar explicar las predicciones de modelos complejos ("cajas negras"), como *LIME*, *DeepLIFT* o métodos basados en gradientes. La gran innovación de *Lundberg y Lee* fue demostrativa y conceptual:

- Atribución Aditiva de Características: Identificaron que todos estos métodos aparentemente dispares pertenecían a una misma clase formal de modelos explicativos de representación lineal aditiva.
- La solución única: Demostraron matemáticamente que los valores de *Shapley* constituyen la única regla de distribución dentro de esta clase que satisface simultáneamente un conjunto de tres propiedades convenientes:
    1. Exactitud Local (*Local Accuracy*): La suma de las contribuciones de las características es exactamente igual a la diferencia entre la predicción individual y el valor promedio (*baseline*) del modelo.

2. Ausencia (*Missingness*): Si una variable está ausente o no tiene impacto en el modelo, su valor de atribución es cero.
3. Consistencia (*Consistency*): Si un modelo se modifica de modo que la contribución marginal de una variable aumenta o se mantiene, su valor de atribución asignado nunca disminuye.

3. Desarrollo algorítmico y herramientas prácticas

Además del marco teórico, *Lundberg y Lee* aportaron soluciones prácticas para calcular estas explicaciones en la práctica:

- *KernelSHAP*: Un método agnóstico al modelo que utiliza una regresión lineal ponderada con un núcleo específico (*SHAP kernel*) para estimar los valores de *Shapley* conectando conceptualmente con *LIME*.
- *TreeSHAP*: Desarrollado posteriormente por *Lundberg, Erion y Lee* (2020), este algoritmo aprovecha la estructura de árbol de modelos como *XGBoost*, *Random Forest* o *LightGBM*, reduciendo la complejidad computacional exponencial a un cálculo polinomial rápido.
- La librería *shap*: Crearon el paquete *open-source* en Python (shap), permitiendo pasar fácilmente de explicaciones individuales (locales) a análisis globales de todo un conjunto de datos (mediante *summary plots*, *beeswarm plots* y *dependence plots*).

4. Relevancia para la valuación inmobiliaria y sectores regulados

Gracias a esta unificación, modelos de alta precisión como *XGBoost* pasaron de ser vistos como "cajas negras" incomprensibles a herramientas transparentes e interpretables. En la valuación inmobiliaria y la evaluación financiera, el trabajo de *Lundberg y Lee* hizo posible auditar exactamente cuánto suma o resta cada variable (superficie, antigüedad, ubicación) en la estimación del valor de un inmueble individual.

Tarjeta 3

En el anverso pone: "¿Cuál es la representación matemática del modelo de explicación aditiva de *SHAP*?" La respuesta del anverso es:

$$g(z') = \phi_0 + \sum_{j=1}^{M} \phi_j z_j'$$

donde $g$ es el modelo de explicación, $z' \in \{0,1\}^M$ es el vector de coalición y $\phi_j$ representa el valor de *Shapley* de la característica $j$.

¿Podrías explicarme este tema más detalladamente?

Esta fórmula es la piedra angular de *SHAP* (*SHapley Additive exPlanations*). Representa la definición formal de un modelo de explicación aditiva local (*Additive Feature Attribution Method*), un marco desarrollado por *Scott Lundberg y Su-In Lee* (2017) para transformar cualquier modelo predictivo complejo (como una «caja negra» de *XGBoost*) en una representación lineal e interpretable para una observación específica.

$$g(z') = \phi_0 + \sum_{j=1}^{M} \phi_j \, z_j'$$

1. Desglose término por término

Para comprender la fórmula, analicemos qué representa cada uno de sus elementos:

- $g(z')$ — El modelo explicativo local: No es el modelo predictivo original (como *XGBoost*), sino un modelo lineal simple diseñado exclusivamente para aproximar y explicar el comportamiento del modelo complejo en el entorno de una predicción concreta.
- $\mathbf{z}' \in \{0,1\}^M$ — El vector de coalición (o características simplificadas): Es un vector binario de longitud $M$ que indica la presencia (1) o ausencia (0) de cada característica en una combinación o juego cooperativo de variables.
    - $z_j' = 1$: La característica $j$ está presente (se utiliza su valor real observado).
    - $z_j' = 0$: La característica $j$ se considera "ausente" o reemplazada por valores de referencia de la población.
- M — El número total de características (o el tamaño máximo de la coalición de variables).
- $\phi_0$— El valor base (o valor esperado): Representa el resultado o predicción promedio que generaría el modelo si no tuviera información sobre ninguna característica en particular. Matemáticamente equivale a la expectativa general del modelo sobre todo el conjunto de datos: $\phi_0 = E[f(X)]$.
- $\phi_j$— El valor *SHAP* de la característica $j$: Es la contribución aditiva o el impacto numérico atribuido a la variable $j$ (expresado en las mismas unidades de la variable objetivo, como euros, dólares o probabilidad). Indica cuánto incrementa o reduce esa variable la predicción respecto al valor base $\phi_0$.

2. La simplificación para una predicción real ($x$)

Cuando deseamos explicar una observación real y concreta $x$ (por ejemplo, una vivienda específica que queremos valorar), todas las variables están presentes. Por lo tanto, el vector de coalición para esa instancia es $x' = (1,1, \dots ,1)^T$, donde $z_j' = 1$ para todo $j$. En este caso, la fórmula se simplifica a:

$$f(x) = g(x') = \phi_0 + \sum_{j=1}^{M} \phi_j$$

Esta propiedad se conoce como Exactitud Local (*Local Accuracy*) o Eficiencia: la predicción final exacta del modelo $f(x)$ es siempre igual a la suma del valor base promedio $\phi_0$ más todas las contribuciones individuales de Shapley ($\phi_j$).

3. Ejemplo práctico aplicado a la Valuación Inmobiliaria

Imaginemos que un modelo *XGBoost* estima el precio de un inmueble específico en 242.000 €:

1. Valor base ($\phi_0$): 200.000 € *(*el precio promedio de todas las viviendas en el conjunto de datos).
2. Aportación por Superficie ($\phi_{superficie}$): +35.000 € (posee más metros cuadrados que la media).
3. Aportación por Ubicación ($\phi_{ubicacion}$): +20.000 € (zona de alta demanda).
4. Aportación por Garaje ($\phi_{garaje}$): +12.000 € *(*dispone de plaza de garaje*)*.
5. Penalización por Antigüedad ($\phi_{edad}$): -25.000 € (la edificación es antigua).

Sustituyendo estos valores en la ecuación aditiva:

$$g(x') = 200.000 + 35.000 + 20.000 + 12.000 - 25.000 = 242.000\ €$$

La suma aditiva reconstruye exactamente la estimación del modelo de 242.000 €, descomponiendo la "caja negra" en una explicación clara e intuitiva para el tasador o cliente.

4. ¿Por qué es importante esta representación?

La relevancia de esta fórmula radica en que unifica teóricamente diferentes métodos de explicabilidad post-hoc (como *LIME*, *DeepLIFT* o métodos de gradiente). Demuestra que todos ellos intentan construir un modelo lineal simplificado $g(z')$, pero *SHAP* demuestra que la asignación basada en los valores de Shapley ($\phi_j$) es la única solución matemática que garantiza propiedades esenciales como la consistencia y la exactitud local sin distorsionar la atribución.

Tarjeta 4

En el anverso pone: "En la fórmula de explicación aditiva de *SHAP*, ¿qué representa el término $\phi_0$?" La respuesta del anverso es: "Representa el valor base o la predicción promedio esperada del modelo sobre todo el conjunto de datos ($E[\hat{f}(X)]$).

¿Podrías explicarme este tema más detalladamente?

En el modelo de explicación aditiva de **SHAP** ($g(z') = \phi_0 + \sum_{j=1}^{M} \phi_j z_j'$), el término $\phi_0$ (denominado *base value* o *expected value*) es la piedra angular sobre la que se construyen todas las explicaciones individuales.

1. Definición formal e intuitiva

- Concepto intuitivo: Es la predicción "a ciegas" o neutral del modelo; es decir, el valor que pronosticaría si no dispusiera de información sobre ninguna de las características o variables específicas de una observación concreta.
- Definición matemática: Se define formalmente como el valor esperado o el promedio poblacional de las predicciones emitidas por el modelo sobre el conjunto de datos de entrenamiento o de fondo (*background dataset*): $\phi_0 = E[\hat{f}(X)]$.

2. ¿Cómo funciona $\phi_0$ dentro de la ecuación?

*SHAP* descompone cada predicción individual en la suma de una línea base más las contribuciones específicas de cada variable. Cuando evaluamos una instancia real específica $x$ donde todas sus características están presentes (haciendo $z_j' = 1$), el principio de Exactitud Local (*Local Accuracy*) garantiza que la estimación exacta del modelo $f(x)$ equivale a la suma de $\phi_0$ y los valores *SHAP* ($\phi_j$) de cada variable:

$$f(x) = \phi_0 + \sum_{j=1}^{M} \phi_j$$

Visualmente (como en los gráficos de fuerza o de cascada), $\phi_0$ actúa como el punto de anclaje inicial. A partir de este valor promedio, cada característica ejerce una "fuerza" que empuja el resultado final hacia arriba (contribución positiva, $\phi_j > 0$) o hacia abajo (contribución negativa, $\phi_j < 0$).

3. Ejemplo práctico en Valuación Inmobiliaria (AVM)

Imagina un modelo *XGBoost* entrenado para valorar inmuebles en una ciudad:

1. Valor Base ($\phi_0$): Tras evaluar todo el mercado, el modelo calcula que el precio medio de una vivienda es de 200.000 €.
2. Propiedad específica: Evaluamos una casa concreta cuya predicción final es de 242.000 €.
3. Desglose *SHAP*:
    - Superficie (120 m²): Amplia → $\phi_{\text{superficie}} = +35.000$ €.
    - Ubicación: Zona céntrica → $\phi_{\text{ubicación}} = +20.000$ €
    - Antigüedad: 40 años → $\phi_{\text{edad}} = -13.000$ €.

Sustituyendo en la fórmula:

$$f(x) = 200.000\,€\ (\phi_0) + 35.000\,€ + 20.000\,€ - 13.000\,€ = 242.000\,€$$

Sin el término $\phi_0 = 200.000$ €, los +35.000 € o 13.000 € no tendrían punto de referencia; $\phi_0$ aclara exactamente desde qué promedio de mercado la vivienda gana o pierde valor.

4. Una advertencia técnica: La sensibilidad al conjunto de datos de fondo

Debido a que $\phi_0$ se calcula como el promedio esperado $E\left[\hat{f}(X)\right]$, su valor exacto depende del conjunto de datos utilizado como referencia (*background dataset*). Si se cambia ese conjunto de referencia (por ejemplo, evaluando únicamente viviendas de lujo en lugar de todo el mercado general), el valor base $\phi_0$ subirá, ajustando en consecuencia las contribuciones relativas $\phi_j$ de cada inmueble.

Tarjeta 5

En el anverso pone: "¿En qué consiste la propiedad de Exactitud Local (*Local Accuracy*) de *SHAP*?" La respuesta del anverso es: "Garantiza que la suma de los valores de *Shapley* de todas las características más el valor base sea exactamente igual a la predicción final de la instancia:

$$\hat{f}(x) = \phi_0 + \sum_{j=1}^{M} \phi_j$$

¿Podrías explicarme este tema más detalladamente?

La propiedad de Exactitud Local (*Local Accuracy*) es una de las garantías matemáticas más potentes de *SHAP*. En esencia, asegura que las explicaciones generadas no son aproximaciones burdas ni estimaciones con "margen de error", sino una reconstrucción aditiva exacta de la predicción emitida por el modelo.

1. El origen: Del Axioma de Eficiencia a la Exactitud Local

En la teoría clásica de juegos cooperativos propuesta por *Lloyd Shapley* en 1953, esta propiedad se conoce como el Axioma de Eficiencia. Establece que el total del beneficio o pago generado por una coalición de jugadores debe distribuirse por completo entre ellos, sin que quede nada sobrante ni faltante.

Cuando *Scott Lundberg y Su-In Lee* (2017) adaptaron los valores de *Shapley* al aprendizaje automático en el marco *SHAP*, renombraron esta propiedad como Exactitud Local:

$$\hat{f}(x) = \phi_0 + \sum_{j=1}^{M} \phi_j$$

- $\hat{f}(x)$: La predicción numérica exacta emitida por el modelo para la observación específica $x$.
- $\phi_0$: El valor base o expectativa promedio del modelo en todo el conjunto de datos ($E[\hat{f}(X)]$).
- $\phi_j$: El valor SHAP (contribución aditiva) asignado a la característica $j$.
- $M$: El número total de características.

2. ¿Qué significa esto en la práctica?

1. Suma Cero Residuos (Fidelidad Absoluta): La suma del valor promedio ($\phi_0$) más las atribuciones individuales ($\phi_j$) equivale exactamente al 100% de la predicción. No hay "residuos", "pérdidas" ni diferencias no explicadas entre la caja negra y la explicación.
2. Contraste con otros métodos: Métodos como *LIME* aproximan el modelo complejo entrenando un sustituto lineal local. *LIME* puede tener un error cuadrático o margen de desviación respecto a la predicción real de la caja negra. *SHAP*, en cambio, garantiza exactitud local matemática.

3. Ejemplo en Valuación Inmobiliaria (AVM)

Supongamos que un modelo *XGBoost* predice el precio de un departamento concreto en 250.000 €:

- Valor Base del mercado ($\phi_0$): 200.000 € (promedio de la base de datos).
- Diferencia total a explicar: +50.000 €.

Las atribuciones *SHAP* desglosan esa diferencia exacta:

- Superficie (110 m²): $\phi_{\text{superficie}} = +30.000€$
- Ubicación (Centro): $\phi_{\text{ubicación}} = +25.000€$

- Garaje (Sí): $\phi_{\text{garaje}} = +10.000€$
- Antigüedad (35 años): $\phi_{\text{edad}} = -15.000€$

Aplicando la propiedad de Exactitud Local:

$$\text{Predicción} = 200.000 + 30.000 + 25.000 + 10.000 - 15.000 = 250.000\,€$$

La suma encaja al centavo. Esto es lo que permite graficar la explicación en un gráfico de cascada (*waterfall plot*) o de fuerzas (*force plot*), donde cada barra empuja el valor exactamente desde la línea base hasta el resultado final.

4. Importancia para la Auditoría y el Sector Inmobiliario

Gracias a la Exactitud Local, un valuador o auditor puede justificar ante un regulador o cliente el impacto económico exacto de cada variable: no se trata de una "estimación aproximada de importancia", sino de la cuantificación exacta de cuánto sumó o restó cada atributo en el valor de ese inmueble específico.

Tarjeta 6

En el anverso pone: "¿Qué establece la propiedad de Ausencia (*Missingness*) en *SHAP*?" La respuesta del anverso es: "Establece que si una característica está ausente en la coalición simplificada ($x_j' = 0$), su valor de atribución atribuido debe ser cero ($\phi_0$ = 0)."

¿Podrías explicarme este tema más detalladamente?

La propiedad de Ausencia (*Missingness*) es una de las tres propiedades deseables definidas por *Scott Lundberg y Su-In Lee* en 2017 para estructurar el marco de *SHAP*.

1. Definición formal y contexto

En el marco de *SHAP*, las explicaciones locales se representan mediante un modelo lineal simplificado:

$$g(z') = \phi_0 + \sum_{j=1}^{M} \phi_j \, z_j'$$

Donde $z' = (z_1', \ldots, z_M') \in \{0,1\}^M$ es el vector de coalición (o características simplificadas):

- $z_j' = 1$: Indica que la característica $j$ está presente en la combinación.
- $z_j' = 0$: Indica que la característica $j$ está ausente.

La propiedad de Ausencia establece formalmente que:

$$x_j' = 0 \Longrightarrow \phi_j = 0$$

Esto significa que si una característica está marcada como ausente en la coalición simplificada ($x_j' = 0$), el valor de atribución o valor *SHAP* asignado a dicha característica debe ser exactamente cero ($\phi_j = 0$).

2. ¿Por qué es necesaria? El motivo de «contabilidad matemática»

Esta regla no formaba parte de los axiomas tradicionales formulados por *Lloyd Shapley* en 1953 para la teoría de juegos cooperativos, sino que fue incorporada específicamente para el desarrollo de *SHAP*.

El propio *Scott Lundberg* la describe como una «propiedad menor de contabilidad» (*minor book-keeping property*)2:

1. Evitar valores indefinidos: En la suma del modelo lineal $g(z') = \phi_0 + \sum \phi_j z_j'$, si una variable no está presente ($x_j' = 0$), el producto $\phi_j \cdot 0$ resulta en cero automáticamente. Sin esta propiedad, la regla de Exactitud Local se cumpliría numéricamente incluso si a esa característica ausente se le asignara un valor *SHAP* arbitrario (como $\phi_j = 100$).
2. Forzar la coherencia: Para prevenir que variables no presentes en una coalición simplificada tengan valores no nulos indefinidos, la propiedad de Ausencia exige explícitamente que su contribución sea estrictamente 0.

3. Interpretación práctica

- En explicaciones de instancias reales: Cuando explicamos una observación concreta (como la estimación del precio de una vivienda $x$), todas sus características reales están presentes, por lo que el vector asigna $x_j' = 1$ a cada una de ellas.
- En el cálculo algorítmico: Durante la evaluación interna de subconjuntos de variables (por ejemplo, al simular coaliciones en *KernelSHAP*) o al trabajar con variables que son constantes en el conjunto de datos, la regla de Ausencia garantiza que las características excluidas en una combinación específica no absorban ni aporten impacto explicativo.

Tarjeta 7

En el anverso pone: "¿Cómo se define la propiedad de Consistencia (*Consistency*) en *SHAP*?" La respuesta del anverso es: "Si un modelo se modifica de tal manera que la contribución marginal de una característica aumenta o se mantiene para todas las coaliciones, su valor de *Shapley* asignado no disminuirá."

¿Podrías explicarme este tema más detalladamente?

La propiedad de Consistencia (*Consistency* o Monotonicidad) es considerada por *Scott Lundberg y Su-In Lee* como la propiedad más importante del marco *SHAP*, ya que es la que garantiza la confiabilidad de las explicaciones cuando los modelos cambian.

1. Definición intuitiva

En términos sencillos, la Consistencia asegura que si un modelo se modifica de modo que el impacto de una variable aumenta o se mantiene igual en todas las combinaciones posibles, la atribución de importancia (valor *SHAP*) asignada a esa variable nunca disminuirá.

Es un principio de equidad e intuición lógica: si modificas un algoritmo para que dependa más de la variable «Superficie», la importancia atribuida a la «Superficie» debe aumentar o, al menos, mantenerse; jamás debería bajar.

2. Formulación matemática formal

Matemáticamente, supongamos dos modelos (o versiones de un modelo),$\hat{f}$ y $\hat{f}'$, y una observación $x$ que deseamos explicar:

Sea $z' \in \{0,1\}^M$ un vector de coalición (donde 1 representa variables presentes y 0 ausentes), y $z'_{-j}$ representa esa misma coalición, pero con la variable $j$ desactivada ($z_j' = 0$).

Si para todas las coaliciones posibles $z'$ se cumple que la contribución marginal de la variable $j$ en el modelo nuevo es mayor o igual que en el modelo anterior:

$$\hat{f}'(z') - \hat{f}'(z'_{-j}) \geq \hat{f}(z') - \hat{f}(z'_{-j})$$

Entonces, el valor *SHAP* de la variable *j* en el modelo nuevo debe ser mayor o igual que en el anterior:

$$\phi_j(\hat{f}', x) \geq \phi_j(\hat{f}, x)$$

3. ¿Por qué es tan crucial? El problema con la importancia tradicional

Antes de la llegada de *SHAP* en 2017, la forma estándar de medir la importancia de las variables en modelos basados en árboles (como *XGBoost* o *Random Forest*) era mediante la Ganancia (*Gain*) o la reducción de impureza.

*Lundberg y Lee* demostraron que las métricas tradicionales de *Feature Importance* violan sistemáticamente la Consistencia. Ocurría la paradoja de que, si un científico de datos modificaba un árbol para darle intencionalmente más peso a una variable clave, la métrica de *Gain* de esa variable podía descender de manera contraintuitiva.

*SHAP* soluciona este fallo estructural: al estar fundamentado en los valores de *Shapley*, garantiza Consistencia matemática.

4. Relación con los demás axiomas de *Shapley*

En el suplemento matemático de su trabajo de 2017, *Lundberg y Lee* demostraron que las propiedades clásicas de *Shapley* (Simetría, Jugador Nulo / *Dummy* y Aditividad / Linealidad) se derivan matemáticamente de la combinación de la Exactitud Local y la Consistencia. Esto convierte a la Consistencia en el pilar fundamental que sostiene todo el esquema axiomático de *SHAP*.

5. Ejemplo en Valuación Inmobiliaria

Imaginemos un modelo predictivo de valoración inmobiliaria (AVM) entrenado con *XGBoost*:

1. En la Versión 1 del modelo ($\hat{f}$), disponer de Piscina incrementa la estimación en +10.000 € respecto a cualquier combinación de variables.
2. Tras un año de alta demanda estival, reentrenamos el modelo a la Versión 2 ($\hat{f}'$), donde el mercado valora más este atributo y Piscina pasa a sumar +18.000 € en todas las combinaciones.

La propiedad de Consistencia garantiza que el valor *SHAP* asignado a la piscina en la Versión 2 ($\phi_{\text{piscina}}(\hat{f}', x)$) será estrictamente mayor o igual que en la Versión 1 ($\phi_{\text{piscina}}(\hat{f}, x)$). Un valuador o auditor puede confiar en que la explicación reflejará fielmente el incremento de importancia asignado por el algoritmo.

Tarjeta 8

En el anverso pone: "¿Qué novedad clave aportó *SHAP* con respecto al uso clásico de los valores de *Shapley*?" La respuesta del anverso es: "Unificó los valores de *Shapley* con otros métodos de atribución post-hoc (como *LIME*) mediante un marco aditivo lineal e introdujo métodos eficientes como *TreeSHAP*."

¿Podrías explicarme este tema más detalladamente?

La aportación de *SHAP* (*SHapley Additive exPlanations*), introducido por *Scott Lundberg y Su-In Lee* en 2017, transformó los valores de *Shapley* de un concepto teórico complejo en la herramienta estándar de la Inteligencia Artificial Explicable (XAI).

Aunque *Lloyd Shapley* formuló su teoría en 1953 para la economía, y autores como *Štrumbelj* y *Kononenko* propusieron su uso en *machine learning* en 2011 y 2014, el método apenas se utilizaba en la práctica por su costo computacional exponencial ($O(2^M)$) y la falta de una teoría unificada.

Las tres novedades clave que introdujo *SHAP* para resolver estas limitaciones se desglosan a continuación:

1. Marco Unificado: Modelos de Atribución Aditiva

*Lundberg y Lee* demostraron que diversos métodos post-hoc aparentemente desconectados (como *LIME*, *DeepLIFT*, o coeficientes de regresiones lineales) pertenecen a una misma clase formal llamada modelos de atribución aditiva de características.

Bajo este marco, la explicación del modelo complejo se representa como un modelo sustituto lineal simple:

$$g(z') = \phi_0 + \sum_{j=1}^{M} \phi_j \, z_j'$$

Además, demostraron matemáticamente que los valores de *Shapley* son la única solución posible dentro de esta clase que satisface simultáneamente las propiedades de Exactitud Local (Local *Accuracy*), Ausencia (*Missingness*) y Consistencia (Consistency). Esto otorgó un respaldo axiomático riguroso a explicaciones aditivas que antes eran puramente heurísticas (como *LIME*).

2. Eficiencia Algorítmica: Nacimiento de *TreeSHAP*

Calcular valores de *Shapley* exactos de manera agnóstica al modelo mediante perturbaciones (*KernelSHAP*) requiere evaluar hasta $2^M$ combinaciones de variables, lo que resulta impracticable en datasets reales.

*Lundberg, Erion y Lee* desarrollaron *TreeSHAP*, un algoritmo específico para modelos basados en árboles (como *XGBoost*, *Random Forest* o *LightGBM*). *TreeSHAP* aprovecha la estructura jerárquica de los árboles de decisión para rastrear los caminos de división, reduciendo la complejidad computacional exponencial a una complejidad polinomial $O(TLD^2)$(donde $T$ es el número de árboles, $L$ los nodos hoja y $D$ la profundidad máxima). Esto aceleró los cálculos en varios órdenes de magnitud, haciendo viable explicar millones de datos en segundos.

3. De Explicaciones Locales a Visualizaciones Globales

Antes de *SHAP*, los valores de *Shapley* se calculaban de forma aislada para observaciones individuales (explicación local). Gracias a la velocidad de *TreeSHAP*, *SHAP* introdujo la capacidad de calcular matrices completas de valores de *Shapley* para todo un conjunto de datos y agregarlas para el análisis global:

- Importancia global de variables (Mean |SHAP|): Mide el impacto promedio absoluto de cada variable en la predicción.
- Gráficos de resumen (*Summary / Beeswarm Plots*): Muestran simultáneamente la importancia de las variables, la magnitud de su impacto y la dirección de la relación (valores altos o bajos de la característica).
- Valores de Interacción *SHAP*: Permiten aislar los efectos puros de interacción entre pares de variables.

4. Ejemplo en Valuación Inmobiliaria

En un Modelo de Valoración Automatizada (AVM) basado en *XGBoost*, esta unificación aditiva permite descomponer una estimación opaca de 242.000 € en un modelo lineal interpretable:

$$\text{Precio Predicho} = \text{Valor Base } (\phi_0: 200.000\,€) + \text{Superficie } (+35.000\,€) + \text{Ubicación } (+25.000\,€) - \text{Antigüedad } (-18.000\,€)$$

Gracias a las innovaciones de *Lundberg y Lee*, el valuador o auditor obtiene una justificación matemática exacta y consistente sin perder la potencia predictiva de *XGBoost*.

Tarjeta 9

En el anverso pone: "¿Qué es *KernelSHAP*?" La respuesta del anverso es: "Es un método de estimación de *SHAP* agnóstico al modelo que aproxima los valores de *Shapley* ajustando un modelo de regresión lineal ponderado mediante el núcleo *SHAP*."

¿Podrías explicarme este tema más detalladamente?

*KernelSHAP* es el método de estimación de valores de *Shapley* diseñado por *Scott Lundberg y Su-In Lee* (2017) para ser completamente agnóstico al modelo (*model-agnostic*).

A diferencia de algoritmos específicos como *TreeSHAP* (que aprovechan la estructura interna de los árboles de decisión), *KernelSHAP* trata al modelo predictivo como una caja negra, requiriendo únicamente evaluar las entradas y las salidas del algoritmo.

1. ¿Cómo funciona *KernelSHAP*? El proceso en 5 pasos

*KernelSHAP* combina la teoría de los valores de *Shapley* con la filosofía de *LIME* (explicaciones locales interpretables), resolviendo un problema de optimización mediante una regresión lineal ponderada:

1. Muestreo de coaliciones binarias ($z' \in \{0,1\}^M$): Se generan combinaciones sintéticas en un espacio binario simplificado donde 1 indica que una variable está "presente" y 0 que está "ausente".
2. Mapeo al espacio original ($h_x(z')$): Como un modelo de machine learning no puede procesar un 0 ("variable ausente") en un dataset tabular de forma directa, *KernelSHAP* reemplaza las variables marcadas con 0 con valores tomados al azar de un conjunto de datos de fondo (*background dataset*).
3. Obtención de predicciones: El modelo evalúa estas instancias sintéticas creadas.
4. Ponderación con el Núcleo *SHAP* (*SHAP Kernel*): Cada combinación recibe un peso calculado mediante la fórmula del núcleo *SHAP*:

$$\pi_x(z') = \frac{M-1}{\binom{M}{|z'|}\,|z'|(M-|z'|)}$$

   A diferencia de *LIME* (que pondera por cercanía geográfica), el *SHAP Kernel* otorga los pesos más altos a las coaliciones muy pequeñas (pocas variables en 1) y a las coaliciones muy grandes (casi todas en 1). Esto se debe a que se aprende más del efecto aislado o total de una variable cuando se observa sola o cuando se retira sola del grupo.
5. Ajuste de la regresión lineal: Se entrena un modelo de regresión lineal ponderado para minimizar el error cuadrático medio entre las predicciones sintéticas y el modelo sustituto. Los coeficientes resultantes ($\phi_j$) de esta regresión son los valores de *Shapley* estimados.

2. Ventajas y Limitaciones de *KernelSHAP*

- Ventajas:
  - Universalidad: Funciona con cualquier arquitectura de *machine learning* (redes neuronales profundas, SVM, regresiones complejas, ensambles o modelos híbridos) y cualquier tipo de dato (tabulares, imágenes o texto).
  - Fundamentación sólida: Garantiza que las explicaciones locales sigan los principios de equidad de *Shapley* al usar el núcleo adecuado.
- Limitaciones:
  - Costo computacional elevado: Al requerir múltiples evaluaciones y muestreos del modelo para cada observación, es significativamente más lento que *TreeSHAP*.

- Supuesto de independencia (Muestreo Marginal): Al reemplazar las variables "ausentes" por valores aleatorios del conjunto de datos de fondo, puede generar combinaciones de datos poco realistas si existen variables fuertemente correlacionadas.

3. Ejemplo práctico en Valuación Inmobiliaria

Supongamos que entrenaste una Red Neuronal Profunda (un modelo agnóstico donde no se puede aplicar *TreeSHAP*) para estimar precios de viviendas:

- Para explicar por qué una casa específica se valoró en 300.000 €, *KernelSHAP* crea miles de combinaciones binarias (ej. [Superficie=1, Garaje=0, Ubicación=1, Edad=0]).
- Sustituye el "Garaje" y la "Edad" ausentes con los valores de viviendas tomadas al azar del dataset general.
- Evalúa la Red Neuronal con esos datos sintéticos, pondera los resultados usando el núcleo *SHAP* y ajusta la regresión lineal.
- Las pendientes del modelo lineal dirán exactamente cuánto aporta cada variable (ej. Superficie = +40.000 €, Edad = -15.000 €) a partir del valor base.

Tarjeta 10

En el anverso pone: "¿Cuál es la fórmula del núcleo de ponderación $\pi_x(z')$ utilizado en KernelSHAP?" La respuesta del anverso es:

$$\pi_x(z') = \frac{M-1}{\binom{M}{|z'|}\,|z'|(M-|z'|)}$$

donde $M$ es el tamaño máximo de coalición y $|z'|$ es el número de características presentes."

¿Podrías explicarme este tema más detalladamente?

La fórmula del núcleo de ponderación SHAP (*SHAP kernel*) es el elemento matemático que conecta el método *KernelSHAP* con la teoría de juegos cooperativos.

$$\pi_x(z') = \frac{M-1}{\binom{M}{|z'|}\,|z'|(M-|z'|)}$$

A diferencia de *LIME* (que asigna pesos en función de la proximidad física o distancia euclidiana entre instancias), *Lundberg y Lee* diseñaron esta función de ponderación específica para garantizar que la regresión lineal ponderada resultante devuelva exactamente los valores de *Shapley*.

1. Desglose de los componentes de la fórmula

- $M$: El tamaño máximo de la coalición, equivalente al número total de características o variables del modelo.
- $|z'|$: El número de características presentes en la coalición simplificada $z'$ (es decir, la cantidad de valores 1 en el vector binario de la coalición).
- $M - |z'|$: El número de características ausentes en la coalición (la cantidad de valores 0).
- $\binom{M}{|z'|}$: El coeficiente binomial, que representa el número total de combinaciones o coaliciones posibles que tienen exactamente el tamaño $|z'|$.
- $M - 1$: Factor constante de normalización en el numerador.

2. Intuición: ¿Por qué las coaliciones extremas reciben los mayores pesos?

El comportamiento más sobresaliente de esta fórmula es que asigna los pesos más altos a las coaliciones extremadamente pequeñas (pocas variables en 1) y a las coaliciones extremadamente grandes (casi todas las variables en 1). Las coaliciones de tamaño intermedio (alrededor de $|z'| \approx M/2$) reciben pesos muy bajos.

La razón detrás de esta distribución de pesos es profunda:

1. Coaliciones de tamaño 1 ($|z'| = 1$): Permiten aislar y medir el efecto principal e individual de una variable cuando juega sola, sin interferencia de otras características.
2. Coaliciones de tamaño $M - 1$ ($|z'| = M - 1$): Permiten medir el efecto total de una variable (su efecto principal más todas sus interacciones con las demás), evaluando qué sucede cuando es la única variable que se retira del grupo completo.
3. Coaliciones intermedias ($|z'| \approx M/2$): Aportan poca información sobre la contribución individual de una variable específica, ya que hay una enorme cantidad de combinaciones de tamaño medio y los efectos de las variables se mezclan. Además, el coeficiente binomial $\binom{M}{|z'|}$ en el denominador alcanza su valor máximo en $M/2$, lo que reduce drásticamente el peso asignado a cada coalición individual de ese tamaño.

3. Ejemplo numérico de la ponderación

Si evaluamos un modelo con $M$ = 5 características:

- Para una coalición pequeña ($|z'| = 1$):

$$\pi_x(z') = \frac{5-1}{\binom{5}{1} \cdot 1 \cdot (5-1)} = \frac{4}{5 \cdot 1 \cdot 4} = \frac{1}{5} = 0{,}20$$

- Para una coalición intermedia ($|z'| = 2$):

$$\pi_x(z') = \frac{5-1}{\binom{5}{2} \cdot 2 \cdot (5-2)} = \frac{4}{10 \cdot 2 \cdot 3} = \frac{4}{60} \approx 0{,}066$$

El peso por combinación baja de 0,20 a 0,066 al pasar de tamaño 1 a tamaño 2

4. Importancia práctica para el algoritmo *KernelSHAP*

Debido a que las coaliciones de tamaño 1 y $M-1$ concentran la mayor parte de la masa de ponderación, las implementaciones de KernelSHAP no muestrean coaliciones de forma puramente aleatoria:

1. Priorización de extremos: Utilizan el presupuesto de muestreo para evaluar primero de manera explícita todas las combinaciones de 1 elemento y $M-1$ elementos.

2. Muestreo del resto: Solo si queda presupuesto adicional de evaluación, se muestrean aleatoriamente coaliciones de tamaños intermedios ajustando sus pesos relativos.

Tarjeta 11

En el anverso pone: "¿Cuál es una de las principales desventajas de *KernelSHAP* al evaluar conjuntos de datos complejos?" La respuesta del anverso es: "Es computacionalmente lento e ignora la dependencia entre características al muestrear valores de la distribución marginal, lo que puede generar instancias poco probables."

¿Podrías explicarme este tema más detalladamente?

La tarjeta aborda los dos cuellos de botella fundamentales de *KernelSHAP* al enfrentarse a conjuntos de datos tabulares complejos o de alta dimensionalidad.

1. Desventaja 1: Elevada lentitud computacional

*KernelSHAP* es un método agnóstico al modelo (*model-agnostic*), lo que significa que trata al algoritmo predictivo como una caja negra y solo observa sus entradas y salidas.

- Causa técnica: Para calcular los valores de *Shapley* de una sola observación, *KernelSHAP* debe generar miles de combinaciones binarias sintéticas (coaliciones $z' \in \{0,1\}^M$), evaluar el modelo de *Machine Learning* para cada una de ellas y luego resolver una regresión lineal ponderada.

- Escalabilidad: En conjuntos de datos complejos con muchas variables (alta dimensionalidad $M$), la cantidad de combinaciones posibles escala de forma exponencial ($2^M$). Evaluar repetidamente la función de predicción para miles de instancias sintéticas vuelve a *KernelSHAP* extremadamente lento e impracticable para *datasets* grandes, a diferencia de algoritmos optimizados como *TreeSHAP*, que reducen este cálculo a tiempo polinomial aprovechando la estructura interna de los árboles.

2. Ignorar la dependencia entre características (Muestreo Marginal)

Para simular qué ocurre cuando una característica está "ausente" en una coalición (marcada con un 0), *KernelSHAP* reemplaza el valor real de esa variable por un valor tomado al azar de un conjunto de datos de fondo (*background dataset*).

- El supuesto de independencia: Al hacer este reemplazo aleatorio, *KernelSHAP* asume implícitamente que las variables son independientes entre sí e integra las predicciones sobre la distribución marginal:

$$E_{X_{-j}}\left[\hat{f}\left(x_j, X_{-j}\right)\right]$$

- Generación de instancias irrealistas (*out-of-distribution*): En la vida real, las variables de datasets complejos están fuertemente correlacionadas o dependen unas de otras. Al ignorar esta estructura de dependencia, el muestreo marginal genera combinaciones sintéticas absurdas que nunca ocurrirían en la población real.

Ejemplo aplicado a la Valuación Inmobiliaria

Imagina que un modelo evalúa dos variables fuertemente correlacionadas: Superficie útil y Número de habitaciones:

1. Para evaluar la contribución de la Superficie útil de una casa de lujo de 400 m², *KernelSHAP* mantiene esa superficie (1), pero reemplaza el Número de habitaciones (0) por un valor aleatorio de la base de datos de fondo, como 1 habitación (perteneciente a un estudio pequeño).
2. Se crea una instancia sintética imposible en el mercado real: una propiedad de 400 m² con solo 1 habitación.
3. El modelo de *Machine Learning* se ve obligado a emitir una predicción sobre un punto fuera de su distribución conocida (*out-of-distribution*), donde sus fronteras de decisión no fueron entrenadas.

4. Resultado: El modelo da una respuesta errática sobre la cual *KernelSHAP* calcula la importancia, lo que puede otorgar un peso desproporcionado a combinaciones absurdas y sesgar los valores *SHAP* calculados.

3. El dilema teórico: ¿Por qué *KernelSHAP* usa la distribución marginal?

Podría pensarse que la solución es muestrear usando la distribución condicional (tomando solo valores aleatorios de habitaciones que sean coherentes con casas de 400 m². Sin embargo, esto plantea un dilema teórico crucial:

- Si se usa el muestreo condicional, se altera la función de valor del juego de *Shapley*.
- Como consecuencia, una variable que el modelo no utiliza en absoluto para hacer sus predicciones podría recibir un valor *SHAP* distinto de cero simplemente por estar correlacionada con otra variable que sí se usa, violando el axioma de Jugador Nulo (*Dummy*).

Por esta razón, *KernelSHAP* opta por el muestreo marginal para mantenerse fiel al modelo y preservar estrictamente los axiomas matemáticos de *Shapley*, aceptando como costo colateral la generación de instancias sintéticas poco probables.

Tarjeta 12

En el anverso pone: "¿Para qué tipo específico de modelos de aprendizaje automático fue diseñado *TreeSHAP*?" La respuesta del anverso es: "Para modelos basados en ensambles de árboles de decisión, tales como *XGBoost*, *LightGBM*, *CatBoost* y *Random Forest*."

¿Podrías explicarme este tema más detalladamente?

*TreeSHAP* es una variante especializada de *SHAP* diseñada específicamente para modelos de aprendizaje automático basados en árboles de decisión y sus ensambles.

A diferencia de métodos agnósticos como *KernelSHAP* —que tratan a cualquier algoritmo como una «caja negra» opaca—, *TreeSHAP* fue desarrollado por *Scott Lundberg, Gabriel Erion y Su-In Lee* (2018/2020) como un algoritmo *específico de modelo* (*model-specific*) que aprovecha la estructura interna jerárquica de los árboles.

1. ¿Por qué se diseñó específicamente para árboles?

1. Aprovechamiento de la topología en árbol: Un árbol de decisión realiza predicciones dividiendo el espacio de datos mediante reglas secuenciales desde la raíz hasta las hojas. *TreeSHAP* rastrea directamente los caminos de decisión y las proporciones de datos que

fluyen por cada rama, identificando qué combinaciones de variables alteran realmente la predicción sin necesidad de simular combinaciones a ciegas.

2. Reducción drástica del costo computacional:
    - Calcular valores de Shapley exactos con métodos agnósticos (como *KernelSHAP*) requiere evaluar un número exponencial de combinaciones de variables ($O(2^M)$), lo que resulta extremadamente lento en datasets reales.
    - *TreeSHAP* reduce esa complejidad exponencial a una complejidad polinomial de $O(TLD^2)$ donde $T$ es el número de árboles, $L$ es el número máximo de hojas y $D$ es la profundidad máxima de los árboles.
3. Propiedad de Aditividad en Ensambles: Como los modelos de ensamble (*como Random Forest* o *XGBoost*) combinan las predicciones de múltiples árboles individuales (ya sea por promedio o por suma ponderada de residuos), la propiedad matemática de Aditividad de *Shapley* permite calcular los valores *SHAP* para cada árbol individualmente y luego sumarlos o promediarlos para obtener la explicación del ensamble completo.

2. Algoritmos y familias compatibles

*TreeSHAP* se adapta de forma transparente a las principales arquitecturas basadas en árboles:

- *Gradient Boosted Decision Trees* (GBDT): *XGBoost* (crecimiento secuencial ponderado por gradiente), *LightGBM* (crecimiento por hojas o *leaf-wise*), *CatBoost* (árboles simétricos y manejo de categóricas) y *Gradient Boosting* clásico.
- Bosques y *Bagging*: Random Forest y *Extra Trees* (construcción paralela de múltiples árboles de decisión).
- Árboles individuales: Árboles de Clasificación y Regresión (CART / *Decision Trees*).

3. Relevancia práctica en la industria y valuación inmobiliaria

En el análisis de datos estructurados tabulares (como la estimación de precios inmobiliarios o el riesgo crediticio), los modelos de ensamble de árboles como *XGBoost* y *LightGBM* son los algoritmos con mejor rendimiento predictivo.

Antes de la llegada de *TreeSHAP*, resultaba casi imposible explicar de manera exacta las predicciones de un bosque con cientos de árboles en datasets masivos. La velocidad de *TreeSHAP* permitió pasar de explicaciones aisladas a calcular matrices globales completas en segundos,

habilitando gráficos de resumen (*beeswarm plots*), gráficos de cascada (*waterfall plots*) y análisis de interacciones para auditar modelos complejos.

Tarjeta 13

En el anverso pone: "¿Cuál es la complejidad computacional exacta del algoritmo *TreeSHAP*?" La respuesta del anverso es:$O(TLD^2)$ donde $T$ es el número de árboles, $L$ es el número máximo de hojas y $D$ es la profundidad máxima de los árboles."

¿Podrías explicarme este tema más detalladamente?

Como vimos anteriormente, la fórmula $O(TLD^2)$ define la complejidad computacional del algoritmo *TreeSHAP* para calcular los valores de *Shapley* exactos en modelos basados en conjuntos de árboles de decisión (como *XGBoost*, *LightGBM* y Random Forest):

- $T$ (Número de árboles / Trees): Es la cantidad total de árboles en el ensamble. Gracias a la propiedad axiomática de Aditividad, el cálculo se realiza de forma independiente en cada árbol y se suma o promedia al final.
- $L$ (Número máximo de hojas / *Leaves*): Representa los nodos terminales del árbol por los que fluyen los datos.
- $D$ (Profundidad máxima / *Depth*): Es el número máximo de divisiones desde la raíz hasta la hoja. Aparece elevado al cuadrado ($D^2$) porque *TreeSHAP* utiliza programación dinámica para rastrear y actualizar las combinaciones de características presentes y ausentes a lo largo de cada camino de decisión.

¿Por qué representa una revolución frente a *KernelSHAP*?

1. De Exponencial a Polinomial: Mientras que la estimación agnóstica exacta (*KernelSHAP*) requiere evaluar hasta $O(2^M)$ combinaciones sintéticas (donde *M* es el número total de variables de la base de datos), *TreeSHAP* aprovecha la estructura interna jerárquica del árbol para reducir el tiempo de cálculo a una escala polinomial dependiente solo de las dimensiones del árbol.
2. Optimización en Grandes Datasets: Para un dataset con 30 características y un ensamble de 100 árboles de profundidad 6, evaluar la caja negra con *KernelSHAP* exigiría más de 1.000 millones de iteraciones por observación, mientras que *TreeSHAP* lo resuelve en apenas 230.000 operaciones.

Avances y optimizaciones más recientes en las fuentes

Las fuentes académicas destacan cómo la literatura ha evolucionado para acelerar aún más esta complejidad cuadrática:

- *Linear TreeSHAP*: Reduce la complejidad a $O(TLD)$ mediante el uso de propiedades de aritmética polinomial y puntos de *Chebyshev*, eliminando redundancias.
- *GPUTreeSHAP*: Paraleliza el cálculo masivo mediante arquitecturas CUDA/SIMT en GPUs NVIDIA, logrando aceleraciones de hasta 19× a 340× en conjuntos de datos masivos.
- *Quadrature-TreeSHAP* (*Depth-Independent*): Reformula la integración mediante cuadratura numérica de *Gauss-Legendre* (usando 8 puntos fijos), desvinculando la complejidad de la profundidad del árbol para alcanzar un rendimiento lineal $O(TLN)$.

Tarjeta 14

En el anverso pone: "¿Cuáles son las dos variantes de *TreeSHAP* y en qué se diferencian?" La respuesta del anverso es: "La variante intervencional (que calcula los valores de *Shapley* clásicos mediante muestreo marginal) y la dependiente de la ruta (*path-dependent*, que calcula valores condicionales usando la estructura del árbol)."

¿Podrías explicarme este tema más detalladamente?

La distinción entre la variante intervencional (*Interventional TreeSHAP*) y la variante dependiente de la ruta (*Path-dependent TreeSHAP*) aborda uno de los debates teóricos y metodológicos más importantes en la Inteligencia Artificial Explicable (XAI): cómo tratar las características "ausentes" o no observadas al evaluar un árbol de decisión.

Ambas variantes se diseñaron para aprovechar la topología jerárquica de los modelos basados en árboles (como *XGBoost* o *Random Forest*), pero difieren en la distribución probabilística que utilizan para simular la ausencia de variables.

1. Variante Intervencional (*Interventional TreeSHAP*)

Esta variante calcula los valores de *Shapley* clásicos y verdaderos según la teoría de juegos cooperativos, apoyándose en la distribución marginal de las características.

- Mecanismo de funcionamiento: Para evaluar qué ocurre cuando una variable está "ausente" en una combinación (coalición), no utiliza únicamente la estructura del árbol individual, sino un conjunto de datos de referencia o fondo (*background dataset*). Reemplaza el valor real de

la variable por valores muestreados al azar desde este *background dataset*, asumiendo independencia entre las variables presentes y las ausentes.

- Ventaja clave (Cumplimiento estricto de los axiomas): Al usar la distribución marginal, respeta estrictamente el Axioma de Jugador Nulo (*Dummy axiom*)1. Esto garantiza que, si una variable no es utilizada activamente por el modelo para realizar la predicción, su valor *SHAP* asignado será estrictamente cero.
- Limitación / Riesgo: Al ignorar las correlaciones existentes entre las variables (asumir independencia marginal), puede generar combinaciones sintéticas o irrealistas que están fuera de la distribución real observada (*out-of-distribution*).

2. Variante Dependiente de la Ruta (*Path-dependent TreeSHAP*)

Esta variante estima valores de *Shapley* condicionales, aprovechando de manera determinista las estadísticas de división grabadas en los nodos del árbol durante el entrenamiento.

- Mecanismo de funcionamiento: No requiere un conjunto de datos de fondo externo (*background dataset*). Cuando una variable está "ausente" en un camino de decisión, el algoritmo distribuye el peso de la predicción hacia los nodos hijos basándose en la proporción de muestras de entrenamiento que fluyeron por cada rama (*cover weighting* o probabilidad condicional estimada por el propio árbol).
- Ventaja clave (Fidelidad a los datos reales): Evita por completo la creación de instancias sintéticas irrealistas o fuera de distribución (*out-of-distribution*), ya que solo evalúa los caminos y proporciones de datos que el árbol realmente observó en su entrenamiento.
- Limitación / Riesgo: Al condicionar las probabilidades sobre la estructura del árbol, modifica la función de valor del juego de *Shapley*. Como consecuencia:
    1. Violación del axioma *Dummy*: Una variable que el modelo no utiliza en absoluto para predecir puede recibir un valor *SHAP* distinto de cero simplemente por estar altamente correlacionada con otra variable que sí se utiliza.
    2. Inconsistencia entre árboles equivalentes: Árboles con estructuras distintas pero idéntico comportamiento predictivo pueden devolver explicaciones diferentes.

Resumen comparativo directo

| Aspecto | Intervencional (*Interventional*) | Dependiente de la ruta (*Path-dependent*) |
|---|---|---|
| Distribución de referencia | Marginal (asume independencia). | Condicional (basada en el árbol). |
| Uso de datos externos | Requiere un *background dataset*. | No requiere datos externos (usa estadísticas del árbol). |
| Axioma de Jugador Nulo (*Dummy*) | Se cumple estrictamente (variables no usadas reciben $\phi = 0$). | Puede violarse (variables no usadas pueden recibir $\phi \neq 0$ por correlación). |
| Muestras fuera de distribución. | Puede generar combinaciones irrealistas. | Evita evaluar puntos irrealistas (*out-of-distribution*). |
| Interpretación teórica. | Valores de *Shapley* clásicos ("Verdadero al modelo"). | Valores de *Shapley* condicionales ("Verdadero a los datos"). |

Evolución práctica en la librería *shap*

En las primeras versiones del paquete *Python shap*, la variante predeterminada en *TreeExplainer* era la dependiente de la ruta (*tree-path dependent*) debido a que no requería un dataset de fondo. Sin embargo, en versiones más recientes, la variante intervencional se convirtió en la opción por defecto para garantizar explicaciones alineadas con la definición axiomática formal de los valores de *Shapley*.

En un modelo *XGBoost* de valoración inmobiliaria donde variables como la Superficie y el Número de habitaciones presentan alta colinealidad:

- Intervencional asegurará que, si el árbol descartó la variable Número de habitaciones, su contribución económica explicada sea exactamente 0€.
- *Path-dependent* evitará evaluar viviendas hipotéticas absurdas (como 400 m² con 1 habitación), pero podría asignarle parte del valor a Número de habitaciones debido a su correlación histórica con la superficie.

Tarjeta 15

En el anverso pone: "¿Qué problema puede presentar la variante de *TreeSHAP* dependiente de la ruta (*path-dependent*)?" La respuesta del anverso es: "Puede asignar valores *SHAP* no nulos a características que el modelo no utilizó directamente en sus decisiones si están correlacionadas con variables influyentes."

¿Podrías explicarme este tema más detalladamente?

El problema señalado en la tarjeta se debe a la forma en que la variante dependiente de la ruta (*Path-dependent TreeSHAP*) simula la «ausencia» de una variable al evaluar los caminos de decisión de un árbol.

1. El origen técnico: La Distribución Condicional

Para determinar qué sucede cuando una variable está «ausente» en una combinación, *Path-dependent TreeSHAP* calcula la expectativa condicional aprovechando directamente la proporción de datos de entrenamiento que descendió por cada nodo del árbol (*cover weighting*).

Al hacer esto, el algoritmo no evalúa la variable en aislamiento marginal, sino que condiciona la predicción a la estructura de datos observada en el árbol. Esto altera la función de valor subyacente del juego de *Shapley*.

2. Violación del Axioma de Jugador Nulo (*Dummy Axiom*)

En la teoría pura de juegos cooperativos, el Axioma de Jugador Nulo establece que si un jugador (característica) no altera en absoluto el resultado del juego (la predicción del modelo), su asignación de valor debe ser estrictamente cero ($\phi = 0$).

Sin embargo, en *Path-dependent TreeSHAP*:

- Si el modelo no utiliza la variable $A$ en ninguna división de sus árboles, pero $A$ está fuertemente correlacionada con una variable $B$ que sí es muy influyente en el modelo,
- condicionar probabilísticamente sobre la estructura del árbol hace que conocer el valor de $A$ altere las probabilidades implícitas de tomar ciertas ramas asociadas a $B$.
- Resultado: La variable $A$ recibe un valor *SHAP* diferente de cero ($\phi_A \neq 0$), a pesar de que el modelo nunca la consultó para emitir la predicción.

3. Ejemplo práctico en Valuación Inmobiliaria

Imaginemos un modelo *XGBoost* entrenado para estimar precios de viviendas:

1. Supongamos que en el *dataset* existen dos variables fuertemente correlacionadas: Superficie útil y Superficie del tejado.
2. Al entrenar el algoritmo, *XGBoost* selecciona únicamente Superficie útil para realizar las divisiones de los árboles y descarta por completo Superficie del tejado (esta última jamás aparece en ningún nodo del modelo).

3. Si explicamos la valoración de una vivienda concreta utilizando *Path-dependent TreeSHAP*:
    - Debido a la alta correlación entre ambas superficies en las estadísticas del árbol, el algoritmo le asignará a *Superficie del tejado* una contribución económica (por ejemplo, +6.000 €}.
    - Un auditor o valuador podría concluir erróneamente que el modelo está considerando el tejado para fijar el precio, cuando en realidad el código del modelo lo ignoró totalmente.
4. Si explicáramos la misma valoración mediante *Interventional TreeSHAP* (que utiliza muestreo de la distribución marginal):
    - La Superficie del tejado recibiría estrictamente 0 €, respetando la lógica interna del algoritmo y el axioma de Jugador Nulo.

4. El dilema: «Verdadero al Modelo» vs. «Verdadero a los Datos»

Este comportamiento refleja una discusión filosófica central en XAI6:

- *Path-dependent TreeSHAP* («Verdadero a los Datos» / *True to the Data*): Mide cuánta información sobre la realidad aporta una variable considerando el contexto de los datos observados. Evita evaluar combinaciones irrealistas fuera de distribución (*out-of-distribution*), pero a costa de atribuir importancia a variables ignoradas por el algoritmo.
- *Interventional TreeSHAP* **(**«Verdadero al Modelo» / *True to the Model***)**: Mide estrictamente la lógica de ejecución del código dentro de los árboles. Si la variable no está en el modelo, su atribución es exactamente cero.

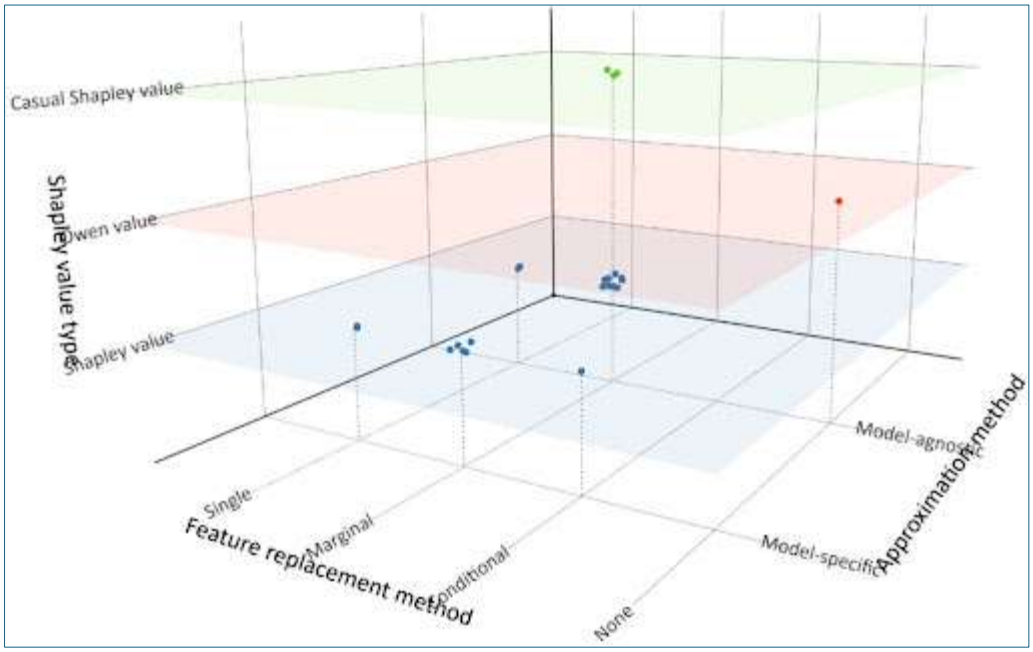
Conclusión práctica

Por este motivo, aunque *Path-dependent TreeSHAP* sigue siendo ampliamente utilizado por ser muy rápido y no requerir un conjunto de datos de fondo (*background dataset*), la librería oficial *shap* cambió en sus versiones recientes la opción predeterminada de *TreeExplainer* hacia la variante Intervencional, garantizando así explicaciones axiomáticamente puras e inmunes a este sesgo de correlación.

Tarjeta 16

En el anverso pone: "¿Cómo clasifica la literatura las aproximaciones de los valores de *Shapley* según el tipo de reemplazo de características y la arquitectura del modelo?" La respuesta del anverso es: "Diagrama en 3D que muestra la taxonomía de métodos de aproximación de *Shapley* comparando

tipos de reemplazo de características y especificidad de modelo. Se organizan en tres ejes: tipo de valor (*Shapley*, *Owen*, *Causal*), método de reemplazo de características (Marginal, Condicional) y especificidad del método de aproximación (Agnóstico vs Específico del modelo)."




Esta taxonomía proviene de la literatura especializada en IA explicable (como el trabajo de *Li et al*., 2023) y se diseñó para organizar el panorama de métodos de atribución basados en *Shapley* mediante un marco tridimensional (3D):

1. Eje 1: Tipo de Valor de *Shapley* (*Shapley Value Type*)

Define la formulación teórica del juego cooperativo que se utiliza para calcular y distribuir la importancia:

- *Shapley Value tradicional*: Asigna atribuciones evaluando la contribución marginal de cada característica a través de todas las combinaciones posibles de variables, sin imponer estructuras o restricciones previas.
- *Owen Value*: Extiende el valor de *Shapley* para juegos con estructuras de coalición *a priori* (*priority coalitions*), es decir, agrupaciones de variables que prefieren actuar juntas antes de distribuir las ganancias.
- *Causal Shapley Value*: Incorpora conocimiento causal de los datos (por ejemplo, mediante grafos causales y el *do-calculus* de *Pearl*), lo que permite diferenciar entre impactos directos e indirectos y evitar atribuciones sesgadas cuando existen relaciones de causa-efecto entre variables.

2. Eje 2: Método de Reemplazo de Características (*Feature Replacement Method*)

Indica cómo el algoritmo simula que una variable está «ausente» al evaluar las diferentes coaliciones:

- Marginal: Sustituye las características ausentes muestreando valores de la distribución marginal de un conjunto de fondo, asumiendo independencia entre las variables presentes y ausentes.
- Condicional: Sustituye las características ausentes muestreando desde la distribución condicional dada la presencia de las demás variables, respetando las correlaciones y dependencias observadas.
- *Single* (o Referencia Única): Reemplaza la variable ausente utilizando un único valor base o de referencia fijo (por ejemplo, un perfil neutro o un valor promedio global).

3. Eje 3: Especificidad del Método de Aproximación (*Approximation Method*)

Clasifica las aproximaciones según si dependen o no de la arquitectura interna del modelo predictivo:

- Agnóstico al modelo (*Model-agnostic*): Trata al modelo como una caja negra sin importar su arquitectura. Incluye aproximaciones por muestreo Monte Carlo, regresión lineal ponderada (*KernelSHAP*) o muestreo multilineal.
- Específico del modelo (*Model-specific*): Aprovecha la estructura matemática e interna de familias de algoritmos concretas para acelerar drásticamente el cálculo. Incluye aproximaciones para modelos lineales, modelos basados en árboles (*TreeSHAP*) y redes neuronales profundas (*DeepSHAP* / *DeepLIFT*).

¿Por qué es útil esta clasificación 3D?

Permite ubicar cualquier herramienta de explicabilidad en un punto preciso de este espacio tridimensional. Por ejemplo:

- *TreeSHAP Intervencional*: (*Shapley Value, Marginal, Model-specific / Tree-based*).
- *TreeSHAP Path-dependent*: (*Shapley Value, Conditional, Model-specific / Tree-based*).
- *KernelSHAP*: (*Shapley Value, Marginal, Model-agnostic / Linear Regression*).

Esta estructuración ayuda a evitar confusiones operativas y a seleccionar la variante adecuada según si se busca rigor axiomático, manejo de correlaciones o máxima eficiencia computacional.

Tarjeta 17

En el anverso pone: "¿En qué consiste el Método de Permutación para la estimación de valores *SHAP*?" La respuesta del anverso es: "Estima los valores de *Shapley* promediando las contribuciones marginales a través de permutaciones de características evaluadas de adelante hacia atrás (muestreo antitético)."

¿Podrías explicarme este tema más detalladamente?

El Método de Permutación (*Permutation Sampling Method*) es uno de los estimadores agnósticos al modelo (*model-agnostic*) más eficientes para aproximar los valores de *Shapley* cuando no se puede utilizar una implementación específica para la arquitectura (como *TreeSHAP* para árboles de decisión).

1. El problema que resuelve: De la combinación a la secuencia

El cálculo exacto del valor de *Shapley* requiere evaluar $2^M$ subconjuntos (coaliciones) de características. Para un modelo con 30 variables, esto exigiría más de 1.000 millones de evaluaciones del modelo, lo cual es inviable.

En lugar de evaluar todas las coaliciones posibles a ciegas, el Método de Permutación aprovecha el concepto de ordenamiento secuencial (permutaciones de tamaño $M!$). Si ordenamos de forma aleatoria las variables de un *dataset*, podemos medir la contribución marginal de cada variable añadiéndola secuencialmente una a una.

2. El mecanismo: Pasadas de adelante hacia atrás (*Forward-Backward*)

Supongamos que queremos explicar la predicción de una vivienda con 4 variables: [Ubicación, Superficie, Garaje, Antigüedad].

1. Pasada Directa (*Forward*): Tomamos una permutación aleatoria, por ejemplo: (Ubicación → Superficie → Garaje → Antigüedad). Añadimos las variables una a una desde un grupo vacío (Ø) y medimos cómo cambia la predicción del modelo en cada paso:
    - Impacto de Ubicación al agregarse a Ø.
    - Impacto de Superficie al agregarse a {Ubicación}.
    - Impacto de Garaje al agregarse a {Ubicación, Superficie}.
    - Impacto de Antigüedad al agregarse a {Ubicación, Superficie, Garaje}.

2. Pasada Inversa (Backward / Reverse): Recorremos la misma permutación, pero en sentido exactamente opuesto: (Antigüedad → Garaje → Superficie → Ubicación).

- Impacto de Antigüedad al agregarse a Ø.
- Impacto de Garaje al agregarse a {Antigüedad}.
- Impacto de Superficie al agregarse a {Antigüedad, Garaje}.
- Impacto de Ubicación al agregarse a {Antigüedad, Garaje, Superficie}.

3. ¿Por qué es tan potente el Muestreo Antitético (*Antithetic Sampling*)?

Esta técnica de evaluar cada permutación $o$ junto con su versión invertida $-o$ se conoce estadísticamente como muestreo antitético (*antithetic sampling*):

- Reutilización de predicciones: La predicción de un subconjunto evaluado en la pasada directa (por ejemplo, f({Ubicación, Superficie}) sirve directamente como punto de partida para la pasada inversa. Esto reduce a la mitad las llamadas computacionales al modelo.
- Reducción drástica de la varianza: Al promediar la contribución marginal directa $\hat{\Delta}_{o(k),j}$ y la inversa $\hat{\Delta}_{-o(k),j}$, se compensan los sesgos del orden de entrada:

$$\hat{\phi}_j = \frac{1}{2m}\sum_{k=1}^{m}\left(\hat{\Delta}_{o(k),j} + \hat{\Delta}_{-o(k),j}\right)$$

- Garantía del Axioma de Eficiencia: El muestreo antitético asegura que la suma de los valores *SHAP* calculados equivalga exactamente a la predicción final menos el valor promedio (cumpliendo la exactitud local en todo momento).

4. Ejemplo práctico en Valuación Inmobiliaria

Imaginemos que queremos estimar el impacto de la variable Garaje en una vivienda:

- En la pasada directa, Garaje se evalúa cuando la vivienda ya tiene contabilizadas la Ubicación y la Superficie (mide su contribución como "elemento complementario").
- En la pasada inversa, Garaje se evalúa primero cuando la vivienda no tiene asignada ninguna otra variable (mide su contribución como "elemento aislado").

Al promediar ambas contribuciones a lo largo de unas pocas permutaciones aleatorias (la librería *shap* utiliza por defecto tan solo 10 permutaciones), se obtiene una estimación sumamente precisa del valor *SHAP* de Garaje sin importar la complejidad del modelo subyacente (redes neuronales, SVM o *ensembles*)

Tarjeta 18

En el anverso pone: "¿Qué función cumple un *Waterfall Plot* (o gráfico de cascada) en el análisis *SHAP*?" La respuesta del anverso es: "Explica una predicción individual mostrando el valor base inicial *E*[*f*(*X*)] y cómo cada característica ajusta de manera incremental el resultado hacia arriba o hacia abajo."

¿Podrías explicarme este tema más detalladamente?

El gráfico de cascada (*Waterfall Plot*) es una de las herramientas de visualización más intuitivas y rigurosas de *SHAP* para realizar explicaciones locales, es decir, para auditar y justificar la predicción de una sola observación específica.

¿Cómo funciona la «cascada»?

El gráfico de cascada representa visualmente la propiedad de Exactitud Local (*Local Accuracy*), demostrando cómo el modelo pasa de la predicción promedio general a la estimación particular de esa instancia concreta:

$$f(x) = E[f(X)] + \sum_{j=1}^{M} \phi_j$$

1. El punto de anclaje inicial (Valor base / $E[f(X)]$): Ubicado habitualmente en la parte inferior (o derecha), representa el precio o valor promedio esperado que predice el modelo sobre todo el conjunto de datos cuando no conoce los datos específicos del caso.

2. Los peldaños de la cascada (Contribuciones individuales / $\phi_j$): Cada característica de la instancia actúa como un escalón horizontal que ajusta el resultado acumulado:

- Valores *SHAP* positivos (barras rojas o flechas hacia la derecha): Representan atributos que incrementan la predicción por encima del promedio.
- Valores *SHAP* negativos (barras azules o flechas hacia la izquierda): Representan atributos que reducen la predicción por debajo del promedio.
- Longitud de la barra: Refleja la magnitud exacta (en euros, dólares o la unidad de la variable objetivo) que aporta esa característica específica.
- Valor observado: Al lado del nombre de la variable se muestra el dato real de esa instancia (por ejemplo, Superficie = 120 m²).

3. El punto de llegada (Predicción final / $f(x)$): Ubicado en la parte superior, muestra la predicción matemática exacta del modelo tras sumar y restar todas las contribuciones individuales a partir del valor base.

2. Ejemplo práctico en Valuación Inmobiliaria

Imaginemos un modelo *XGBoost* que predice el precio de un departamento en 242.000 €:

- Valor base ($E[f(X)]$): 200.000 € *(*precio promedio de todas las viviendas en la base de datos).
- + Superficie (120 m²): +35.000 € (barra roja que desplaza el valor a 235.000 €).
- + Ubicación (Zona centro): +20.000 € *(*barra roja que desplaza el valor a 255.000 €*)*.
- - Antigüedad (40 años): -13.000 € (barra azul que reduce el valor final a 242.000 €).

El gráfico de cascada ordena las variables de mayor a menor impacto absoluto, permitiendo que un tasador o auditor entienda de un vistazo los factores dominantes en esa valoración específica.

3. Diferencia entre *Waterfall Plot* y *Force Plot*

Aunque ambos visualizan una predicción individual:

- El *Force Plot* dispone todas las fuerzas positivas y negativas empujando de forma continua sobre un único eje horizontal comprimido.
- El *Waterfall Plot* desglosa cada variable secuencialmente en su propio renglón, mostrando el subtotal acumulado paso a paso, lo que facilita enormemente la lectura en informes técnicos y auditorías profesionales.

Tarjeta 19

En el anverso pone: "¿Cómo se calcula la Importancia Global de Características en *SHAP* (*SHAP Feature Importance*)?" La respuesta del anverso es: "Se obtiene calculando el promedio de los valores absolutos de *SHAP* para cada característica a través de todas las observaciones:

$$I_j = \frac{1}{n}\sum_{i=1}^{n}\left|\phi_j^{(i)}\right|$$

La fórmula de Importancia Global de Características en SHAP (*SHAP Feature Importance*) es el puente que permite pasar de las explicaciones locales (un inmueble específico) a una visión global de todo el modelo.

$$I_j = \frac{1}{n}\sum_{i=1}^{n}\left|\phi_j^{(i)}\right|$$

1. Desglose término por término de la fórmula

- $I_j$: Es el índice de importancia global de la característica $j$ .
- $n$: El número total de observaciones o instancias evaluadas en el conjunto de datos.
- $\phi_j^{(i)}$: El valor *SHAP* local atribuido a la característica $j$ para la observación específica $i$.
- $|\cdot|$: El valor absoluto de la contribución individual.

2. ¿Por qué se toma el Valor Absoluto? (La razón fundamental)

El uso del valor absoluto es el detalle técnico más importante de esta definición:

- Si promediáramos los valores *SHAP* directamente sin valor absoluto ( $\frac{1}{n}\sum \phi_j^{(i)}$), las contribuciones positivas de un inmueble (por ejemplo, +30.000 € por tener un jardín amplio) se cancelarían con las contribuciones negativas de otro inmueble (por ejemplo, -30.000 € por la falta de garaje). El resultado medio sería cercano a 0, dando la falsa impresión de que la variable no influye en la predicción.
- Al aplicar el valor absoluto ($\left|\phi_j^{(i)}\right|$), medimos la magnitud pura del impacto (cuánto «mueve la aguja» esa variable respecto al valor base), independientemente de si empuja el precio hacia arriba o hacia abajo.

3. De lo Local a lo Global: Explicaciones consistentes

A diferencia de otras metodologías de aprendizaje automático que usan métricas diferentes para la explicación local y la explicación global, *SHAP* utiliza los valores de *Shapley* locales como la «unidad atómica» de la que se derivan todas las métricas globales.

Al calcular la matriz completa de valores *SHAP* (donde cada fila es una propiedad y cada columna es una característica), el promedio de los valores absolutos por columna genera una clasificación global perfectamente consistente con las decisiones locales del modelo

4. *SHAP Feature Importance* vs. Métricas Tradicionales

Existe una diferencia conceptual clave entre la importancia global de *SHAP* y otras métricas de importancia:

1. Importancia tradicional en Árboles (*Gain* / *Split count*): Mide con qué frecuencia o cuánta reducción de impureza produce una variable al dividir los nodos de los árboles (refleja la mecánica interna del algoritmo, pero puede estar sesgada hacia variables continuas).
2. *Permutation Feature Importance*: Mide cuánto cae la precisión o rendimiento del modelo al desordenar al azar una variable.
3. *SHAP Feature Importance*: Mide el impacto promedio real que tiene la variable sobre el valor predicho, expresado en las mismas unidades de la variable objetivo (por ejemplo, euros o dólares en la valoración de un inmueble).

5. Ejemplo práctico en Valuación Inmobiliaria (AVM)

Imagina que entrenamos un modelo *XGBoost* sobre una muestra de 1.000 viviendas:

- Para viviendas grandes, la variable Superficie aporta en promedio +35.000 a la línea base.
- Para viviendas pequeñas, la Superficie restringe el valor en promedio en -25.000 €.
- Al aplicar la fórmula:

$$I_{\text{Superficie}} = \frac{|+35.000| + |-25.000| + \cdots}{1000} = 28.000\,€$$

Interpretación para el valuador: "En promedio, la característica Superficie altera la estimación del precio de una vivienda en 28.000 € respecto al promedio del mercado."

6. Visualización gráfica

En la librería de *Python shap*, esta métrica se visualiza mediante un gráfico de barras global (*shap.plots.bar*), donde las variables se ordenan de arriba a abajo según su valor $I_j$ decreciente.

Aunque el gráfico de barras permite identificar rápidamente el *ranking* de variables dominantes, no muestra la dirección del impacto (si los valores altos aumentan o disminuyen el precio). Para ver la dirección y dispersión simultáneamente, se recurre al gráfico de enjambre (*Beeswarm / Summary Plot*).

Tarjeta 20

En el anverso pone: "¿En qué se diferencia la Importancia de Características basada en *SHAP* de la Importancia por Permutación (*Permutation Feature Importance*)?" La respuesta del anverso es: "La importancia por permutación mide la pérdida de rendimiento (error) del modelo al desordenar una variable; la importancia *SHAP* mide la magnitud absoluta del impacto en las predicciones."

¿Podrías explicarme este tema más detalladamente?

La diferencia entre la Importancia de Características basada en *SHAP* y la Importancia por Permutación (*Permutation Feature Importance* - PFI) radica en qué mide exactamente cada método para determinar el peso de una variable en el modelo.

1. El principio fundamental de cada método

- Importancia por Permutación (PFI): Mide la pérdida de rendimiento o aumento del error del modelo al "destruir" intencionadamente la información de una variable.
    - Procedimiento: Toma una característica del conjunto de prueba, desordena (permuta) al azar sus valores entre las filas y mide cuánto cae la métrica de evaluación (como el $R^2$, el RMSE o la exactitud) respecto a los valores reales.
    - Pregunta que responde: ¿Cuánto empeora la capacidad predictiva del modelo si privamos a la variable de su estructura real?
- Importancia Global de SHAP: Mide la magnitud media de la contribución o impacto real que ejerce esa variable sobre el resultado emitido por el modelo.
    - Procedimiento: Promedia el valor absoluto de las contribuciones *SHAP* individuales de cada observación en el dataset ($I_j = \frac{1}{n}\sum_{i=1}^{n}\left|\phi_j^{(i)}\right|$).
    - Pregunta que responde: ¿En qué medida (en magnitud o unidades de la predicción) desplaza esta variable el resultado final respecto al valor promedio esperado ($\phi_0$)?

2. Cuadro comparativo de diferencias operativas

| Criterio | *Permutation Feature Importance* (*PFI*) | *SHAP Feature Importance* |
|---|---|---|
| Métrica subyacente | Degradación de la precisión o rendimiento ($R^2$, *MSE, AUC*). | Promedio de los valores absolutos de *Shapley* (Mean \|SHAP\|). |
| Dependencia de la variable real ($y$) | Sí: Necesita comparar la predicción contra las etiquetas reales para calcular el error. | No: Evalúa directamente las salidas y variaciones del modelo $\hat{f}(X)$. |
| Unidades de medida | Unidades de la métrica de rendimiento (ej. pérdida de 0,05 en $R^2$ o aumento de RMSE). | Mismas unidades que la variable objetivo (ej. euros, dólares o puntos de probabilidad). |
| Perspectiva de cálculo | Puramente global (se permuta la columna completa en todo el dataset). | De lo local a lo global (agrega las contribuciones atómicas calculadas para cada instancia). |

| Criterio | *Permutation Feature Importance* (*PFI*) | *SHAP Feature Importance* |
|---|---|---|
| Dirección del efecto | Unidimensional: solo indica cuánto depende el modelo de la variable, no si el efecto es positivo o negativo. | Bidireccional: la métrica de barras se puede enriquecer mediante un *summary plot* (gráfico de enjambre) para ver si valores altos o bajos suben o bajan la predicción. |

3. Ejemplo práctico en Valuación Inmobiliaria (AVM)

Imagina un modelo *XGBoost* ajustado para predecir precios de viviendas:

1. Análisis mediante *PFI*: Si desordenamos al azar la columna Superficie, el error del modelo pasa de un *RMSE* de 15.000 € a 45.000 €. *PFI* dirá que la superficie es la variable más importante porque la precisión del modelo colapsó sin ella.
2. Análisis mediante *SHAP*: *SHAP* evalúa cada vivienda individualmente y descubre que, en promedio, tener más o menos metros cuadrados modifica la valoración en $\pm 35.000$ € respecto al precio base del mercado ($\phi_0$). El valuador obtiene una cifra en euros que refleja la magnitud del impacto en el precio real, no un mero indicador de degradación del algoritmo.

4. ¿Por qué esta diferencia es relevante en la práctica?

*PFI* mide la dependencia mecánica del modelo hacia un atributo para mantener su precisión, mientras que *SHAP* mide la influencia o peso económico directo que tiene el atributo en la formación de la predicción final. Por esta razón, *SHAP* permite mantener una base metodológica única e integrada para la explicación individual y la auditoría global.

Tarjeta 21

En el anverso pone: "¿Qué información adicional proporciona un *SHAP Summary Plot* comparado con un gráfico de barras de importancia tradicional?" La respuesta del anverso es: "Muestra la distribución completa de los efectos individuales, la dirección del impacto (positivo/negativo) y la magnitud de la característica mediante código de colores."

¿Podrías explicarme este tema más detalladamente?

El *SHAP Summary Plot* (también conocido como *beeswarm plot* o gráfico de enjambre) supera la principal limitación de los gráficos de barras tradicionales: un gráfico de barras colapsa toda la información en un único número promedio por variable, ocultando la dirección del efecto, las asimetrías y las no linealidades de los datos.

1. ¿Por qué el gráfico de barras tradicional se queda corto?

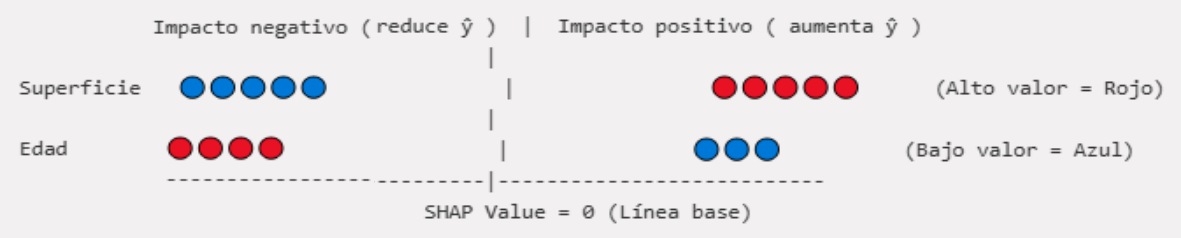
Un gráfico de barras de importancia tradicional (ya sea basado en *Gain*, *Permutation Importance* o en el promedio de los valores absolutos de *SHAP*, Mean |SHAP|) se limita a ordenar las variables de mayor a menor según la magnitud promedio de su impacto.

Aunque responde a la pregunta "¿Cuáles son las variables más importantes en el modelo?", deja sin respuesta preguntas críticas para la auditoría y la interpretación:

- ¿Un valor alto de la variable aumenta o disminuye la predicción?
- ¿El impacto es constante para todas las observaciones o varía drásticamente según la propiedad o cliente?
- ¿Existen valores atípicos (*outliers*) o no linealidades en el efecto?

2. Las 3 dimensiones adicionales que aporta el *SHAP Summary Plot*

El *Summary Plot* condensa múltiples dimensiones de análisis en una sola imagen al representar cada observación del dataset como un punto individual:




A. Dirección del impacto (Eje X)

- Puntos a la derecha del cero (SHAP $> 0$): La variable aumentó la predicción para esa observación respecto al valor base promedio.
- Puntos a la izquierda del cero (SHAP$< 0$ \$): La variable redujo la predicción para esa observación.

B. Magnitud del valor original mediante código de colores

- Color Rojo (*High*): Indica un valor alto de la característica original en esa observación específica.
- Color Azul (*Low*): Indica un valor bajo de la característica original.

- La relación entre color y posición: Permite identificar de inmediato el sentido de la relación. Por ejemplo, si los puntos rojos están a la derecha y los azules a la izquierda, significa que a mayor valor de la variable, mayor es la predicción emitida por el algoritmo.

C. Distribución y dispersión muestral (Eje Y + *Jittering*)

- Las variables se ordenan en el eje vertical por su importancia global (de más a menos influyente).
- Cuando múltiples observaciones tienen valores *SHAP* similares, los puntos se apilan verticalmente (*jittering*), creando una forma de "enjambre" o de densidad. Esto revela si el efecto se concentra en un rango estrecho o si tiene colas anchas y valores atípicos

### 3. Comparación directa

| Aspecto de análisis | Gráfico de barras tradicional | SHAP Summary Plot (*Beeswarm*) |
|---|---|---|
| **Métrica mostrada** | Un único número por variable ( $\text{Mean}\,\lvert\text{SHAP}\rvert$ o *Gain*) [2]. | Distribución completa de todas las instancias del dataset [1]. |
| **Dirección del efecto** | ❌ No disponible (no discrimina signo) [2]. | ✅ Visible mediante el eje horizontal (positivo / negativo) [1]. |
| **Efecto según el valor de entrada** | ❌ No disponible. | ✅ Código de colores (Rojo = Alto, Azul = Bajo) [1] [4]. |
| **Detección de heterogeneidad / outliers** | ❌ Oculta tras el promedio. | ✅ Muestra puntos dispersos y colas de distribución [1] [4]. |

4. Ejemplo en un Modelo *XGBoost* de Valuación Inmobiliaria (AVM)

Imagina un modelo de estimación de precios de vivienda evaluado con un *Summary Plot*:

1. Superficie (m²): Ocupa el primer lugar en el eje vertical. A la derecha del eje central vemos una larga hilera de puntos rojos (SHAP $= +30.000\ € + 80.000\ €$), mientras que los puntos azules se acumulan a la izquierda. Interpretación*:* A mayor superficie, mayor es el precio predicho de forma consistente.
2. Antigüedad (años): Los puntos rojos (casas antiguas) se ubican fuertemente en el lado negativo (SHAP $= -25.000\ €$), mientras que los puntos azules (casas nuevas) están en el positivo. Interpretación: La antigüedad penaliza el precio estimado del inmueble.

3. Distancia a servicios: Muestra una masa densa de puntos azules cerca de cero y una cola delgada de puntos rojos muy a la izquierda. Interpretación: Para la mayoría de las viviendas la ubicación es aceptable y no altera mucho el valor, pero para unas pocas viviendas muy aisladas (puntos rojos alejados), la penalización económica es severa.

Tarjeta 22

En el anverso pone: "En un *SHAP Summary Plot*, ¿qué significado tiene el código de colores asignado a los puntos?" La respuesta del anverso es: "Los puntos rojos indican valores altos de la característica en esa instancia, mientras que los puntos azules representan valores bajos."

¿Podrías explicarme este tema más detalladamente?

En un *SHAP Summary Plot* (también conocido como *Beeswarm Plot* o gráfico de enjambre), el código de colores es la dimensión visual que nos permite pasar de medir simplemente «cuánto importa una variable» a entender cómo influyen los valores reales de esa variable en la predicción del modelo.

1. El significado de la escala cromática

Cada punto en el gráfico representa una observación o instancia individual de tu conjunto de datos12. El color de cada punto codifica la magnitud de la característica de entrada (*feature value*) para ese caso particular:

- Rojo / Magenta (*High*): Indica que la variable toma un valor alto en esa instancia específica (por ejemplo, una vivienda muy grande de 200 m² o un cliente con un ingreso elevado).
- Azul (*Low*): Indica que la variable toma un valor bajo en esa instancia específica (por ejemplo, un departamento pequeño de 45 m² o una vivienda recién construida con 0 años de antigüedad).
- Tonalidades intermedias (Morado / Púrpura): Representan valores cercanos a la media o mediana de esa variable en el conjunto de datos.

2. Cómo leer la relación combinando Color y Eje X

La verdadera potencia del *Summary Plot* surge al observar hacia dónde se desplazan los puntos de cada color a lo largo del eje horizontal (eje X, que mide el impacto o valor SHAP $\phi_j$):

1. Relación Directa o Positiva:
    - Si los puntos rojos están alineados a la derecha SHAP > 0) y los puntos azules a la izquierda (SHAP < 0), significa que a mayor valor de la característica, mayor es la predicción emitida por el modelo.

2. Relación Inversa o Negativa:
    - Si los puntos rojos están a la izquierda (SHAP < 0) y los azules a la derecha (SHAP > 0), indica que, a mayor valor de la característica, más se reduce la predicción del modelo.

3. Ejemplo en Valuación Inmobiliaria (AVM)

Imagina evaluar un modelo *XGBoost* ajustado para predecir precios de viviendas:

- Variable Superficie (m²): Verás una masa de puntos rojos en la zona derecha (impulsando el precio por encima del valor base $\phi_0$) y puntos azules en la zona izquierda (penalizando la estimación). Conclusión rápida: Cuantos más metros cuadrados tiene la casa, más alto es el valor asignado por el modelo.
- Variable Antigüedad (años): Los puntos rojos (casas viejas de 50 años) se acumulan a la izquierda del eje cero, mientras que los puntos azules (casas a estrenar) se posicionan a la derecha. Conclusión rápida: Una mayor antigüedad penaliza el valor estimado del inmueble.

4. ¿Por qué es superior a un gráfico de barras tradicional?

Un gráfico de barras tradicional solo muestra el promedio absoluto (Mean |SHAP|) y nos diría únicamente que la Antigüedad es importante. El código de colores del *Summary Plot* resuelve esa restricción al mostrarnos instantáneamente la dirección, el sentido y la variabilidad del impacto de cada variable.

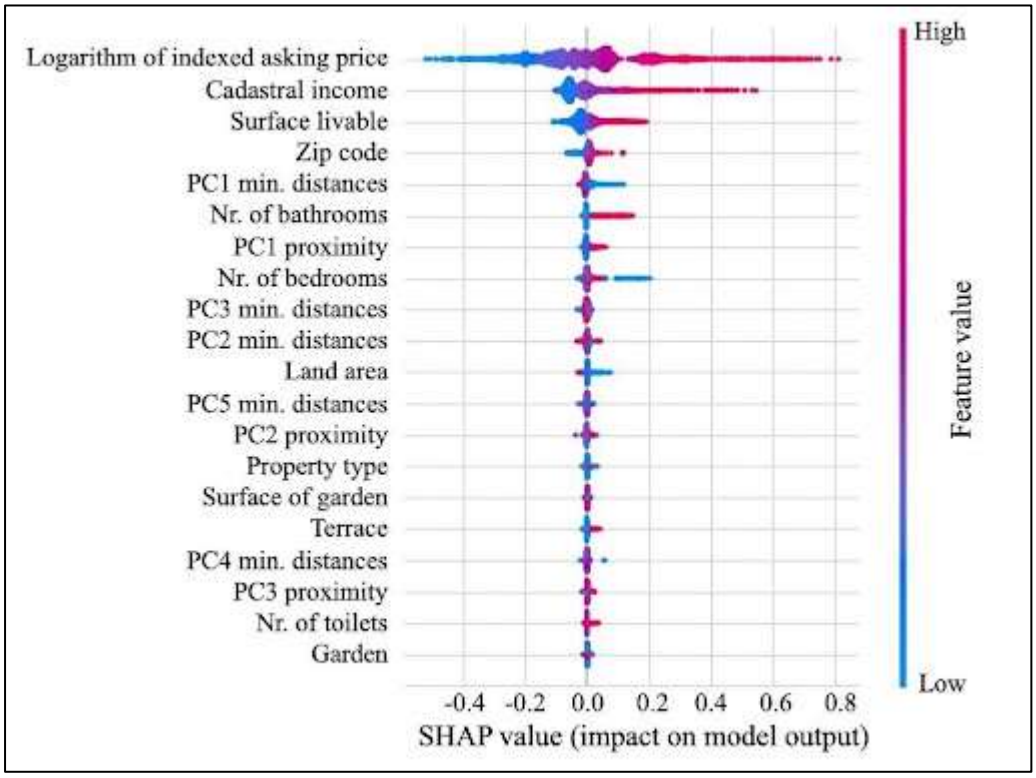
Tarjeta 23

En el anverso pone: "¿Cómo ayuda un *SHAP Summary Plot* a identificar relaciones no lineales en un modelo de predicción inmobiliaria?" La respuesta del anverso es: "*SHAP Summary Plot* para un modelo de regresión inmobiliaria que ilustra el impacto de diferentes características en el alquiler esperado permite observar descensos o saltos no proporcionales en los valores *SHAP* a medida que los valores de las características (rojo o azul) cambian, revelando efectos umbral o rendimientos decrecientes."

¿Podrías explicarme este tema más detalladamente?

El *SHAP Summary Plot* (gráfico de enjambre o *beeswarm plot*) es una herramienta fundamental para identificar relaciones no lineales porque no condensa la información en un único coeficiente o promedio, sino que representa visualmente la distribución individual de cada observación del conjunto de datos.

Al graficar cada inmueble como un punto en el eje horizontal (donde el eje X mide el valor *SHAP* o impacto en la predicción, y el color representa el valor real de la variable: rojo = valor alto, azul = valor bajo), se pueden identificar patrones complejos que los modelos lineales tradicionales no logran capturar.




1. Patrones de No Linealidad observables en el *Summary Plot*

A. Rendimientos Decrecientes (*Diminishing Returns*) y Saturación

- Lo que se observa en el gráfico: Los puntos rojos (valores altos) se desplazan hacia la derecha aumentando la predicción, pero a partir de cierto límite horizontal se "apelotonan" o comprimen en una franja vertical sin poder avanzar más hacia la derecha.
- Interpretación inmobiliaria: Aumentar la superficie de una vivienda de 50 m² a 100 m² genera un gran incremento en la estimación de precio o alquiler (fuerte desplazamiento hacia la derecha). Sin embargo, pasar de 250 m² a 300 m² aporta un valor marginal cada vez menor; el mercado ya no paga la misma cantidad adicional por metro cuadrado extra.

B. Efectos Umbral (*Threshold Effects*) y Discontinuidades

- Lo que se observa en el gráfico: Se aprecia una separación drástica o "salto" en la posición horizontal de los puntos al cambiar el color de azul a rojo, en lugar de un gradiente continuo.

- Interpretación inmobiliaria: En variables de accesibilidad o distancia (por ejemplo, distancia a transporte público o servicios), los puntos de distancias cortas y medias (azules o morados) se agrupan cerca del impacto neutro (SHAP $\approx 0$). Pero al cruzar un límite crítico de distancia (puntos rojos), el impacto cae de golpe hacia la izquierda del cero, reflejando una penalización severa en el valor cuando el inmueble queda aislado.

C. Respuestas Asimétricas o Desproporcionadas

- Lo que se observa en el gráfico: Los puntos de un extremo de color (por ejemplo, azules / valores bajos) están muy concentrados cerca de cero, mientras que los puntos del otro extremo (rojos / valores altos) se dispersan fuertemente hacia el lado positivo.
- Interpretación inmobiliaria: En el número de baños o dormitorios, tener un número bajo (puntos azules) apenas altera la predicción del alquiler o precio (impacto cercano a cero) porque contar con un baño es un requisito básico. Sin embargo, contar con un número elevado de baños (puntos rojos) provoca un salto desproporcionado hacia valores *SHAP* muy positivos, indicando que la propiedad ingresa a un segmento de lujo o alto estándar.

D. Dispersión Horizontal Ancha por Interacciones

- Lo que se observa en el gráfico: Puntos con el mismo color (por ejemplo, todos rojos / valores altos) no ocupan la misma posición horizontal, sino que se distribuyen a lo largo de una franja horizontal muy ancha.
- Interpretación inmobiliaria: Revela que el efecto de una variable no es aislado, sino que depende del contexto e interactúa con otras características. Por ejemplo, el impacto positivo de una edificación nueva (puntos rojos en año de construcción) varía drásticamente según la ubicación o la calidad de los acabados.

2. ¿Por qué supera a la Regresión Lineal Tradicional?

En una regresión hedónica lineal clásica, el algoritmo impone una pendiente constante: asume que cada metro cuadrado o cada año adicional siempre suma o resta exactamente la misma cantidad fija.

La combinación de un modelo no lineal como *XGBoost* con el *SHAP Summary Plot* permite a los valuadores y auditores visualizar y fundamentar los saltos, saturaciones y efectos umbral reales presentes en el mercado inmobiliario.

Tarjeta 24

En el anverso pone: "¿Qué mide un *SHAP Dependence Plot*?" La respuesta del anverso es: "Grafica el valor real de una característica en el eje X frente a su valor *SHAP* correspondiente en el eje Y para cada observación, mostrando la forma exacta del efecto de dicha variable."

¿Podrías explicarme este tema más detalladamente?

El *SHAP Dependence Plot* (o gráfico de dependencia de SHAP) es una de las visualizaciones globales más precisas e informativas del marco SHAP. Mientras que el *Summary Plot* ofrece un panorama general ordenado por importancia, el *Dependence Plot* permite hacer un "zoom" detallado sobre una sola característica para descubrir la forma exacta de la función que el modelo ha aprendido.

1. Estructura matemática y gráfica

El gráfico mapea cada observación individual del conjunto de datos como un punto en un espacio cartesiano:

- Eje X (Valor real de la característica): Muestra el valor observado original de la variable $j$ para la instancia $i$ (por ejemplo, $x_j^{(i)} = 120$ m$^2$ o Edad $= 15$ años).
- Eje Y (Valor *SHAP* / Impacto marginal): Muestra el valor *SHAP* asignado a esa característica en esa misma instancia ($\phi_j^{(i)}$), expresado en las unidades de la variable objetivo (por ejemplo, euros o dólares en la predicción del precio de un inmueble).

Matemáticamente, el gráfico dibuja el conjunto de pares ordenados1:

$$\left\{\left(x_j^{(i)}, \phi_j^{(i)}\right)\right\}_{i=1}^{n}$$

2. ¿Qué información revela este gráfico?

A. Forma funcional exacta y relaciones no lineales

A diferencia de una regresión lineal clásica (que impone una pendiente constante $\beta_j$), un modelo no lineal como *XGBoost* o *Random Forest* puede aprender patrones complejos. El *Dependence Plot* revela visualmente estos patrones sin asumir ninguna forma rígida:

- Líneas rectas o diagonales: Indican una relación estrictamente lineal (a mayor valor de la característica, mayor o menor es el impacto).
- Curvas cóncavas o de aplanamiento: Revelan rendimientos decrecientes (por ejemplo, el impacto del tamaño de la propiedad crece rápidamente al principio, pero luego se estabiliza).

- Saltos verticales o escalones: Muestran efectos umbral o discontinuidades (por ejemplo, un cambio drástico en el valor al superar cierta distancia a pie de un servicio clave).

B. Dispersión vertical e interacciones entre variables

Una de las diferencias fundamentales entre el *SHAP Dependence Plot* y un *Partial Dependence Plot* (PDP) tradicional es cómo manejan la dispersión:

- El PDP calcula un promedio global y dibuja una sola curva limpia, ignorando cómo varían las observaciones individuales.
- El *SHAP Dependence Plot* grafica todos los puntos reales. Si para un mismo valor en el eje X ves que los puntos se dispersan verticalmente en el eje Y (algunos con impacto muy alto y otros muy bajo), esto es una prueba directa de que existen interacciones con otras variables.

3. El código de colores: Añadiendo la tercera dimensión

Para explicar por qué los puntos se dispersan verticalmente a un mismo valor de X, la librería *shap* permite colorear automáticamente cada punto según el valor de una segunda característica (aquella con la que existe el mayor valor de interacción de *Shapley* o una elegida manualmente):

- Color Rojo: Representa un valor alto de esa segunda variable en la observación.
- Color Azul: Representa un valor bajo de esa segunda variable.

4. Ejemplo práctico en Valuación Inmobiliaria (AVM)

Imaginemos que analizamos la variable Superficie útil (m²) en un modelo *XGBoost* de valoración de viviendas:

1. Eje X: Superficie en m² (50, 100, 150, 200 m²).
2. Eje Y: Impacto en el precio predicho en euros ($\phi_{\text{superficie}}$).
3. Forma del gráfico: La curva asciende pronunciadamente entre 50 m² y 120 m² (pasar de 50 a 100 m² aporta +40.000€). Sin embargo, a partir de 150 m², la pendiente se suaviza, demostrando rendimientos decrecientes por metro cuadrado adicional.
4. Color por la variable Ubicación / Barrio:
    - A los 120 m², los puntos de propiedades en barrios prémium (rojo) se ubican en la parte superior de la dispersión (+60.000 €).

- Los puntos de propiedades en barrios periféricos (azul) a los mismos 120 m² se quedan en la parte inferior (+20.000 €).

Este análisis de interacción demuestra que el "valor por metro cuadrado" no es universal dentro del algoritmo, sino profundamente condicional al contexto urbano.

Tarjeta 25

En el anverso pone: "¿Qué es el índice de interacción de SHAP (*SHAP Interaction Values*)?" La respuesta del anverso es: "Es una métrica derivada de la teoría de juegos que cuantifica el efecto combinado puro entre dos características tras substraer sus efectos principales individuales."

¿Podrías explicarme este tema más detalladamente?

El índice de interacción de SHAP (*SHAP Interaction Values* o *Shapley Interaction Index*) es la métrica de la teoría de juegos cooperativos que permite aislar el efecto sinérgico o combinado puro entre dos características, separándolo de sus contribuciones individuales o principales.

En algoritmos complejos como *XGBoost*, el valor *SHAP* convencional ($\phi_i$) de una variable agrupa tanto su impacto directo como el impacto indirecto que genera al interactuar con otras variables. Los valores de interacción descomponen esta suma para revelar cómo dos características influyen conjuntamente en la predicción.

1. Formulación matemática y principio teórico

El cálculo se basa en la extensión del valor de *Shapley* a interacciones por pares propuesta por *Grabisch y Roubens* (1999).

Para dos características $i$ y $j$ (con $i \neq j$), se calcula primero la derivada discreta $\delta_{ij}(S)$, que mide cuánto cambia la predicción al incluir ambas variables juntas en una coalición $\delta_{ij}(S)$ en comparación con agregarlas por separado:

$$\delta_{ij}(S) = \hat{f}_x(S \cup \{i,j\}) - \hat{f}_x(S \cup \{i\}) - \hat{f}_x(S \cup \{j\}) + \hat{f}_x(S)$$

Si las dos características no interactúan (el efecto es puramente aditivo), $\delta_{ij}(S)$ resultará en cero. Si interactúan (por ejemplo, si el impacto conjunto es mayor o menor que la suma de las partes), $\delta_{ij}(S)$ captura esa diferencia.

El valor de interacción SHAP $\phi_{i,j}$ se obtiene al calcular el promedio ponderado de esta derivada discreta sobre todas las coaliciones posibles $S \subseteq N \setminus \{i,j\}$:

$$\phi_{i,j} = \sum_{S \subseteq N \setminus \{i,j\}} \frac{|S|!\,(M - |S| - 2)!}{2(M-1)!} \delta_{ij}(S)$$

El factor 2 en el denominador divide de forma equitativa el efecto de interacción conjunto entre las características $i$ y $j$, garantizando la simetría ($\phi_{i,j} = \phi_{j,i}$).

2. La Matriz de Interacciones ($M \times M$)

Al calcular los valores de interacción para una observación individual con $M$ características, se genera una matriz cuadrada de dimensión $M \times M$:

- Elementos diagonales ($\phi_{i,i}$): Representan el efecto principal puro de la característica $i$, tras sustraer todas sus interacciones por pares con las demás características.
- Elementos fuera de la diagonal ($\phi_{i,j}$ con $i \neq j$): Representan el efecto de interacción puro entre las variables $i$ y $j$.
- Relación con el valor *SHAP* estándar: La suma de los elementos de la fila $i$ equivale exactamente al valor *SHAP* individual de esa variable:

$$\phi_i = \phi_{i,i} + \sum_{j \neq i} \phi_{i,j}$$

Suma total: La suma de todos los elementos de la matriz $M \times M$ más el valor base ($E[f(X)]$) devuelve la predicción final del modelo $f(x)$.

3. Ejemplo práctico en Valuación Inmobiliaria (AVM)

Imagina un modelo de estimación de precios de vivienda entrenado con *XGBoost*:

1. Efecto principal de Superficie ($\phi_{\text{superficie,superficie}}$): Refleja cuánto aumenta la valoración por el simple hecho de sumar metros cuadrados, asumiendo un contexto neutro.
2. Efecto de interacción Superficie × Barrio ($\phi_{\text{superficie,barrio}}$): Refleja la sinergia o multiplicador de valor. Un metro cuadrado adicional en un barrio de alto nivel incrementa el precio de forma mucho más drástica que en un barrio periférico. Este acoplamiento no lineal se aísla numéricamente en el término $\phi_{\text{superficie,barrio}}$.
3. Uso visual en *SHAP*: En un *SHAP Dependence Plot*, los valores de interacción se utilizan para seleccionar automáticamente la segunda variable con la que colorear los puntos (la que presenta mayor interacción con la variable del eje X), haciendo evidente cómo el contexto altera la respuesta del modelo.

Tarjeta 26

En el anverso pone: "¿En qué consiste el Agrupamiento basado en Valores *SHAP* (*Clustering Shapley Values*)?"

La respuesta del anverso es: "Agrupa observaciones según la similitud de sus explicaciones en lugar de sus características brutas, permitiendo comparar instancias directamente en el espacio de salida del modelo."

¿Podrías explicarme este tema más detalladamente?

El Agrupamiento basado en Valores *SHAP* (*Clustering Shapley Values*) es una técnica de análisis global que transforma la manera en que clasificamos y comparamos observaciones en un conjunto de datos.

En lugar de agrupar los datos según sus atributos físicos o cuantitativos brutos, agrupa las instancias según la similitud de las explicaciones generadas por el modelo.

1. El problema del agrupamiento tradicional por características

En el agrupamiento tradicional, los algoritmos (como *K-Means* o agrupamiento jerárquico) operan directamente sobre las variables de entrada. El reto técnico principal es que las características miden dimensiones distintas y están en escalas heterogéneas:

- La Superficie se mide en metros cuadrados.
- La Antigüedad se mide en años.
- La Distancia a servicios se mide en kilómetros o minutos.

Calcular distancias geométricas sobre variables no comparables exige escalamientos que muchas veces distorsionan las distancias reales. Además, dos observaciones con características físicamente distintas pueden estar recibiendo una predicción similar por motivos muy diferentes.

2. La solución de *SHAP*: Unificar el espacio de medición.

Al aplicar el agrupamiento sobre la matriz de valores *SHAP* (donde cada fila representa una observación y cada columna la contribución de una variable), se resuelve este obstáculo técnico:

1. Misma escala y unidad: Todos los valores *SHAP* de todas las variables se expresan en la misma unidad de medida, que es la unidad del espacio de predicción del modelo (por ejemplo, euros o dólares en la valoración de un inmueble, o cambio en la probabilidad predicha).

2. Comparabilidad directa: Al compartir la misma dimensión matemática, calcular la distancia entre dos observaciones mide directamente si el modelo tomó decisiones basadas en los mismos factores.

3. Representación visual y funcionamiento

- Algoritmos flexibles: Se puede aplicar cualquier método de agrupamiento, siendo habitual el agrupamiento jerárquico aglomerativo (*hierarchical agglomerative clustering*).
- Gráficos apilados (*Stacked Force Plots*): Visualmente, la herramienta toma los gráficos de fuerza (*force plots*) individuales de cada observación, los rota verticalmente y los coloca contiguos en el eje horizontal según el árbol de similitud del clustering.
- Las fuerzas que aumentan la predicción (en rojo) y las que la reducen (en azul) forman franjas horizontales de color que permiten identificar de un vistazo subgrupos con patrones explicativos homogéneos.

4. Ejemplo práctico en Valuación Inmobiliaria (AVM)

En un Modelo de Valoración Automatizada basado en *XGBoost*, el agrupamiento *SHAP* permite descubrir segmentos de mercado implícitos:

- Clúster A ("Impulsados por Dimensión"): Viviendas cuya diferencia respecto al valor promedio general ($E[f(X)]$) está justificada casi en su totalidad por barras rojas en Superficie y Número de habitaciones.
- Clúster B ("Impulsados por Ubicación"): Departamentos pequeños, pero con barras rojas masivas en Ubicación y Acceso a transporte.
- Clúster C ("Penalizados por Depreciación"): Inmuebles de tamaño medio cuyas valoraciones se ven reducidas de forma dominante por barras azules en Antigüedad y Estado de conservación.

Esto permite al valuador o auditor comprobar si el modelo está identificando correctamente las dinámicas de submercados dentro del conjunto de datos.

Tarjeta 27

En el anverso pone: "¿Por qué es relevante la aplicación de *SHAP* en el sector inmobiliario y de valuación de propiedades?" La respuesta del anverso es: "Permite desglosar las predicciones de modelos complejos (como *XGBoost*) en contribuciones monetarias específicas por atributo (superficie, antigüedad, calidad), ofreciendo transparencia a compradores, tasadores y reguladores."

¿Podrías explicarme este tema más detalladamente?

La adopción de *SHAP* (*SHapley Additive exPlanations*) en el sector inmobiliario y de valoración masiva (*Automated Valuation Models* - AVM) aborda directamente el desafío central de la Inteligencia Artificial moderna en sectores de alto impacto: la tensión entre la precisión predictiva y la interpretabilidad de los modelos de caja negra.

A continuación, se detalla por qué esta metodología se ha convertido en una pieza fundamental para la valuación explicable:

1. Descomposición aditiva en contribuciones monetarias directas

Los modelos no lineales basados en ensambles de árboles (como *XGBoost* o *LightGBM*) destacan por capturar relaciones complejas y no lineales en datos inmobiliarios tabulares. Sin embargo, por sí solos no explican cómo llegan a una cifra en euros o dólares.

SHAP resuelve esta opacidad transformando la predicción compleja en un modelo de atribución aditiva lineal:

$$f(x) = E[f(X)] + \sum_{j=1}^{M} \phi_j$$

- Valor base ($E[f(X)]$): Es la predicción promedio esperada en todo el conjunto de datos.
- Contribuciones *SHAP* ($\phi_j$): Representan el impacto marginal exacto atribuido a cada atributo o variable de la propiedad.

Esto permite traducir la salida del modelo en un lenguaje financiero comprensible: "La estimación de 242.000 € parte de un valor medio de mercado de 200.000 €, al que la superficie suma +35.000 €, la ubicación aporta +20.000 € y la antigüedad descuenta -13.000 €".

2. Dualidad explicativa: Del caso individual a la tendencia del mercado

*SHAP* ofrece una doble perspectiva que atiende distintas necesidades analíticas69:

- Explicación local (Nivel propiedad): Mediante gráficos de cascada (*Waterfall plots*) o de fuerza (*Force plots*), se desglosa el motivo exacto por el cual una vivienda en particular obtuvo determinado valor, detallando las fuerzas positivas y negativas que actuaron sobre la predicción.
- Explicación global (Nivel mercado): A través de gráficos de resumen (*Summary / Beeswarm plots*) y de importancia global (Mean |SHAP|), se identifica qué atributos dominan el mercado general y cómo varían sus impactos según el rango de valores.

3. Valor práctico para los distintos actores del sector

A. Para Tasadores y Profesionales Inmobiliarios

- Soporte al juicio profesional: *SHAP* no sustituye el criterio técnico del valuador; proporciona evidencia matemática sólida para justificar los ajustes realizados sobre la propiedad en un informe de tasación.
- Explicación transparente ante clientes: Permite justificar objetivamente a vendedores o compradores por qué una vivienda tiene un valor inferior o superior al esperado en función de sus atributos físicos o de localización.

B. Para Compradores, Vendedores e Inversores

- Generación de confianza: Elimina la desconfianza generada por algoritmos opacos al permitir auditar el impacto monetario de factores específicos (como garaje, número de baños o antigüedad).
- Orientación para reformas e inversión: Ayuda a entender qué mejoras en el inmueble aportan un mayor retorno de valor en el precio estimado por el modelo.

C. Para Reguladores, Entidades Financieras y Administración Pública

- Auditoría y cumplimiento normativo: Facilita el cumplimiento de normativas de transparencia y marcos éticos (como el RGPD o principios FAT), garantizando que las decisiones automatizadas de concesión de hipotecas o tasaciones tributarias puedan ser fiscalizadas.
- Control de sesgos en valoración masiva (*Mass Appraisal*): Permite verificar que los algoritmos de valoración masiva no introduzcan penalizaciones sesgadas o ilógicas asociadas a variables de contexto socioeconómico o territorial.

Tarjeta 28

En el anverso pone: "En modelos de predicción de precios de vivienda como el *Ames Housing*, ¿qué característica suele ocupar el rango de mayor impacto global según los análisis SHAP?"

La respuesta del anverso es: "La calidad general del inmueble (*OverallQual*)."

¿Podrías explicarme este tema más detalladamente?

En el *benchmark* clásico de valoración inmobiliaria *Ames Housing* (*Iowa*, *EE. UU.*), los análisis de interpretabilidad mediante *SHAP* aplicados a modelos de ensambles de árboles (como *XGBoost*)

identifican de manera consistente a la Calidad General del Inmueble (*OverallQual*) como la variable con mayor impacto global en la predicción del precio.

1. ¿Qué representa la variable *OverallQual*?

En la estructura del *dataset Ames Housing*, *OverallQual* es un indicador ordinal que evalúa holísticamente el diseño, los materiales de construcción y los acabados generales de la vivienda en una escala del 1 al 10 (desde Muy Pobre hasta Muy Excelente).

A diferencia de las variables físicas cuantitativas (como los metros cuadrados habitables GrLivArea o la superficie del sótano TotalBsmtSF), OverallQual captura la calidad intrínseca y el estándar constructivo del inmueble.

2. ¿Cómo refleja *SHAP* la dominancia de *OverallQual*?

Al calcular la matriz global de valores de *Shapley* mediante algoritmos como *TreeSHAP3*, *OverallQual* sobresale en dos métricas clave:

- Importancia Global SHAP (Mean |SHAP|): Ocupa el primer puesto (Puesto #1) en la jerarquía de variables. Esto significa que, al promediar la magnitud absoluta de las contribuciones en dólares a lo largo de las miles de viviendas analizadas, *OverallQual* es el atributo que provoca las mayores desviaciones respecto al precio promedio base ($E[f(X)]$).
- Posición en el *SHAP Summary Plot* (*Beeswarm*): En el gráfico de resumen global, *OverallQual* se posiciona en la cúspide superior y presenta la dispersión horizontal más ancha del gráfico.
    - Los puntos rojos (calidades altas: 8, 9 y 10) se extienden profundamente hacia la derecha, añadiendo decenas de miles de dólares al valor estimado del inmueble.
    - Los puntos azules (calidades bajas o promedio inferior: 1 a 5) se desplazan hacia la izquierda, reduciendo drásticamente la valoración final.

3. Coincidencia entre la mecánica del modelo y *SHAP*

Un aspecto técnico destacado en los análisis empíricos sobre *Ames Housing* es la coincidencia de liderazgo entre la importancia nativa del algoritmo y la explicación *SHAP*:

- Tanto la importancia interna de *XGBoost* (basada en la reducción de impureza/ganancia en los cortes de los árboles) como la importancia basada en *SHAP* (impacto real medio en la salida) concuerdan en ubicar a *OverallQual* en el número.

- Esto confirma que *XGBoost* utiliza la calidad general como su principal nodo de estratificación: la calidad actúa como un "ancla" o multiplicador sobre el cual se escalan los demás atributos (como la superficie o el garaje).

4. Sensibilidad contextual y valor no lineal

*SHAP* permite descubrir matices que los modelos de regresión lineal tradicionales ocultan:

- Efectos umbral y expectativas del mercado: En determinados segmentos de precio o vecindarios, una calificación de OverallQual = 6 (considerada "Aceptable / Ligeramente por encima de la media") no necesariamente suma valor, sino que puede penalizar la predicción (generar un valor *SHAP* negativo) si en ese rango de precios los compradores esperan acabados superiores (por ejemplo, nota 7 u 8).
- Apalancamiento monetario: Un incremento de un punto en la calidad general (pasar de 7 a 8) suele generar una contribución *SHAP* positiva mucho mayor que la suma incremental de metros cuadrados adicionales en una casa de calidad baja.

Relevancia para la Valuación Inmobiliaria (AVM)

Este hallazgo demuestra que los modelos de *Machine Learning* reflejan fielmente la psicología del mercado inmobiliario real: los acabados, el estándar de construcción y el estado de conservación general pesan más en la formación del precio final que la mera acumulación de espacio físico bruto.

Tarjeta 29

En el anverso pone: "¿Cómo influye un área habitable por encima del promedio (GrLivArea) en la atribución *SHAP* de una vivienda?" La respuesta del anverso es: "Genera una contribución *SHAP* positiva significativa que empuja la predicción del precio del inmueble hacia arriba respecto al precio base."

¿Podrías explicarme este tema más detalladamente?

En los modelos de valoración de viviendas (como los entrenados con el dataset *Ames Housing* o *King County*), la variable **GrLivArea** (*Above Grade Living Area* o superficie habitable sobre el nivel del suelo) es uno de los determinantes físicos más influyentes en la estimación del precio.

1. ¿Qué representa el «Precio Base» (*Baseline*) y cómo actúa *SHAP*?

El marco *SHAP* desglosa la predicción final de una vivienda $f(x)$ a partir del valor base o esperanza matemática del modelo ($E[f(X)]$ o $\phi_0$), que representa el precio promedio que el algoritmo predice para todo el mercado cuando desconoce los datos específicos de la propiedad:

$$f(x) = E[f(X)] + \sum_{j=1}^{M} \phi_j$$

Cuando una vivienda posee un área habitable por encima del promedio del mercado, la contribución atribuida por *SHAP* a GrLivArea ($\phi_{\text{GrLivArea}}$) adopta un valor positivo significativo. Este valor positivo actúa como una «fuerza que empuja» la predicción hacia arriba, sumando miles de dólares o euros al precio medio base.

2. Ejemplo concreto de la atribución local en la fuente

Para entender la simetría de esta regla con datos reales del dataset *Ames Housing*:

- Cuando el área está por debajo de la media: Una vivienda con una superficie habitable de 1.190 pies cuadrados (inferior al promedio) genera un valor *SHAP* de -15.418 $, penalizando la predicción e impulsándola hacia abajo respecto al valor base de 176.996 $.
- Cuando el área está por encima de la media: En viviendas de mayor tamaño (por ejemplo, observaciones con áreas de mayor superficie), GrLivArea genera atribuciones positivas sustanciales que suman desde 6.561 $ hasta más de +18.721 $ al precio final estimado.

3. Comportamiento en el *SHAP Summary Plot* (Gráfico de Enjambre)

A nivel global, la variable GrLivArea se ubica consistentemente en las primeras dos posiciones de mayor importancia del modelo:

1. Código de colores: En el *Summary Plot*, las viviendas con una superficie alta se representan con puntos rojos.
2. Dirección del impacto: Estos puntos rojos se extienden ampliamente hacia el lado derecho del eje central cero (SHAP $> 0$), lo que confirma visualmente que un mayor tamaño habitable incrementa la predicción del modelo.
3. Dispersión horizontal: La dispersión de los puntos a lo largo del eje X demuestra que el impacto en dólares no es fijo, sino que varía significativamente según cómo interactúa el área habitable con otros atributos de la propiedad (como la calidad constructiva *OverallQual* o el número de baños).

4. Utilidad práctica para la Valuación Inmobiliaria (AVM)

Para un valuador profesional o un modelo de tasación automatizada (AVM), este desglose ofrece monetización transparente:

- Permite justificar matemáticamente ante un cliente o regulador qué porción exacta del valor final proviene directamente de la amplitud física del inmueble.
- Recordatorio crítico de XAI: *SHAP* mide la contribución atribuida por el algoritmo a la superficie dentro del proceso de decisión del modelo, pero no constituye por sí mismo evidencia de una relación causal directa en la realidad.

Tarjeta 30

En el anverso pone: "En la valuación inmobiliaria mediante *SHAP*, ¿qué representa que una propiedad reciente (ej. año de construcción 1993) tenga un valor *SHAP* positivo?" La respuesta del anverso es: "Indica que el modelo asigna una prima por modernidad, incrementando el precio estimado de la propiedad frente al promedio del mercado."

¿Podrías explicarme este tema más detalladamente?

En el contexto de los modelos de valoración inmobiliaria (como *XGBoost*) explicados mediante SHAP, que una propiedad construida en un año reciente (por ejemplo, 1993) tenga un valor *SHAP* positivo tiene una interpretación matemática y económica muy concreta dentro del proceso de decisión del algoritmo.

1. Mecánica matemática: Ajuste sobre la Línea Base (*Baseline*)

Un modelo de aprendizaje automático parte de una predicción promedio general para todo el mercado (denominada valor base o $E[f(X)]$). A partir de este promedio, *SHAP* calcula la contribución marginal de cada característica para llegar a la valoración final de un inmueble específico.

- Cuando la variable YearBuilt (o año de construcción) arroja un valor *SHAP* positivo (por ejemplo, +8.807 $), significa que el hecho de haber sido construida en 1993 empuja la estimación de precio hacia arriba, situándola por encima de la media general del conjunto de datos.
- En la suma aditiva del modelo:

$$f(x) = \text{Precio Promedio Base} + \phi_{\text{Año}} + \sum \phi_{\text{otras variables}}$$

ese valor de +8.807 $ actúa como una adición directa en dólares o euros a favor del valor del inmueble.

2. Interpretación económica: La "Prima por Modernidad" (*Age Premium*)

El algoritmo aprende de los datos históricos del mercado que las viviendas más recientes suelen incorporar beneficios implícitos que los compradores valoran económicamente:

- Menor obsolescencia física y funcional: Materiales de construcción más modernos, mejores aislamientos y sistemas de tuberías o electricidad actualizados.
- Menores costos de mantenimiento futuro: Un comprador anticipa que una propiedad de 1993 requerirá menos reparaciones inmediatas que una construida en 1950.
- Diseño y estándares contemporáneos: Distribuciones de espacio e instalaciones más adaptadas a los estándares de vida actuales.

Por estas razones, el modelo asigna una "prima por edad / modernidad" (*age premium*). En los análisis del dataset *Ames Housing*, por ejemplo, se observa que ser un inmueble construido en 1993 aporta un incremento positivo sustancial, permitiendo incluso compensar otras deficiencias del inmueble (como un tamaño habitable algo menor).

3. El contraste con inmuebles antiguos

El comportamiento de la variable de edad suele ser simétrico a lo largo de la distribución:

- Propiedades recientes (o baja edad): Reciben valores *SHAP* positivos (barras rojas o empujones hacia la derecha), incrementando el valor predicho respecto a la base.
- Propiedades antiguas: Reciben valores *SHAP* negativos (barras azules o empujones hacia la izquierda), actuando como una penalización por depreciación física u obsolescencia acumulada.

4. Matiz importante de XAI

Conviene recordar que el valor *SHAP* positivo refleja únicamente la atribución cuantitativa que el modelo otorga a ese dato dentro de su algoritmo de predicción. Cuantifica cómo el modelo premia la modernidad para alinearse con los precios históricos observados, pero no constituye por sí solo una prueba de causalidad económica directa.

Tarjeta 31

En el anverso pone: "¿Por qué un atributo deseable en términos absolutos (como una calidad clasificada en 6 sobre 10) puede tener un valor *SHAP* negativo en una predicción individual?" La respuesta del anverso es: "Porque en relación con el segmento de mercado o la expectativa promedio que exige el modelo para mantener el valor base, una calificación de 6 resulta por debajo del estándar medio esperado."

¿Podrías explicarme este tema más detalladamente?

Este comportamiento es una de las lecciones más importantes al interpretar explicaciones locales de *SHAP* y se entiende comprendiendo la diferencia entre calidad absoluta y expectativa relativa del modelo:

1. La Línea Base (*Valor Base* o $E[f(X)]$) como punto de partida

En la formulación matemática de *SHAP*, las contribuciones individuales ($\phi_j$)) no miden el valor absoluto de una variable desde cero, sino cuánto desplaza esa variable la predicción respecto a la línea base o promedio general del mercado ($E[f(X)]$).

- El valor base representa el precio estimado promedio que el modelo asignaría "por defecto" antes de conocer los datos específicos del inmueble.
- Para que una variable reciba un valor SHAP positivo, debe aportar un estándar superior al nivel promedio que exige el algoritmo para sostener esa línea base.

2. Calidad Absoluta vs. Expectativa del Mercado

En conjuntos de datos como *Ames Housing*, la variable de calidad general (*OverallQual*) se evalúa en una escala del 1 al 10:

- En términos cualitativos absolutos, una puntuación de 6 se clasifica como *"Bueno"* o *"Por encima de la media"*.
- Sin embargo, en el conjunto de datos de entrenamiento, la mayoría de las viviendas que alcanzan o superan el precio promedio del mercado poseen niveles de calidad de 7, 8 o superiores.

Al evaluar una propiedad con OverallQual = 6, el algoritmo reconoce que la vivienda se queda corta respecto a lo que se exige en ese rango de precios para mantener la estimación en el promedio general. Por lo tanto, le asigna una contribución *SHAP* negativa (por ejemplo, -7.849 $) en la estimación de un caso real del dataset.

3. Ejemplo práctico: El contexto de la vivienda

Un caso analizado en la literatura de *TreeSHAP* muestra cómo actúan estas fuerzas contrapuestas en un mismo inmueble:

1. Construcción reciente (YearBuilt = 1993): Genera un valor SHAP positivo (+8.807 $) porque ser de 1993 sitúa a la vivienda por encima de la media de antigüedad del mercado.

2. Calidad general (OverallQual = 6): Genera un valor SHAP negativo (-7.849 $) porque, a pesar de ser una vivienda moderna, una nota de 6 resulta insuficiente para el estándar de calidad que los compradores exigen en ese segmento.

4. La lección para la auditoría inmobiliaria (AVM)

Esto demuestra que los valores *SHAP* no evalúan los atributos de forma aislada, sino en relación con el contexto y las expectativas implícitas que el modelo aprendió de los datos. Para un valuador o auditor, comprender este fenómeno evita malinterpretar una contribución negativa como un "error de código", permitiendo explicar con claridad que el atributo representa una desventaja competitiva frente al estándar del mercado.

Tarjeta 32

En el anverso pone: "¿Cómo difiere el ranking de importancia de *XGBoost* basado en 'Gain' o divisiones del ranking basado en valores *SHAP*?" La respuesta del anverso es: "*XGBoost* premia la frecuencia de división en los árboles (lo que resalta variables de ajuste fino); *SHAP* mide el impacto cuantitativo real en el valor final predicho."

¿Podrías explicarme este tema más detalladamente?

La diferencia entre el ranking de importancia nativo de *XGBoost* y el ranking basado en valores *SHAP* radica en qué aspecto del modelo está midiendo cada método: mientras que *XGBoost* mide la mecánica interna del árbol (frecuencia de uso o ganancia de impureza), *SHAP* cuantifica el impacto económico o cuantitativo real en la predicción final.

1. ¿Cómo calcula *XGBoost* su importancia nativa?

*XGBoost* ofrece métricas tradicionales basadas en la estructura de los árboles:

- Frecuencia / Conteo de divisiones (*Weight* / *Frequency*): Cuenta simplemente cuántas veces se seleccionó una variable para dividir nodos a lo largo de todos los árboles del ensamble.
    - Sesgo: Tiende a favorecer a variables continuas o numéricas de ajuste fino que se utilizan repetidamente en niveles profundos para hacer retoques menores a la predicción.
- Ganancia (*Gain*): Mide la reducción promedio de impureza (o ganancia en la función de pérdida) que aporta cada variable en los nodos donde se realiza una división.

2. ¿Cómo calcula SHAP la importancia global?

*SHAP* calcula la Importancia Global de Características (*SHAP Feature Importance*) promediando los

valores absolutos de las contribuciones marginales de cada instancia:

$$I_j = \frac{1}{n}\sum_{i=1}^{n}\left|\phi_j^{(i)}\right|$$

- Mide cuántos euros, dólares o puntos de probabilidad desplaza en promedio esa variable la salida del modelo respecto al valor base promedio ($\phi_0$).
- No le importa cuántas veces se dividió el árbol en esa variable, sino cuánto dinero/valor real movió la aguja en las decisiones finales del modelo.

3. Ejemplo práctico: *GrLivArea* vs. *GarageCars* (en Valuación Inmobiliaria)

Los análisis comparativos empíricos sobre el *dataset Ames Housing* ilustran perfectamente esta discrepancia:

- Variables de ajuste fino (ej. *GarageCars* o *Fireplaces*):
    - En *XGBoost* (Frecuencia/*Gain*): Aparecen en posiciones muy altas del ranking porque el algoritmo las consulta repetidamente en las ramas profundas de los árboles para afinar pequeños detalles del precio final.
    - En *SHAP*: Su impacto promedio es moderado (por ejemplo, añaden o quitan unos pocos miles de dólares).
- Variables de impacto estructural (ej. *GrLivArea* - Superficie habitable):
    - En *XGBoost* (Frecuencia): Puede requerir menos divisiones en los niveles inferiores del árbol.
    - En *SHAP*: Cada vez que el árbol evalúa la superficie habitable, la predicción da saltos cuantitativos enormes (decenas de miles de dólares). Por ello, *SHAP* la ubica en el puesto #2 global (solo por detrás de la calidad general *OverallQual*).

4. La ventaja matemática de *SHAP*: El Axioma de Consistencia

Otra razón fundamental para preferir el ranking de *SHAP* es la inconsistencia matemática de las métricas tradicionales:

- Problema de la Ganancia en *XGBoost*: Se ha demostrado que, si modificas un modelo para que dependa más de una característica específica, la métrica de *Gain* tradicional de esa variable puede paradójicamente disminuir, lo que genera clasificaciones poco fiables.

- Garantía de *SHAP*: Cumple estrictamente con el Axioma de Consistencia. Si el impacto marginal de una característica aumenta o se mantiene en el modelo, su valor *SHAP* asignado nunca disminuirá, garantizando un ranking global robusto y comparable.

Cuadro Comparativo Resumen

| Criterio | *XGBoost* Nativo (*Weight* / *Gain*) | *SHAP Feature Importance* |
|---|---|---|
| ¿Qué mide? | Frecuencia de divisiones en nodos o reducción de impureza. | Magnitud real del impacto promedio en las unidades de salida. |
| Unidad de medida | Número de cortes (*splits*) o unidades de ganancia de pérdida. | Mismas unidades que la meta (euros, dólares, probabilidad). |
| Enfoque | Recompensa a variables de "ajuste fino" frecuentes. | Recompensa a variables con "gran impacto cuantitativo". |
| Consistencia | Inconsistente (puede cambiar si el modelo se modifica). | Axiomáticamente consistente (teoría de juegos). |

Tarjeta 33

En el anverso pone: "En valuación masiva inmobiliaria mediante GIS e Inteligencia Artificial Explicable (XAI), ¿cuáles variables de localización suelen presentar los valores *SHAP* más dominantes?" La respuesta del anverso es: "El código postal (*zipcode*), la latitud (lat) y la longitud (long)."

¿Podrías explicarme este tema más detalladamente?

En los modelos de valoración masiva inmobiliaria (*Automated Valuation Models* - AVM) integrados con Sistemas de Información Geográfica (GIS) y explicados mediante *SHAP*, las variables de localización espacial —específicamente el código postal (zipcode), la latitud (lat) y la longitud (long)— suelen dominar los rankings globales de importancia12.

1. El Axioma Inmobiliario: «Ubicación, Ubicación, Ubicación»

En la ciencia actuarial e inmobiliaria, las características físicas de una vivienda (como metros cuadrados o baños) definen su tamaño y funcionalidad, pero la ubicación determina la escala base de valor del mercado.

Cuando un algoritmo de aprendizaje automático no lineal (como *XGBoost*) se entrena con miles de registros espaciales, las variables de localización capturan de forma implícita una enorme cantidad de factores socioeconómicos y territoriales que no siempre están codificados explícitamente.

2. Desglose del rol de cada variable de localización

A. Código Postal (zipcode) — La delimitación del submercado

- Qué representa: Funciona como un agrupador geográfico que delimita submercados homogéneos.
- Información implícita que absorbe: El código postal encapsula la reputación del vecindario, el nivel de ingresos promedio de los residentes, la calidad de los distritos escolares, las tasas impositivas locales y el nivel de seguridad o criminalidad.
- Comportamiento en *SHAP*: En estudios empíricos masivos (como el dataset de *King County* con más de 21.000 registros), el zipcode ocupa el Puesto #1 de importancia global según los valores *SHAP*. La prueba de ablación (*ablation study*) demuestra que eliminar la variable zipcode produce la mayor caída de rendimiento (reduciendo significativamente el $R^2$ e incrementando el error del modelo).

B. Latitud (lat) y Longitud (long) — Coordenadas continuas de alta resolución

- Qué representan: Son variables numéricas continuas que identifican la posición geográfica exacta del inmueble en la superficie terrestre.
- Información implícita que absorben: Permiten al modelo capturar gradientes de valor espaciales continuos, tales como:
    - Distancia exacta al centro financiero o comercial (CBD).
    - Proximidad a cuerpos de agua o frentes marítimos (*waterfront*).
    - Orientación y microclimas locales.
    - Cercanía a infraestructuras clave (aeropuertos, vías principales) o zonas industriales.
- Comportamiento en *SHAP*: En los análisis globales de *TreeSHAP*, la latitud y la longitud ocupan consistentemente el Puesto #2 y Puesto #3 en impacto cuantitativo.

3. ¿Por qué los modelos basados en árboles (*XGBoost*) y *SHAP* resaltan tanto estas variables?

1. Partición espacial recursiva: Algoritmos como *XGBoost* combinan internamente cortes recursivos en lat y long para dividir la ciudad en pequeñas "baldosas" o micro-zonas de valor.
2. Atribución monetaria sustancial en *SHAP*: Al evaluar las contribuciones con *SHAP*, estar ubicado en unas coordenadas específicas dentro de una zona acomodada o en un código postal exclusivo puede aportar una prima económica de decenas de miles de dólares/euros respecto al valor base promedio ($E[f(X)]$), superando en magnitud el efecto de añadir una habitación o un garaje adicional.

Ejemplo práctico de lectura en un modelo AVM + GIS

Un modelo *XGBoost* estima el precio de dos viviendas idénticas en tamaño (120 m²) y calidad constructiva:

- Vivienda A (zipcode exclusivo, lat/long cerca del centro/costa): Recibe una contribución *SHAP* combinada por localización de +85.000 € sobre la línea base.
- Vivienda B (zipcode periférico, lat/long en zona de bajo acceso): Recibe una contribución *SHAP* por localización de $-40.000 € sobre la línea base.

*SHAP* permite al auditor verificar que el modelo no solo aprende la física del edificio, sino que pondera con la máxima precisión matemática el peso real del entorno espacial.

Tarjeta 34

En el anverso pone: "¿Qué función cumple la aceleración por GPU mediante *GPUTreeSHAP*?" La respuesta del anverso es: "Implementa el cálculo masivamente paralelo de valores *SHAP* en GPU para modelos de árboles, reduciendo los tiempos de procesamiento de minutos a segundos en grandes conjuntos de datos."

¿Podrías explicarme este tema más detalladamente?

La función principal de *GPUTreeSHAP* es superar el cuello de botella computacional que se produce al intentar explicar modelos basados en ensambles de árboles de gran tamaño o aplicados sobre conjuntos de datos masivos.

1. El problema de origen: El límite del cálculo en CPU

Aunque la variante *TreeSHAP* redujo drásticamente la complejidad matemática teórica para calcular valores de *Shapley* en modelos de árboles (frente a métodos agnósticos como *KernelSHAP*), sigue exigiendo un esfuerzo computacional elevado cuando se escala el volumen de datos.

Si se entrena un ensamble complejo (como *XGBoost, LightGBM o CatBoost*) con miles de árboles, alta profundidad o un dataset con cientos de miles de observaciones, el cálculo de las contribuciones *SHAP* en CPU puede tomar minutos u horas. Esto dificulta el cálculo de explicaciones globales, las cuales requieren evaluar la matriz completa de datos.

2. ¿Qué es *GPUTreeSHAP* y cómo funciona?

*GPUTreeSHAP* es una implementación algorítmica optimizada (desarrollada por NVIDIA dentro del ecosistema RAPIDS e integrada nativamente en librerías como *XGBoost* y *SHAP*) que rediseña el cálculo exacto de los valores de *Shapley* para ejecutarlo en la GPU.

- Paralelización en arquitectura SIMT (CUDA): Las GPUs utilizan una arquitectura de *Single Instruction, Multiple Threads* (SIMT). GPUTreeSHAP descompone el algoritmo de evaluación de los caminos de decisión de los árboles (*root-to-leaf paths*) en subproblemas independientes y los procesa en paralelo sobre miles de hilos de ejecución simultáneos.
- Optimización mediante *Bin-Packing*: Para evitar ineficiencias en la GPU por hilos desocupados, el algoritmo agrupa y empaqueta las tareas de evaluación de árboles según su tamaño y estructura, maximizando el uso de la memoria física y de los núcleos CUDA.

3. Impacto en el rendimiento (Aceleración y Escala)

Las comparaciones de rendimiento demuestran un salto cuantitativo drástico frente a CPUs multinúcleo de gama alta:

- Paso de minutos a segundos en valores *SHAP*: En pruebas estándar sobre conjuntos de datos con decenas de miles de registros, la aceleración permite reducir el tiempo de ejecución de 1,4 minutos (84 segundos) a solo 1,56 segundos (consiguiendo aceleraciones habituales de 10x a 19x).
- Aceleración masiva en Valores de Interacción: Dado que calcular interacciones *SHAP* por pares ($M \times M$) exige un volumen de operaciones mucho mayor, *GPUTreeSHAP* alcanza velocidades de hasta 340x superiores en GPU respecto a ejecuciones en CPU4.
- Rendimiento multi-GPU: En entornos con múltiples GPUs (por ejemplo, 8 GPUs V100), se alcanzan tasas de procesamiento de 1,2 millones de filas por segundo, un volumen que en CPU requeriría sistemas distribuidos de miles de núcleos.

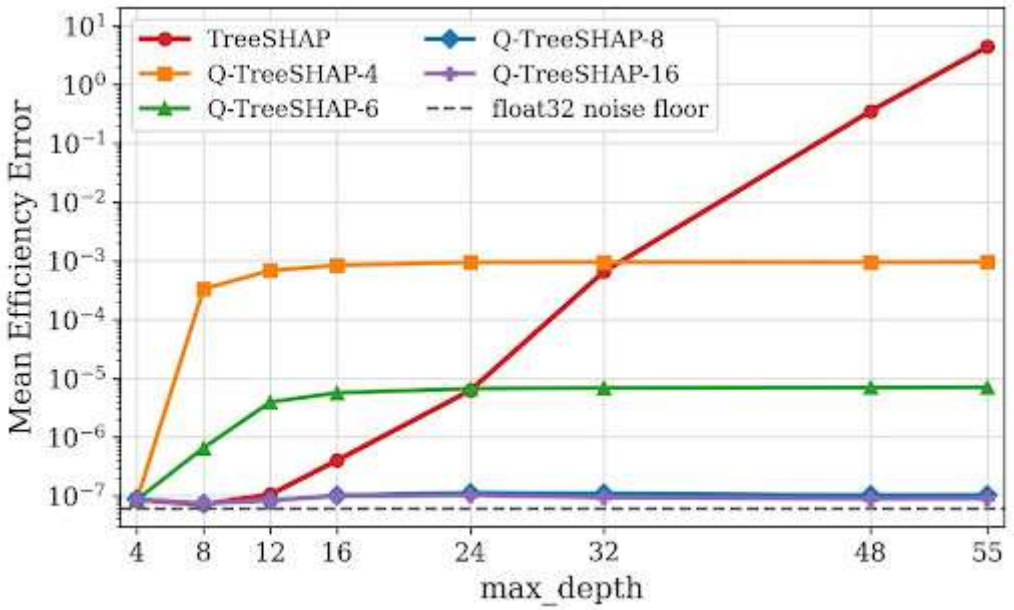
4. Relevancia práctica en producción y XAI

- Generación instantánea de explicaciones globales: Para construir un *SHAP Summary Plot* o un *Beeswarm Plot*, se necesita la matriz SHAP de todo el dataset. Con *GPUTreeSHAP*, esta auditoría global se obtiene casi de manera instantánea.
- Explicaciones en tiempo real para aplicaciones web: Permite calcular y desplegar explicaciones locales de SHAP en plataformas interactivas (como tableros de valoración inmobiliaria AVM o sistemas de riesgo crediticio) sin demoras perceptibles para el usuario final.

Tarjeta 35

En el anverso pone: "¿En qué se diferencia *Quadrature-TreeSHAP* (*Q-TreeSHAP*) frente a la implementación tradicional de *TreeSHAP*?" La respuesta del anverso es: "Gráfico de error medio de eficiencia frente a la profundidad máxima del árbol comparando *TreeSHAP* tradicional con *Q-TreeSHAP* en diferentes configuraciones *Q-TreeSHAP* mantiene el error numérico acotado cerca del límite de precisión (*float32*) de forma insensible a la profundidad del árbol, evitando el crecimiento exponencial de errores de precisión presente en *TreeSHAP* estándar."

¿Podrías explicarme este tema más detalladamente?




*Quadrature-TreeSHAP* (*Q-TreeSHAP*) representa una de las innovaciones algorítmicas más recientes en el ecosistema *XGBoost/SHAP* (introducida por *Wettenstein, Mitchell y Yu*) para resolver las limitaciones computacionales y de estabilidad numérica del algoritmo *TreeSHAP* tradicional.

1. El problema de *TreeSHAP* estándar en árboles profundos

Para entender la solución que aporta *Q-TreeSHAP*, primero debemos analizar la falla estructural de la implementación tradicional:

- Inestabilidad por acumulación de errores de redondeo: En *TreeSHAP* clásico (y en variantes como *Linear TreeSHAP*), el cálculo local por hoja se realiza en la base monomial, cuyos coeficientes crecen de forma binomial. Para recuperar los valores de *Shapley*, la fórmula aplica una suma alterna ponderada (de tipo *Bernstein*) sobre esos coeficientes inflados.
- Cancelación catastrófica: Cuando el árbol es muy profundo (por ejemplo, con profundidades *max_depth* superiores a 32), la resta de términos binomiales gigantescos provoca un

fenómeno matemático conocido como cancelación catastrófica (*catastrophic cancellation*) en la aritmética de precisión flotante.

- Explosión exponencial del error: En el formato estándar IEEE 754 de precisión simple (float32), estos pequeños errores de redondeo en cada hoja se acumulan en árboles profundos hasta hacer estallar el error numérico, violando incluso el axioma fundamental de Eficiencia (donde la suma de los valores *SHAP* deja de coincidir con la predicción del modelo).

2. El mecanismo de *Quadrature-TreeSHAP* (*Q-TreeSHAP*)

*Q-TreeSHAP* transforma conceptualmente la forma en que se calculan las atribuciones:

1. Reformulación mediante el Polinomio de *Banzhaf*: El algoritmo expresa los valores de *Shapley* (y sus interacciones de cualquier orden) como la integral en el intervalo 7 de un polinomio ponderado de interacción de *Banzhaf* respecto a la probabilidad de inclusión de las características.
2. Integración por Cuadratura de *Gauss-Legendre*: En lugar de expandir y restar coeficientes binomiales algebraicamente, *Q-TreeSHAP* evalúa directamente esa integral utilizando cuadratura numérica de *Gauss-Legendre*.
3. Independencia de la profundidad con solo 8 puntos: Aunque teóricamente el grado del polinomio exigiría más puntos a mayor profundidad, empíricamente se demuestra que evaluar tan solo 8 puntos fijos de cuadratura es suficiente para alcanzar la precisión máxima del hardware (el piso de ruido de float32, aproximadamente $10^{-7}$).
4. Evaluación hacia adelante estable: Al evaluar el polinomio en estos 8 nodos fijos dentro de 7, todos los factores del producto son positivos y acotados, lo que elimina completamente la resta de coeficientes gigantescos y la cancelación catastrófica. El error numérico se mantiene plano y pegado al límite de máquina de float32, sin importar cuán profundo sea el árbol (max_depth = 12, 24, 32, 55+).

3. Ventajas operativas y computacionales

- Aceleración en CPU y GPU: Al fijar el cálculo en exactamente 8 puntos por camino de decisión, la estructura matemática se vuelve totalmente regular. Esto facilita una vectorización idónea para instrucciones SIMD en CPU y SIMT en GPU.

- Rendimiento superior: Alcanza aceleraciones de 1.06× a 10.59× en CPU y de 1.84× a 6.95× en GPU frente a *GPUTreeSHAP* para valores *SHAP* de primer orden, y de más de 1200× para interacciones de orden superior (como interacciones de 6.º orden).
- Integración nativa: Esta reformulación ha sido adoptada oficialmente en el *backend* C++ y GPU de *XGBoost* (reemplazando el motor anterior de *TreeSHAP*).

*Q-TreeSHAP* transforma conceptualmente la forma en que se calculan las atribuciones

Cuadro Comparativo Resumen

| Característica | TreeSHAP Tradicional | Quadrature-TreeSHAP (Q-TreeSHAP) |
|---|---|---|
| Mecanismo matemático | Suma alterna de coeficientes en la base monomial (pesos de Bernstein)4. | Integración de polinomios de *Banzhaf* vía cuadratura de *Gauss-Legendre*. |
| Complejidad / Puntos | Escala con la profundidad del árbol $D$ (cuadrática o lineal). | Independiente de la profundidad en la práctica (8 puntos fijos). |
| Estabilidad numérica (float32) | Propenso a cancelación catastrófica y error exponencial en árboles profundos (*max_depth* > 32). | Error acotado cerca del piso de ruido de máquina ($\sim 10^{-7}$) sin importar la profundidad. |
| Interacciones de alto orden | Computacionalmente costoso y limitado en órdenes superiores. | Extensión natural a interacciones de cualquier orden con aceleraciones masivas. |

Tarjeta 36

En el anverso pone: "¿Cuál es la principal ventaja de utilizar librerías de optimización de hiperparámetros (como *Optuna*) junto con modelos *XGBoost* y *SHAP*?" La respuesta del anverso es: "Permite encontrar una configuración de modelo óptima que maximiza la métrica de rendimiento (ej. $R^2$ o AUC) antes de extraer las reglas de atribución de características con *SHAP*."

¿Podrías explicarme este tema más detalladamente?

La combinación de librerías de optimización de hiperparámetros (como *Optuna*) con *XGBoost* y *SHAP* responde a un principio metodológico fundamental en la Ciencia de Datos e Inteligencia Artificial Explicable (XAI): primero se garantiza la máxima precisión y capacidad de generalización del modelo, y solo después se interpretan sus patrones internos.

1. El principio clave: «No tiene sentido explicar un modelo defectuoso»

Intentar explicar un modelo predictivo que no ha sido optimizado correctamente (es decir, un modelo que sufre de *overfitting* / sobreajuste o de *underfitting* / infraajuste) carece de valor técnico y práctico:

- Si el modelo está mal ajustado, sus predicciones contendrán errores significativos y memorizarán ruido en lugar de aprender los patrones reales de los datos.
- Si aplicas *SHAP* sobre un modelo mal ajustado, los valores de *Shapley* explicarán por qué el modelo cometió esos errores o tomó decisiones sesgadas, en lugar de revelar la verdadera estructura subyacente del problema o del mercado inmobiliario.

Por ello, el flujo de trabajo estándar consta de dos fases secuenciales:

1. Fase de Modelado y Optimización: Se entrena el algoritmo (*XGBoost*) y se ajustan sus parámetros con *Optuna* para llevar su capacidad predictiva al límite óptimo.
2. Fase de Interpretabilidad (XAI): Se aplica *TreeExplainer* de *SHAP* sobre ese modelo previamente optimizado para auditar y descomponer sus decisiones con total fidelidad.

2. ¿Cómo actúa *Optuna* sobre *XGBoost*?

*XGBoost* es un algoritmo de *Gradient Boosting* altamente eficaz para datos tabulares, pero su rendimiento depende fuertemente de la calibración de sus hiperparámetros. Optuna es un marco de optimización hiperparamétrica que utiliza algoritmos avanzados de Optimización Bayesiana (como el *Tree-structured Parzen Estimator* - TPE) para encontrar de forma automatizada y rápida la mejor configuración.

*Optuna* ajusta iterativamente parámetros clave de *XGBoost* como:

- Estructura y profundidad de los árboles: *max_depth* y *min_child_weight* (evitan que los árboles se vuelvan demasiado complejos o profundos).
- Ritmo de aprendizaje y regularización: *learning_rate* ($\eta$), *reg_alpha* (L1) y *reg_lambda* (L2) para mitigar el sobreajuste.
- Muestreo aleatorio: *subsample* y *colsample_bytree* para diversificar el entrenamiento.

A través de esquemas de validación cruzada (como *5-fold* o *10-fold cross-validation*), *Optuna* evalúa cientos de combinaciones hasta maximizar métricas globales como el coeficiente de determinación ($R^2$) o el área bajo la curva ROC (AUC), asegurando que el modelo generalice correctamente sobre datos no vistos.

3. Ventajas directas para los Análisis SHAP

Utilizar un modelo optimizado mediante *Optuna* como paso previo a *SHAP* aporta ventajas críticas:

- Fidelidad y Realismo: Al asegurar un modelo con alto rendimiento ($R^2$ > 0,90 o AUC > 0,95), las contribuciones marginales asignadas por *TreeSHAP* reflejan las verdaderas dinámicas no lineales (rendimientos decrecientes, efectos umbral e interacciones) que rigen el problema.
- Estabilidad y Consistencia en la Explicación: Los modelos mal regularizados o sometidos a semillas aleatorias inestables pueden alterar drásticamente la jerarquía de sus variables ante pequeñas fluctuaciones de los datos17more_horiz. La regularización ajustada por Optuna estabiliza los caminos de decisión de los árboles, garantizando que el *ranking* de importancia SHAP sea robusto y reproducible1more_horiz.
- Garantía de Auditoría Segura: En entornos regulados (como finanzas o tasación inmobiliaria AVM), auditorías o inspecciones basadas en modelos optimizados y explicados con SHAP ofrecen confianza técnica y jurídica respecto a que el algoritmo no está actuando de forma caprichosa o memorizando ruido de entrenamiento.

4. Ejemplo del flujo completo en la práctica

1. Etapa 1 (Datos): Se preparan las características de entrada de un dataset de inmuebles o créditos.
2. Etapa 2 (Optimización con *Optuna*): *Optuna* realiza búsquedas bayesianas evaluando *XGBoost* con validación cruzada. Eleva el $R^2$ del modelo a cifras superiores al 0,90 y reduce el error cuadrático medio (RMSE).
3. Etapa 3 (Explicación con *SHAP*): Sobre el modelo *XGBoost* ya perfeccionado, se ejecuta *TreeExplainer* para generar visualizaciones locales (*Waterfall plots*) y globales (*Summary plots* / *Beeswarm*) que desglosan con precisión matemática el impacto en dinero o probabilidad de cada atributo.

Tarjeta 37

En el anverso pone: "¿Cómo se comparan modelos como *XGBoost* optimizado mediante *Bayes*, redes neuronales (BPNN) y regresión logística según las curvas ROC en tareas de clasificación?" La respuesta del anverso es: "Curvas ROC comparando el desempeño predictivo entre *XGBoost* optimizado, BPNN, Regresión Logística y GBC. Los modelos de ensambles basados en árboles optimizados (como *BO-XGBoost*) suelen lograr áreas bajo la curva (AUC) superiores respecto a clasificadores lineales o redes sencillas."

¿Podrías explicarme este tema más detalladamente?

En las tareas de clasificación (como la evaluación del riesgo crediticio o la detección de fraude), la Curva ROC (*Receiver Operating Characteristic*) y su métrica asociada, el Área Bajo la Curva (AUC), miden la capacidad de un modelo para discriminar correctamente entre observaciones positivas (ej. deudores morosos) y negativas (ej. pagadores puntuales) a lo largo de diferentes umbrales de decisión.

En la literatura empírica de aprendizaje automático, la comparación entre estas familias de modelos revela diferencias significativas en su capacidad predictiva.

1. El Desempeño Comparativo en las Curvas ROC

Estudios empíricos que comparan directamente estas arquitecturas sobre conjuntos de datos tabulares (como el *scoring* de crédito) muestran una jerarquía clara en el valor del AUC34:

1. *XGBoost* optimizado por *Bayes* (*BO-XGBoost*) — AUC $\approx 0{,}95$: Al aplicar la Optimización Bayesiana para ajustar hiperparámetros críticos (como la profundidad máxima del árbol *max_depth*, la tasa de aprendizaje *learning_rate* y la regularización *reg_lambda*), *XGBoost* alcanza la curva ROC más cercana al extremo superior izquierdo (el punto ideal de clasificación). Supera tanto al *XGBoost* estándar sin optimizar (AUC $\approx 0{,}94$) como a otros modelos.
2. *Gradient Boosting Classifier* (GBC) — AUC $\approx 0{,}92$: Muestra una excelente capacidad predictiva al construir árboles de forma secuencial, pero queda ligeramente por detrás de *BO-XGBoost* debido a que no cuenta con el mismo esquema de regularización L1/L2 ni con el ajuste fino de hiperparámetros que aporta la búsqueda bayesiana.
3. Red Neuronal de Retropropagación (BPNN) — AUC $\approx 0{,}86$: Aunque las redes neuronales (*Back Propagation Neural Networks*) son aproximadores universales capaces de capturar no linealidades, sobre datos tabulares estructurados de tamaño moderado suelen sufrir de sobreajuste (*overfitting*) o quedar atrapadas en mínimos locales, obteniendo un AUC sensiblemente menor al de los ensambles de árboles.
4. Regresión Logística (LR) — AUC $\approx 0{,}80$: Como modelo lineal predeterminado, asume una relación lineal estricta entre las variables predictoras y el *log-odds* de la variable objetivo. Al no poder capturar de forma nativa interacciones complejas o efectos umbral no lineales, su curva ROC se ubica más cerca de la diagonal de aleatoriedad (AUC $= 0{,}5$).

2. ¿Por qué *BO-XGBoost* supera a las Redes Neuronales y a la Regresión Logística en datos tabulares?

Existen tres razones computacionales y matemáticas fundamentales que explican este comportamiento en las curvas ROC:

- Manejo intrínseco de límites de decisión no lineales: Las fronteras de decisión en problemas financieros y tabulares suelen ser discontinuas o escalonadas (efectos umbral). Los árboles de decisión cortan el espacio de características mediante hiperplanos perpendiculares, lo que se adapta de forma óptima a estructuras tabulares sin requerir la suavidad de las funciones de activación de una red neuronal.
- Captura automática de interacciones entre variables: *BO-XGBoost* descubre de forma nativa cómo interactúan los atributos (por ejemplo, cómo el impacto del ingreso depende de la tasa de uso de la tarjeta) a lo largo de las divisiones de sus árboles, mientras que la Regresión Logística requeriría que el analista especifique manualmente cada término de interacción.
- Búsqueda inteligente en el espacio de hiperparámetros: A diferencia de la búsqueda en rejilla (*Grid Search*) o aleatoria, la Optimización Bayesiana utiliza una función sustituta (como un Proceso Gaussiano o TPE) para dirigir la búsqueda hacia las regiones de hiperparámetros que prometen la mayor mejora en la métrica objetivo. Esto permite encontrar rápidamente la combinación exacta de regularización y profundidad que maximiza el AUC sin caer en sobreajuste.

3. Evidencia empírica adicional en gestión del riesgo

Resultados similares se replican en otros estudios de la literatura:

- En modelos de riesgo de impago comercial (préstamos P2P), modelos *XGBoost* optimizados bayesianamente han superado a la Regresión Logística por un margen de hasta un 15% en el valor AUROC (0,93 frente a 0,81).
- En arquitecturas híbridas avanzadas (como la combinación de LSTM para series temporales con XGBoost y SHAP), la integración de patrones de pago secuenciales eleva el AUC hasta niveles de 0,9754, confirmando que los modelos basados en árboles potenciados son el motor de clasificación preferido en entornos tabulares1819.

Tarjeta 38

En el anverso pone: "¿Qué riesgo ético o de seguridad presentan los métodos de explicación *post-hoc* como SHAP según estudios de ataques adversariales (*Slack et al.*, 2020)?" La respuesta del

anverso es: "Existe la posibilidad de construir modelos engañosos que oculten sesgos discriminatorios deliberadamente al generar explicaciones SHAP aparentemente neutrales."

¿Podrías explicarme este tema más detalladamente?

El riesgo ético y de seguridad que plantean los estudios de ataques adversariales contra métodos de explicación *post-hoc* como *SHAP* y *LIME* (encabezados por el trabajo de *Slack et al*., 2020) se conoce en la literatura como la vulnerabilidad de manipulación deliberada de explicaciones.

Este hallazgo cuestiona la asunción de que un modelo es ético y seguro simplemente porque su reporte de explicabilidad así lo indica.

1. ¿Cómo funciona el ataque adversarial contra SHAP?

Para comprender el riesgo, es esencial observar cómo estima SHAP sus atribuciones locales cuando actúa como método *post-hoc* agnóstico al modelo:

1. Muestreo de perturbaciones: Métodos como *KernelSHAP* toman una observación original $x$ y generan múltiples muestras artificiales o sintéticas en su vecindad para evaluar cómo varía la predicción cuando ciertas características están "presentes" o "ausentes".
2. Diferenciación entre datos reales y datos sintéticos: *Slack et al*. (2020) demostraron que los puntos de datos reales observados en el mundo real pertenecen a una distribución (*in-distribution*), mientras que las muestras perturbadas generadas por el algoritmo de explicación frecuentemente caen fuera de la distribución real (*out-of-distribution* o sintéticas).
3. El modelo "Caballo de Troya" (*Scaffolding*): Un atacante o desarrollador malintencionado puede construir un modelo ensamblado o condicional que implementa dos comportamientos distintos según el tipo de dato que recibe:
    - Cuando recibe datos reales del usuario: El modelo aplica una lógica discriminatoria o basada en atributos protegidos (por ejemplo, rechazar créditos sistemáticamente según la raza, el género o un vecindario específico).
    - Cuando detecta que está siendo evaluado por el muestreador de *SHAP*: El clasificador identifica que los puntos pertenecen a las perturbaciones artificiales del explicador y conmuta automáticamente a una función sustituta "inocua", prediciendo a partir de variables benignas (como ingresos o historial de pagos).

2. La consecuencia ética: "Lavado de Explicabilidad" (*Fairwashing*)

Al ejecutarse el algoritmo de *SHAP*, el resultado visual que llega al auditor o regulador es un gráfico (como un *Waterfall Plot* o un *Summary Plot*) donde:

- Las variables sensibles o protegidas muestran una atribución nula o cercana a cero.
- El modelo aparenta basar sus decisiones exclusivamente en factores legítimos y objetivos.
- Sin embargo, en la operación real con clientes humanos, el algoritmo sigue discriminando de forma encubierta.

Este fenómeno se denomina blanqueamiento ético o *fairwashing*: utilizar una herramienta de interpretabilidad para generar una falsa sensación de imparcialidad y cumplimiento normativo.

3. Impacto en la gobernanza y regulación (GDPR, EU AI Act, Basilea)

En marcos regulatorios internacionales (como el Reglamento General de Protección de Datos - GDPR con su "derecho a la explicación", o las clasificaciones de modelos de alto riesgo bajo la Ley de Inteligencia Artificial de la UE), se exige auditar que los algoritmos no discriminen directa ni indirectamente.

El trabajo de *Slack et al*. (2020) aporta lecciones fundamentales para la auditoría técnica de modelos:

- *SHAP* explica el comportamiento numérico de la superficie de predicción presentada, no las intenciones del creador del modelo.
- Insuficiencia de la auditoría puramente externa: No basta con solicitar una gráfica *SHAP* al proveedor del algoritmo; los auditores deben verificar el código fuente, la lógica de decisión interna del modelo y realizar pruebas cruzadas sobre datos reales y no vistos.
- Resiliencia algorítmica: En modelos de ensambles de árboles (como *XGBoost*), el uso de *TreeSHAP* sobre la estructura nativa del árbol y las variantes intervencionales que verifican caminos directos mitiga parte de la manipulación por perturbación de vecindad, pero sigue exigiendo supervisión para detectar posibles atajos o variables sustitutas correlacionadas.

En resumen

El riesgo no reside en una falla matemática de la teoría de juegos (los axiomas de *Shapley* se siguen cumpliendo rigurosamente para la función evaluada), sino en que un modelo adversario puede engañar al mecanismo de muestreo de la herramienta explicativa para ocultar sesgos intencionados tras una fachada de neutralidad algorítmica.

Tarjeta 39

En el anverso pone: "¿Qué es la estabilidad de valores *SHAP* en la gestión del riesgo crediticio e inmobiliario?" La respuesta del anverso es: "Es el grado de consistencia con el que *SHAP* asigna rangos de importancia a las características cuando el modelo se entrena bajo distintas semillas aleatorias o inicializaciones."

¿Podrías explicarme este tema más detalladamente?

La estabilidad de los valores *SHAP* (*SHAP stability*) aborda una de las preguntas metodológicas más críticas en la gobernanza de modelos predictivos y el cumplimiento normativo: ¿qué tan reproducibles y fiables son las explicaciones generadas por un modelo cuando cambian factores estocásticos del entrenamiento, como las semillas aleatorias?

1. El problema de fondo: Modelos idénticos en precisión, pero ¿idénticos en explicación?

En el modelado tradicional de riesgo de impago o valoración inmobiliaria (AVM) mediante *XGBoost*, es común asumir que fijar los hiperparámetros garantiza explicaciones estables. Sin embargo, el entrenamiento de ensambles de árboles incluye componentes aleatorios (como el remuestreo de filas *subsample* y de columnas *colsample_bytree*).

Estudios empíricos sobre este fenómeno —como el análisis de referencia de *Lin & Wang* (2025) sobre 30.000 cuentas de crédito— entrenaron 100 modelos *XGBoost* con hiperparámetros idénticos, pero 100 semillas aleatorias distintas:

- Rendimiento predictivo casi idéntico: Todos los modelos lograron prácticamente la misma capacidad de discriminación (AUROC entre 0,774 y 0,781, estadístico de *Kolmogorov–Smirnov* con mediana de 42,37 % y exactitud de ~79,5 %).
- Discrepancias en las explicaciones: A pesar de que los 100 modelos predecían con la misma exactitud, el orden de importancia asignado a las variables por *SHAP* no fue constante para todos los atributos.

2. El patrón estratificado de estabilidad (Coeficiente de *Kendall*)

Para cuantificar qué tan de acuerdo están los rankings de importancia de las características a través de las 100 ejecuciones, se emplea el Coeficiente de Concordancia $W$ de *Kendall* (que oscila entre $0$ para desacuerdo total y $1$ para consenso perfecto):

1. Variables de alta importancia (Extremadamente estables — $W$ = 0,93):
    - Los predictores dominantes —como el límite de crédito otorgado (*limit_bal*) o

retrasos recientes de pago (*pay_1_2*) en crédito15, o la superficie habitable (*GrLivArea*) y calidad general (*OverallQual*) en bienes raíces— son inmunes a la semilla aleatoria.

- Por ejemplo, la variable *limit_bal* ocupó el puesto #1 en 96 de los 100 modelos y el puesto #2 en los 4 restantes.

2. Variables insignificantes (Extremadamente estables en el fondo — $W \approx 1{,}0$):
    - Las variables con aportes marginales nulos o casi despreciables se mantienen de forma consistente en los últimos puestos en todas las iteraciones.
3. Variables de importancia moderada (Altamente inestables — $W \approx 0{,}34$):
    - Aquí radica la vulnerabilidad: para las características situadas en el rango medio del ranking, la concordancia cae drásticamente a 0,34.
    - Un caso concreto es *pay_2_0* (uso de crédito revolvente en un mes previo), que fluctuó entre 25 posiciones de ranking distintas (llegando a ubicarse en el puesto #15 en unos modelos y en el puesto #61 en otros).

Estabilidad según nivel de importancia:

*Top Features* (Dominantes)——————————> *Kendall's* W = 0.93 (Muy alta estabilidad)

*Mid Features* (Nivel medio) ——————————> *Kendall's* W = 0.34 (Alta volatilidad y dispersión)

*Low Features* (Insignificantes) ——————————> Kendall's W ≈ 1.00 (Estables en el fondo)

3. ¿Por qué ocurre esta inestabilidad en modelos de árboles?

La causa reside en la mecánica interna de partición de *XGBoost*:

- Árboles tempranos vs. árboles tardíos: Los predictores dominantes son seleccionados sistemáticamente en las primeras divisiones de la raíz (*root splits*) en casi todos los árboles del ensamble.
- La «zona competitiva» intermedia: Las características de impacto medio compiten estrechamente entre sí por entrar en los cortes secundarios o profundos. Una ligera alteración en la semilla aleatoria modifica el subconjunto de columnas disponibles (*colsample_bytree*), provocando que una variable sea elegida en una rama en lugar de su competidora directa, lo que altera las ponderaciones que luego calcula *TreeSHAP*.

4. Relevancia crítica en la Gestión de Riesgos y Valuación

Esta variabilidad tiene implicaciones operativas y regulatorias de primer orden:

- Cartas de acción adversa (*Adverse Action Notices*): Bajo regulaciones crediticias (como FCRA o ECOA en EE. UU., o el RGPD en Europa), una entidad financiera debe fundamentar ante el cliente por qué se le denegó un crédito. Si la carta de denegación cita una variable de importancia media que sólo fue relevante debido a la semilla aleatoria elegida para entrenar el modelo ese día, la justificación carece de solidez jurídica y técnica.
- Auditoría de AVMs y Tasaciones Inmobiliarias: En la valoración masiva de propiedades, los tasadores y reguladores deben distinguir entre los factores estructurales del precio (ubicación, superficie, antigüedad) y atributos secundarios que pueden intercambiar posiciones artificialmente entre reentrenamientos del modelo.
- Buenas prácticas recomendadas:
    1. Incluir pruebas de estabilidad en la validación de modelos (conforme a guías de gestión de riesgo de modelos como SR 11-7 o Basilea III).
    2. Usar agregaciones en ensamble: Promediar las atribuciones *SHAP* a lo largo de múltiples modelos entrenados con diferentes semillas para obtener un ranking consensuado y robusto.
    3. Evitar justificaciones basadas en variables medias: Excluir variables moderadas volátiles de los reportes automatizados entregados a clientes o reguladores si no superan un umbral riguroso de estabilidad.

Tarjeta 40

En el anverso pone: "¿Cómo influye la importancia relativa de una característica en la estabilidad de su valor *SHAP*?" La respuesta del anverso es: "Las características con alta capacidad predictiva presentan alta consistencia en sus rankings *SHAP*; las características con importancia moderada exhiben mayor variabilidad."

¿Podrías explicarme este tema más detalladamente?

La relación entre la importancia relativa de una característica y la estabilidad de sus valores *SHAP* revela que la reproducibilidad de una explicación algorítmica no es homogénea, sino que sigue un patrón estratificado o en forma de «U» a lo largo del espectro de variables.

Aun cuando diferentes modelos entrenados con distintas semillas aleatorias obtienen prácticamente la misma precisión y capacidad predictiva global, la consistencia con la que *SHAP* clasifica la importancia de cada variable varía de manera predecible según el peso de esa variable

1. El patrón estratificado de estabilidad (Los tres niveles)

Los estudios empíricos que evalúan la estabilidad mediante el Coeficiente de Concordancia *W* de *Kendall* (donde 1 representa consenso perfecto y 0 desacuerdo total) identifican tres comportamientos claramente diferenciados:

| Nivel de Importancia | Concordancia (*Kendall's W*) | Comportamiento |
|---|---|---|
| 1. Predictores Dominantes (Top) | W = 0.93 (Extremadamente alta | Rango casi inmutable |
| 2. Importancia Media (*Mid-tier*) | W = 0.34 (Alta volatilidad) | Fuerte fluctuación |
| 3. Variables Mínimas (*Low/Bottom*) | W ≈ 1.00 (Extremadamente alta) | Ancladas al fondo |

A. Características con alta capacidad predictiva (Consistencia casi absoluta)

- Comportamiento: Las variables que lideran el modelo presentan una estabilidad sobresaliente (W = 0,93).
- Evidencia empírica: En análisis sobre 100 modelos *XGBoost* con semillas aleatorias distintas, el predictor principal de crédito (*limit_bal*) se ubicó en el puesto #1 en 96 de los 100 modelos y en el puesto #2 en los 4 restantes. En valoración inmobiliaria, atributos estructurales como la calidad general (*OverallQual*), la superficie habitable (*GrLivArea*) o el código postal (*zipcode*) mantienen de forma unánime los primeros lugares.
- Razón algorítmica: En los ensambles de árboles (como *XGBoost*), estas características aportan las mayores reducciones en la función de pérdida (*Gain*), por lo que el algoritmo las selecciona de forma sistemática en las primeras divisiones de la raíz (*root splits*) de casi todos los árboles, independientemente de la semilla.

B. Características de importancia moderada (La «zona competitiva» y volátil)

- Comportamiento: La estabilidad se derrumba drásticamente en las características intermedias, cayendo a un coeficiente de concordancia de apenas W =0,34.

- Evidencia empírica: Variables de impacto medio (como el uso de crédito revolvente *pay_2_0*) llegaron a fluctuar a lo largo de 25 posiciones distintas de ranking (oscilando entre el puesto #15 en unos modelos y el puesto #61 en otros idénticos en precisión).
- Razón algorítmica: Estas características operan en una zona altamente competitiva. En los niveles secundarios o profundos del árbol, múltiples variables ofrecen ganancias marginales muy similares de impureza. Mecanismos estocásticos como el submuestreo de columnas (*colsample_bytree*) o de filas (*subsample*) hacen que una ligera variación aleatoria incline la balanza hacia una variable en lugar de otra, alterando las rutas de partición y provocando que sus ponderaciones *SHAP* bailen entre reentrenamientos.

C. Características de contribución despreciable (Consistencia pasiva)

- Comportamiento: Las variables irrelevantes o con impacto nulo muestran una concordancia casi perfecta $W \approx 1{,}0$) en la parte inferior del ranking.
- Razón algorítmica: Dado que prácticamente nunca se eligen para dividir nodos significativos, sus valores *SHAP* son consistentemente cercanos a cero y se mantienen ancladas al fondo de la clasificación.

2. El fenómeno de la curva en «U» en la superposición de modelos (*Top-K Overlap*)

Esta misma dinámica se comprueba al analizar el consenso entre modelos mediante la tasa de solapamiento (*Top-K Feature Overlap Rate*)1:

- Cuando se evalúan las 5 variables más importantes ($K = 5$), la coincidencia entre algoritmos es muy alta (~ 76,7 % a 100 % entre árboles).
- A medida que se amplía el corte para incluir variables de importancia moderada ($K$ =15), la concordancia cae a su mínimo histórico (~44,2 %), reflejando la dispersión del rango medio.
- Finalmente, al incluir la mayoría de las características ($K = 20$), la tasa repunta (~64,3 %), completando una curva en forma de U que confirma empíricamente la estratificación de la estabilidad explicativa.

3. Consecuencias críticas para la auditoría y la regulación

Comprender esta relación evita errores graves al utilizar *SHAP* en industrias reguladas (finanzas, seguros y tasación inmobiliaria):

- Cartas de Acción Adversa (*Adverse Action Notices*)**:** Si una entidad deniega una hipoteca o un crédito basándose en una variable de nivel medio (por ejemplo, justificando que una

métrica secundaria penalizó la decisión), dicha justificación puede ser un artefacto de la semilla aleatoria del entrenamiento y no un factor de riesgo robusto.

- Ensamble de explicaciones: Como buena práctica de gobernanza, se recomienda no confiar en el reporte *SHAP* de un único modelo, sino promediar las matrices *SHAP* sobre múltiples ejecuciones con diferentes semillas para diluir la volatilidad de los predictores intermedios.
- Filtro de estabilidad: Las variables moderadas que no superen un umbral riguroso de estabilidad deben omitirse de los informes explicativos dirigidos a clientes o reguladores.

Tarjeta 41

En el anverso pone: "¿Por qué el marco regulatorio financiero (como el GDPR) exige el uso de metodologías de XAI como SHAP?" La respuesta del anverso es: "Para cumplir con el 'derecho a una explicación' en decisiones automatizadas (créditos, tasaciones), garantizando auditar la equidad y ausencia de sesgos."

¿Podrías explicarme este tema más detalladamente?

El marco regulatorio financiero y de protección de datos (como el Reglamento General de Protección de Datos - GDPR de la Unión Europea, las directrices de Basilea III y normativas crediticias como la ECOA y la FCRA en EE. UU.) ha transformado la interpretabilidad de una simple herramienta técnica de depuración en una obligación legal de gobernanza y cumplimiento normativo.

Cuando las instituciones financieras y tasadoras sustituyen los modelos lineales tradicionales (como la regresión logística o hedónica) por algoritmos no lineales complejos (como *XGBoost* o redes neuronales), la opacidad de estas «cajas negras» choca frontalmente con las garantías de equidad y transparencia.

1. El mandato legal: El «Derecho a una Explicación» y la notificación adversa

Las regulaciones internacionales imponen exigencias concretas sobre la toma de decisiones automatizada:

- Artículo 22 del GDPR: Otorga a las personas el derecho a una explicación frente a procesos automatizados de toma de decisiones. Establece que las entidades financieras deben divulgar de forma comprensible y significativa la lógica subyacente (*meaningful logic disclosure*) que condujo a una decisión algorítmica específica.
- Notificaciones de Acción Adversa (*Adverse Action Notices*): Regulaciones como la *Equal Credit Opportunity Act* (ECOA) y la *Fair Credit Reporting Act* (FCRA), reforzadas por

circulares de la Oficina para la Protección Financiera del Consumidor (CFPB), exigen que cuando se deniega un crédito o una hipoteca, el prestamista debe entregar al solicitante una declaración con las razones específicas, precisas y comprensibles que motivaron el rechazo.

- Gestión de riesgos sistémicos (FSB y Basilea III): Entidades como el Consejo de Estabilidad Financiera (*Financial Stability Board* - FSB) han advertido que la opacidad e insuficiente interpretabilidad en los modelos de Inteligencia Artificial financiera pueden generar riesgos sistémicos y fallas de mercado.

2. ¿Por qué se exige específicamente *SHAP* frente a otros métodos?

En sectores de alto impacto como la banca y la tasación de activos, los auditores no pueden admitir explicaciones heurísticas o arbitrarias. *SHAP* se ha convertido en el estándar metodológico preferente por cuatro ventajas matemáticas y operativas directas:

1. Garantía axiomática de justicia y unicidad: Fundamentado en la teoría de juegos cooperativos, *SHAP* es el único marco de atribución local aditiva que satisface simultáneamente los axiomas de Eficiencia (Exactitud Local), Simetría, Jugador Nulo (*Missingness*) y Aditividad. Esto asegura que la distribución del impacto predictivo entre las variables sea unívoca y matemáticamente justa.
2. Descomposición aditiva en unidades comprensibles: *SHAP* expresa la predicción como la suma del valor base promedio del mercado más las contribuciones individuales de cada atributo:

$$f(x) = E[f(X)] + \sum_{j=1}^{M} \phi_j$$

   Esto permite traducir el comportamiento del árbol de decisión a métricas comprensibles para el usuario y el regulador: por ejemplo, detallar con exactitud cuántos puntos de probabilidad de impago o cuántos euros restó o sumó un factor específico.
3. Dualidad local y global: Permite satisfacer tanto la auditoría del caso individual (explicación local para el cliente afectado mediante *Waterfall plots*) como la fiscalización del comportamiento global del modelo (mediante gráficos *Summary* / *Beeswarm* y dependencias) para verificar las reglas macro del negocio.

3. Auditoría de sesgos, equidad y variables sustitutas (*Proxies*)

Uno de los principales riesgos señalados por los reguladores es la discriminación indirecta no intencionada. Aunque una entidad financiera o de tasación excluya variables protegidas explícitas (como género, raza u origen étnico) del conjunto de entrenamiento:

- El modelo puede aprender patrones discriminatorios a través de variables correlacionadas (por ejemplo, el código postal, el tipo de empleo o patrones de consumo) que actúan como sustitutos (*proxies*) encubiertos.
- La resolución exhaustiva de las rutas de decisión que ofrece *TreeSHAP* permite a los equipos de cumplimiento normativo y a los auditores externos inspeccionar cuantitativamente las contribuciones de cada atributo y verificar que el algoritmo no esté penalizando a colectivos protegidos mediante atajos indirectos.

4. Estabilidad y validación de modelos (*Model Risk Management*)

Bajo marcos como el Acuerdo de Basilea III o las guías de gestión de riesgo de modelos (SR 11-7 de la Reserva Federal), las entidades deben validar la consistencia interna de sus algoritmos.

El análisis de consistencia de características mediante *SHAP* (como la tasa de superposición *Top-K* entre diferentes modelos y el análisis de estabilidad ante distintas semillas aleatorias) permite a los validadores verificar si los factores clave identificados son señales económicas genuinas y estables o si son artefactos inestables que distorsionarían las justificaciones entregadas al público.

Tarjeta 42

En el anverso pone: "¿Cómo se define conceptualmente el vector de coalición o características simplificadas $z'$?" La respuesta del anverso es: "Es un vector binario $z' \in \{0,1\}^M$ donde '1' representa que una característica está presente en el cálculo y '0' representa que está ausente."

¿Podrías explicarme este tema más detalladamente?

El concepto de vector de coalición o características simplificadas ($z'$) constituye el puente matemático introducido por *Lundberg y Lee* (2017) para conectar la teoría de juegos cooperativos con los modelos de aprendizaje automático.

1. Definición formal y matemática

Formalmente, el vector de coalición se define como un vector binario de dimensión *M*:

$$z' = (z_1', z_2', \dots, z_M')^T \in \{0,1\}^M$$

donde:

- $M$ representa el número total de características simplificadas (o jugadores disponibles para formar coaliciones).
- $z_j' = 1$: Indica que la característica $j$ está presente en la coalición que se está evaluando.
- $z_j' = 0$: Indica que la característica $j$ está ausente de dicha coalición.

2. ¿Por qué se denomina «espacio de características simplificadas»?

En un modelo complejo de aprendizaje automático, el vector de entrada real $x$ vive en un espacio continuo o heterogéneo $\mathbb{R}^p$ (por ejemplo, metros cuadrados, años, coordenadas geográficas o matrices de píxeles). Un modelo de caja negra no puede procesar conceptos abstractos como "un subconjunto de jugadores".

Para resolver esto, *SHAP* crea un espacio simplificado binario $\{0,1\}^M$ que actúa como un tablero de control de interruptores (encendido/apagado):

- En datos tabulares (como precios inmobiliarios): Cada posición suele corresponder directamente a una columna o atributo (por ejemplo, $z_1'$= Superficie, $z_2'$= Antigüedad, $z_3'$= Ubicación).
- En imágenes: Un elemento $z_j'$ no representa un píxel individual, sino un *superpíxel* (un grupo contiguo de píxeles).
- En texto: Cada componente representa la presencia o ausencia de una palabra o *token* específico.

3. El modelo de explicación aditivo $g(z')$ SHAP formula la explicación local como un modelo lineal sustituto que opera exclusivamente sobre este vector binario $z'$:

$$g(z') = \phi_0 + \sum_{j=1}^{M} \phi_j \, z_j'$$

- $\phi_0$: Es el valor base o predicción esperada del modelo sin información ($E[f(X)]).
- $\phi_j$: Es el valor SHAP (coeficiente lineal) asignado a la característica $j$.
- Si una variable está ausente en la coalición ($z_j' = 0$), su término $\phi_j \cdot 0 = 0$ se anula automáticamente de la suma.

La instancia real para explicar ($x'$)

Cuando queremos explicar la predicción del caso de interés concreto (donde conocemos todas sus características reales), todas las variables están presentes3. Por lo tanto, su vector simplificado $x'$es un vector formado únicamente por unos:

$$x' = (1,1,\dots,1)^T$$

Al sustituir todos los $z_j' = 1$, la ecuación se reduce a la propiedad de Exactitud Local / Eficiencia:

$$g(x') = \phi_0 + \sum_{j=1}^{M} \phi_j = f(x)$$

garantizando que la suma del valor base y todas las atribuciones SHAP reproduzca exactamente la predicción final del modelo.

4. ¿Cómo simula el algoritmo que una variable está «ausente» ($z_j' = 0$)?

Un algoritmo entrenado (como *XGBoost*) no puede recibir entradas vacías o valores nulos artificiales sin alterar su estructura de inferencia4. Para evaluar qué predeciría el modelo bajo una coalición $z'$, SHAP utiliza una función de mapeo $h_x(z')$ que traduce el vector binario al espacio original de datos:

$$h_x: \{0,1\}^M \to \mathbb{R}^p$$

- Si $z_j' = 1$ (presente): La función asigna el valor exacto de la instancia $x$ que se está explicando (por ejemplo, 120 m²).
- Si $z_j' = 0$ (ausente): La función simula la ausencia sustituyendo ese atributo por valores de referencia tomados de un conjunto de datos de fondo (*background dataset*), integrando sobre la distribución marginal o condicional del mercado.

5. Ejemplo intuitivo en Valuación Inmobiliaria

Imagina un modelo para tasar un departamento específico $x$ con tres características:

1. Superficie = 90 m²
2. Antigüedad = 5 años
3. Garaje = Sí (1)

Para calcular el valor de *Shapley* de cada atributo, *SHAP* evalúa sistemáticamente diferentes combinaciones de coaliciones $z'$:

- $z' = (1,1,1)$: Evalúa la predicción completa con los tres datos reales del inmueble.

- $z' = (1,0,1)$: Evalúa una coalición donde la Superficie y el Garaje están presentes, pero la Antigüedad se marca como ausente (reemplazada por el promedio de antigüedad del mercado).
- $z' = (0,0,0)$: Ninguna característica está presente; la salida del modelo corresponde únicamente al valor base promedio $\phi_0$.

Al comparar las diferencias de predicción al encender y apagar el bit $z_j'$ a través de todas las coaliciones posibles, se obtiene el valor marginal exacto $\phi_j$ que aporta ese atributo.

Tarjeta 43

En el anverso pone: "Para explicar una imagen mediante *SHAP*, ¿qué representan los 'jugadores' en el juego de coaliciones?" La respuesta del anverso es: "Superpíxeles o grupos de píxeles interconectados en lugar de píxeles individuales."

¿Podrías explicarme este tema más detalladamente?

En la formulación original de *SHAP* desarrollada por *Lundberg y Lee*, la teoría de juegos cooperativos modela la predicción como el resultado de un juego donde los «jugadores» son las variables o características que se unen en coaliciones para recibir una recompensa justa.

Cuando el modelo analiza datos tabulares, cada jugador suele ser una columna o variable concreta (como la superficie o la antigüedad). Sin embargo, al trasladar este enfoque a la visión por computadora e imágenes, tratar cada píxel individual como un jugador presenta serias limitaciones técnicas y conceptuales.

1. ¿Por qué no se utilizan píxeles individuales como jugadores?

- Explosión combinatoria intratable: Una imagen estándar de resolución moderada (por ejemplo, $224 \times 224 \times 3$) contiene más de 150.000 valores numéricos. Calcular o aproximar valores de Shapley considerando cada píxel como un jugador independiente requeriría evaluar combinaciones sobre un espacio de $2^{150.000}$ coaliciones, lo que resulta computacionalmente inviable.
- Ausencia de significado semántico: Un píxel aislado (un único punto de color RGB) carece de contexto visual reconocible por sí mismo. Lo que un modelo de red neuronal convolucional (CNN) u otro clasificador reconoce son patrones, texturas, bordes y formas, los cuales solo emergen cuando múltiples píxeles vecinos actúan de forma conjunta.

2. ¿Qué es un «Superpíxel»?

Un superpíxel es un parche o grupo contiguo de píxeles interconectados que comparten propiedades visuales homogéneas, como color, brillo, intensidad o textura. Mediante algoritmos de segmentación de imágenes (como *SLIC* o *Quickshift*), la imagen se divide en un número manejable de regiones continuas (por ejemplo, entre 30 y 100 superpíxeles). Cada una de estas regiones pasa a ser un jugador individual $j$ en el juego de coaliciones de SHAP.

3. El vector de coalición $z'$y la función de mapeo $h_x(z')$ en imágenes

Para estimar la contribución marginal de cada superpíxel mediante la formulación aditiva de *SHAP*:

$$g(z') = \phi_0 + \sum_{j=1}^{M} \phi_j \, z_j'$$

el algoritmo opera sobre el vector binario de características simplificadas $z' \in \{0,1\}^M$, donde $M$ es el número total de superpíxeles:

- Superpíxel presente ($z_j' = 1$): Indica que el superpíxel $j$ forma parte de la coalición activa. La función de mapeo $h_x(z')$ mantiene intacta esa región con sus píxeles y colores originales.
- Superpíxel ausente ($z_j' = 0$): Simula que ese jugador no participa en la coalición. Como una red neuronal no puede recibir una matriz con "huecos", la función de mapeo $h_x(z')$ oculta esa región reemplazándola por un fondo neutro: habitualmente se pinta de gris, se difumina o se sustituye por el color promedio circundante45.

Al perturbar la presencia o ausencia de estos bloques mediante el muestreador de coaliciones, *SHAP* mide cuánto varía la predicción del modelo cuando un superpíxel específico entra o sale del juego.

Tarjeta 44

En el anverso pone: "¿Qué función cumple la transformación $h_x(z')$ en *KernelSHAP*?" La respuesta del anverso es: "Mapea el vector binario de coalición $z'$ al espacio original de características, reemplazando los 0s por valores de una instancia de referencia o aleatoria."

¿Podrías explicarme este tema más detalladamente?

La función de transformación $h_x(z')$ es el puente matemático y operativo fundamental de *KernelSHAP*. Permite conectar el mundo abstracto de la teoría de juegos cooperativos (donde los atributos se modelan como interruptores binarios encendidos o apagados) con el modelo predictivo de aprendizaje automático real, que exige entradas numéricas válidas para operar.

1. El dilema de origen: ¿Por qué hace falta una función de mapeo?

En el marco teórico de *SHAP*, el análisis local se formula a través de un modelo explicativo aditivo que opera sobre un vector simplificado de coalición binaria:

$$z' = (z_1', z_2', \dots, z_M')^T \in \{0,1\}^M$$

donde un 1 indica que la característica está presente en la coalición y un 0 indica que está ausente.

Sin embargo, los modelos predictivos reales $\hat{f}$ (como *XGBoost*, redes neuronales o SVM) no han sido entrenados con vectores binarios de ceros y unos, ni admiten que una columna "desaparezca" físicamente durante la inferencia. El modelo espera recibir un vector completo en el espacio original de características $\mathbb{R}^p$.

Para resolver esta incompatibilidad, *Lundberg y Lee* definieron la función de mapeo:

$$h_x: \{0,1\}^M \to \mathbb{R}^p$$

cuya misión es transformar cualquier vector binario abstracto $z'$ en una instancia sintética válida $z \in \mathbb{R}^p$ que el modelo sí pueda evaluar.

2. Mecánica operativa de $h_x(z')$

Para una observación específica $x$ que se desea explicar y una coalición muestreada $z'$, la transformación asigna los valores posición por posición:

- Para las características con $z_j' = 1$ (jugador presente): $h_x$ conserva el valor exacto que esa característica tiene en la instancia $x$ que estamos analizando (por ejemplo, Superficie $= 120$ m$^2$).
- Para las características con $z_j' = 0$ (jugador ausente): Dado que el algoritmo no puede evaluar un valor "vacío", $h_x$ simula la ausencia de información sustituyendo ese atributo por el valor correspondiente de otra observación tomada de un conjunto de datos de referencia (*background dataset*) o muestreada aleatoriamente de la distribución marginal de los datos.

Matemáticamente, para datos tabulares, evaluar el modelo sobre esta transformación equivale a integrar sobre la distribución marginal de las características ausentes:

$$\hat{f}\big(h_x(z')\big) = E_{X_{-j}}\big[\hat{f}(x_S, X_{-S})\big]$$

donde $x_S$ representa las variables presentes en la coalición y $X_{-S}$ las variables ausentes reemplazadas con datos de fondo.

3. Comportamiento según el tipo de datos

La naturaleza del reemplazo que realiza $h_x(z')$ varía según el tipo de problema:

| Tipo de dato | Jugador ($z'_j = 1$) | Simulación de ausencia ($z'_j = 0$) |
|---|---|---|
| **Tabular (inmobiliario, crédito)** [1] [6] | Valor real de la instancia $x$ analizada. | Valor tomado de una fila de referencia o muestra aleatoria del dataset de fondo [1] [6]. |
| **Imágenes** [8] [9] | Píxeles del superpíxel original [9]. | Región coloreada de gris neutro, difuminada o con el color promedio circundante [9] [10]. |
| **Texto** [11] | La palabra o token original del texto. | Omisión de la palabra o sustitución por un token de relleno (*mask/padding*). |

4. Su rol en el algoritmo de *KernelSHAP*

*KernelSHAP* estima los valores de *Shapley* mediante una regresión lineal ponderada. En este proceso, $h_x(z')$ interviene directamente en la generación del conjunto de entrenamiento del modelo sustituto:

1. Se muestrean múltiples vectores de coalición binarios $z_k{}' \in \{0,1\}^M$.
2. Se aplica la transformación $h_x(z_k{}')$ para obtener instancias de datos reales y se consulta la predicción del modelo: $\hat{f}\left(h_x(z_k{}')\right)$.
3. Se calcula el peso de cada coalición mediante el núcleo de Shapley $\pi_x(z_k{}')$.
4. Se optimiza la función de pérdida por mínimos cuadrados ponderados:

$$\min_{\phi} \sum_{z' \in Z} \pi_x(z') \left[ \hat{f}\left(h_x(z')\right) - \phi_0 - \sum_{j=1}^{M} \phi_j \, z_j{}' \right]^2$$

5. Los coeficientes lineales resultantes $\phi_j$ corresponden a los valores *SHAP*

5. Implicación metodológica: El supuesto de independencia

Al reemplazar las variables ausentes ($z_j{}' = 0$) con valores muestreados de la distribución marginal de forma independiente, $h_x(z')$ asume que las características presentes y ausentes no están correlacionadas.

En presencia de multicolinealidad severa (por ejemplo, en datos inmobiliarios entre la superficie total y el número de dormitorios), este mecanismo puede construir combinaciones híbridas poco realistas

que caen fuera de la distribución real de los datos, lo que constituye una de las principales limitaciones reconocidas de los métodos basados en permutación y de *KernelSHAP*.

Tarjeta 45

En el anverso pone: "¿Por qué *KernelSHAP* otorga el mayor peso a las coaliciones extremadamente pequeñas o extremadamente grandes?" La respuesta del anverso es: "Porque las coaliciones pequeñas revelan el efecto aislado principal de una característica y las casi completas revelan el efecto total (efecto principal más interacciones)."

¿Podrías explicarme este tema más detalladamente?

En ***KernelSHAP***, la asignación de pesos a las coaliciones mediante el núcleo SHAP (*SHAP kernel*) constituye el núcleo matemático que permite que una regresión lineal estime con exactitud los valores de *Shapley*.

La fórmula matemática del núcleo propuesta por *Lundberg y Lee* para una coalición simplificada $z' \in \{0,1\}^M es$:

$$\pi_x(z') = \frac{M-1}{\binom{M}{|z'|} \cdot |z'| \cdot (M-|z'|)}$$

donde $M$ es el número total de características y $|z'|$ es el tamaño de la coalición (es decir, el número de características presentes con valor 1).

A partir de esta estructura se explican las razones intuitivas, combinatorias y analíticas por las cuales las coaliciones extremadamente pequeñas ($|z'| = 1$) o extremadamente grandes ( $|z'| = M-1$) reciben el mayor peso:

1. Intuición Analítica: ¿Qué nos enseña cada tipo de coalición?

El objetivo fundamental de *SHAP* es aislar la contribución de cada característica individual. La cantidad de información que se puede extraer sobre un atributo depende críticamente del tamaño del grupo en el que se evalúa:

- Coaliciones mínimas (tamaño $|z'| = 1$) — El Efecto Principal Puro:
    - Si una coalición contiene una sola característica presente y todas las demás están ausentes, la predicción del modelo al evaluar esa instancia revela directamente el efecto aislado principal de esa característica respecto a la línea base o promedio del mercado. No hay ruido ni interferencia de otras variables porque ninguna otra está activa.

- Coaliciones casi completas (tamaño $|z'| = M - 1$) — El Efecto Total (Principal + Interacciones):
    - Si una coalición contiene todas las características menos una, al observar qué cambia cuando esa única variable ausente se incorpora a la coalición completa, aprendemos sobre su efecto total (su impacto individual más todas sus interacciones posibles con el resto del conjunto de variables).
- Coaliciones intermedias (ej. $|z'| = M/2$) — Mezcla difusa de interacciones:
    - Cuando participa la mitad de las características, la predicción está fuertemente acoplada por múltiples combinaciones e interacciones no lineales entre las variables activas. A partir de esa salida conjunta, resulta mucho más difícil deducir matemáticamente qué porción del resultado pertenece a una característica en particular.

Por esta razón, desde la perspectiva de la teoría de la información, las coaliciones de los extremos son las que proporcionan la señal más limpia y directa sobre el comportamiento de cada jugador.

2. Fundamento Combinatorio: La penalización por multiplicidad

En la fórmula del núcleo $\pi_x(z')$, el denominador incluye el coeficiente binomial $\binom{M}{|z'|}$:

$$\binom{M}{|z'|} = \frac{M!}{|z'|!\,(M - |z'|)!}$$

- Pocas combinaciones en los extremos: Solo existen $M$ coaliciones posibles con un solo elemento ($\binom{M}{1} = M$) y exactamente $M$ coaliciones con todas las variables excepto una ($\binom{M}{M-1} = M$).
- Explosión combinatoria en el centro: A medida que nos acercamos a la mitad ($|z'| = M/2$), el número de subconjuntos posibles explota exponencialmente (siguiendo el comportamiento de la distribución binomial o el triángulo de Pascal).

Dado que el valor de *Shapley* requiere calcular un promedio no sesgado que represente equitativamente a todos los tamaños de coalición, la fórmula debe compensar este desbalance. Si todas las coaliciones tuvieran el mismo peso, las coaliciones medianas dominarían de forma aplastante la regresión por mera abundancia estadística. Al dividir por $\binom{M}{|z'|}$, el núcleo contrarresta la multiplicidad y otorga a cada subconjunto individual de los extremos una ponderación mucho más alta.

3. Conexión y contraste con *LIME*

Este esquema de pesos marca la diferencia técnica esencial entre *SHAP* y *LIME*:

- *LIME*: Pondera las muestras según su cercanía geométrica a la instancia original. Por ende, cuantas más características estén ausentes (más ceros en el vector binario), menor es el peso que *LIME* asigna a esa observación.
- *KernelSHAP*: Otorga pesos altos tanto a las instancias con muchos unos (cercanas a la instancia original) como a las instancias con muchos ceros (cercanas al valor base). Esta simetría en forma de «U» es precisamente lo que garantiza que la solución de mínimos cuadrados ponderados satisfaga los axiomas de *Shapley*.

4. Consecuencia práctica en la optimización del muestreo

Dado que las coaliciones de tamaño 1 y $M-1$ concentran la gran mayoría del peso del núcleo, las implementaciones eficientes de *KernelSHAP* no muestrean a ciegas. En su lugar:

1. Asignan primero parte del presupuesto de cómputo para evaluar de forma determinista **las $2M$ coaliciones correspondientes a los extremos** (todas las de tamaño 1 y tamaño $M-1$).
2. Si sobra presupuesto, evalúan las de tamaño 2 y $M-2$.
3. Finalmente, muestrean aleatoriamente entre las coaliciones restantes de tamaño medio.

Esto asegura estimaciones mucho más precisas y estables consumiendo un número sensiblemente menor de evaluaciones del modelo.

Tarjeta 46

En el anverso pone: "¿Qué efecto tiene aplicar una regularización L1 (Lasso) a la función de pérdida ponderada de *KernelSHAP*?" La respuesta del anverso es: "Permite generar explicaciones dispersas (con menos características con atribuciones distintas de cero), sacrificando potencialmente la validez estricta de los valores de *Shapley*."

¿Podrías explicarme este tema más detalladamente?

La conexión entre la regularización L1 (*Lasso*) y *KernelSHAP* aborda uno de los dilemas clásicos en la Inteligencia Artificial Explicable (XAI): el conflicto entre la pureza axiomática matemática y la simplicidad cognitiva para el usuario humano.

1. La formulación de *KernelSHAP* como regresión lineal

A diferencia de los métodos de cálculo basados en árboles (*TreeSHAP*), *KernelSHAP* estima las contribuciones mediante un modelo sustituto lineal local:

$$g(z') = \phi_0 + \sum_{j=1}^{M} \phi_j \, z_j'$$

donde $\pi_x(z')$ es el vector binario de características presentes o ausentes. Para encontrar los coeficientes $\phi_j$, *KernelSHAP* optimiza una función de pérdida estándar de mínimos cuadrados ponderados:

$$L\big(\hat{f}, g, \pi_x\big) = \sum_{z\prime \in Z} \big[\hat{f}\big(h_x(z')\big) - g(z')\big]^2 \, \pi_x(z')$$

donde $\pi_x(z')$ es el núcleo SHAP que pondera adecuadamente las coaliciones.

Lundberg y Lee demostraron que, al minimizar esta suma de errores cuadráticos sin ninguna penalización adicional, los coeficientes resultantes $\phi_j$ coinciden exactamente con los valores de Shapley, satisfaciendo de forma unívoca los axiomas fundamentales de *Eficiencia (Exactitud Local)*, *Simetría*, *Jugador Nulo (Missingness)* y *Aditividad*.

2. ¿Qué ocurre al añadir una penalización L1 (Lasso)?

Dado que *KernelSHAP* está formulado dentro del marco clásico de la regresión lineal, es posible incorporar herramientas de regularización como la norma L1:

$$L_{\text{regularizada}} = \sum_{z\prime \in Z} \big[\hat{f}\big(h_x(z')\big) - g(z')\big]^2 \, \pi_x(z') + \lambda \sum_{j=1}^{M} \big|\phi_j\big|$$

La penalización L1 introduce dos efectos inmediatos:

1. Inducción de dispersión (*Sparsity*): Debido a la geometría de la norma L1, a medida que aumenta el parámetro de regularización $\lambda$, muchos coeficientes $\phi_j$ se comprimen y se fuerzan a ser exactamente cero.
2. Generación de explicaciones dispersas: En conjuntos de datos con muchas variables (por ejemplo, 40 atributos de una vivienda o cientos de indicadores financieros), un cálculo estándar de SHAP asigna un número continuo a todas y cada una de las variables. La regularización L1 "apaga" las características de impacto residual o secundario, dejando activas únicamente las $K$ variables más determinantes.

3. El costo teórico: El sacrificio de los axiomas de Shapley

El motivo por el cual la respuesta de la tarjeta didáctica señala que se «sacrifica potencialmente la validez estricta de los valores de *Shapley*» radica en las propiedades axiomáticas:

- Violación de la Exactitud Local (Eficiencia): El axioma de eficiencia exige que la suma de todos los valores de atribución iguale exactamente la diferencia entre la predicción y el valor base promedio ($\sum_{j=1}^{M} \phi_j = f(x) - \phi_0$). Al introducir la penalización *Lasso*, se produce un sesgo de contracción (*shrinkage bias*) que reduce la magnitud de los coeficientes restantes y anula otros, impidiendo que la suma matemática reconstruya de forma exacta la predicción del modelo.
- Pérdida de la solución unívoca: La teoría de juegos cooperativos demuestra que los valores de *Shapley* constituyen la única solución matemática que satisface simultáneamente los cuatro axiomas fundamentales. Al forzar artificialmente coeficientes a cero mediante regularización, los valores obtenidos dejan de ser formalmente valores de *Shapley* teóricos y pasan a ser aproximaciones heurísticas dispersas.

4. El puente entre *LIME* y *SHAP*

Este ajuste ilustra con precisión cómo se relacionan *SHAP* y LIME:

- *LIME*: Nació con la filosofía de entregar explicaciones dispersas y humanamente legibles (limitadas por defecto a unas pocas características principales, a menudo seleccionadas mediante Lasso).
- *SHAP*: Nació con el objetivo de garantizar solidez teórica y justicia matemática completa, atribuyendo crédito a todas las variables sin excepción.

Aplicar regularización L1 a *KernelSHAP* es el punto intermedio donde un analista decide de forma deliberada resignar rigor axiomático a cambio de obtener un reporte más conciso y fácil de comunicar a usuarios no técnicos.

Tarjeta 47

En el anverso pone: "¿Cuál es la diferencia entre *LIME* y *SHAP* respecto a la función de ponderación de instancias en su regresión local?" La respuesta del anverso es: "*LIME* pondera las instancias por proximidad a la instancia original; *SHAP* las pondera estrictamente de acuerdo con el núcleo de *Shapley* de la teoría de juegos."

¿Podrías explicarme este tema más detalladamente?

La diferencia entre *LIME* y *SHAP* en cuanto a la función de ponderación de instancias en su regresión local es el punto técnico exacto donde divergen la intuición geométrica heurística y el rigor axiomático de la teoría de juegos.

Ambos métodos operan bajo el principio de construir un modelo sustituto lineal local (*local surrogate model*):

$$g(z') = \phi_0 + \sum_{j=1}^{M} \phi_j \, z_j'$$

donde $z' \in \{0,1\}^M$ es el vector binario de características simplificadas (presentes o ausentes), y ambos ajustan los coeficientes $\phi_j$ minimizando una función de pérdida por mínimos cuadrados ponderados:

$$\min_{\phi} \sum_{z' \in Z} \pi_x(z') \left[\hat{f}\left(h_x(z')\right) - g(z')\right]^2$$

La clave de la diferencia radica exclusivamente en cómo define cada método la función de ponderación $\pi_x(z')$.

1. La ponderación en *LIME*: Proximidad geométrica heurística

*LIME* (*Local Interpretable Model-agnostic Explanations*) se diseñó bajo una idea intuitiva: el modelo sustituto lineal solo necesita ser fiel en la vecindad inmediata de la instancia $x$ que queremos explicar.

Por ello, *LIME* define su función de peso utilizando un núcleo de distancia exponencial (habitualmente basado en la distancia euclidiana o coseno):

$$\pi_x(z') = \exp\left(-\frac{D\left(x, h_x(z')\right)^2}{\sigma^2}\right)$$

Comportamiento del peso:

- A las instancias sintéticas que tienen casi todos los $z_j' = 1$ (es decir, muy cercanas a la observación original $x$), *LIME* les asigna un peso muy alto.
- A medida que una instancia pierde características reales (más ceros $z_j' = 0$), se aleja geométricamente de $x$ y su peso decae exponencialmente hacia cero.

El problema de la heurística:

- La elección del ancho de banda $\sigma$ y de la métrica de distancia $D$ es completamente arbitraria y empírica.

- Modificar ligeramente $\sigma$ puede alterar radicalmente los coeficientes resultantes.
- *LIME* no garantiza el cumplimiento de propiedades de equidad (por ejemplo, puede asignar atribución a variables que no aportan o dar distinta importancia a variables idénticas).

2. La ponderación en *SHAP* (*KernelSHAP*): El Núcleo de *Shapley*

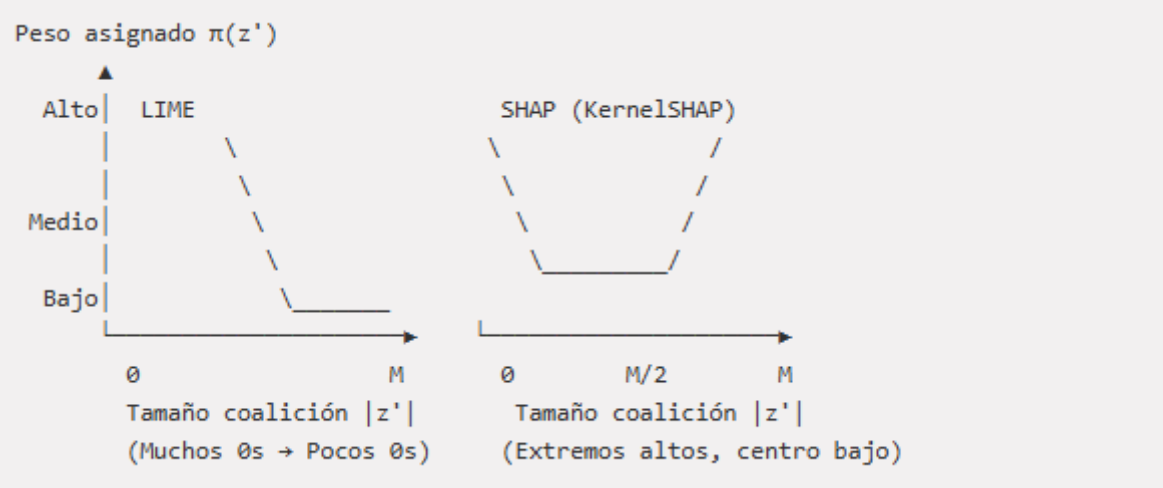
*Lundberg y Lee* (2017) abordaron el problema desde el extremo opuesto: se preguntaron qué ponderación $\pi_x(z')$ debía tener la regresión local para que sus coeficientes $\phi_j$ coincidan de forma unívoca con los valores de *Shapley* de la teoría de juegos cooperativos. La deducción matemática demostró que la única función de ponderación que satisface simultáneamente los axiomas de Eficiencia (Exactitud Local), Simetría, Jugador Nulo (*Missingness*) y Aditividad (Consistencia) es el *SHAP Kernel*:

$$\pi_x(z') = \frac{M-1}{\binom{M}{|z'|} \cdot |z'| \cdot (M-|z'|)}$$

onde $M$ es el número total de características y $|z'|$ es el número de características activas (unos) en la coalición.

3. Comparación de la forma de la curva de pesos: «Monótona decreciente» vs. «Forma de U»

La consecuencia más notable de esta diferencia se observa en la forma matemática que adoptan los pesos según el tamaño de la coalición $|z'|$:




LIME (Monótona decreciente):

- Solo le interesan las coaliciones cercanas a $M$ (casi todos unos).
- Las coaliciones pequeñas (muchos ceros, cercanas a la línea base) prácticamente no tienen peso en la regresión.

*SHAP* (Simétrica en forma de «U»):

- Coaliciones de tamaño 1 (un solo uno): Reciben un peso gigantesco, porque revelan el efecto principal puro de una característica aislada respecto al valor base.
- Coaliciones de tamaño $M - 1$ (todas menos una): Reciben también un peso gigantesco, porque revelan el efecto total (efecto principal + todas las interacciones posibles) al ver qué cambia cuando esa última variable se une al grupo.
- Coaliciones intermedias (cerca de $M/2$): Reciben un peso menor porque la presencia de muchas variables mezcladas diluye la señal marginal individual y existen exponencialmente más combinaciones de ese tamaño.

📊 Cuadro Comparativo Resumen

| Criterio | LIME | SHAP (KernelSHAP) |
|---|---|---|
| **Origen del núcleo** | Heurística geométrica (distancia euclidiana / coseno). | Deducción formal desde la teoría de juegos cooperativos. |
| **Parámetros libres** | Requiere calibrar arbitrariamente el ancho de banda $\sigma$. | **Cero parámetros arbitrarios**; la fórmula es única y exacta. |
| **Tratamiento de coaliciones pequeñas** | Peso casi nulo (se consideran "demasiado lejanas"). | **Peso máximo**, para aislar el efecto principal individual. |
| **Garantías matemáticas** | Sin garantías de aditividad o simetría. | **Garantiza los 4 axiomas de Shapley** (unicidad y consistencia). |

Tarjeta 48

En el anverso pone: "¿Cómo se interpreta la presencia de dispersión vertical en un *SHAP Dependence Plot* para una característica determinada?" La respuesta del anverso es: "Indica la presencia de efectos de interacción con otras variables del modelo que modifican la atribución de esa característica para un mismo valor en el eje X."

¿Podrías explicarme este tema más detalladamente?

1. ¿Cómo se construye un *SHAP Dependence Plot*?

A diferencia de los gráficos tradicionales de dependencia parcial (PDP), que calculan y grafican un promedio marginal aislado, un gráfico de dependencia *SHAP* grafica cada observación individual del conjunto de datos:

- Eje horizontal (X): El valor real de la característica $j$ de interés (por ejemplo, la superficie habitable en metros o pies cuadrados).
- Eje vertical (Y): El valor *SHAP* asignado a esa característica ($\phi_j$) para cada observación particular.

Si el modelo predictivo fuera puramente aditivo lineal (sin interacciones entre atributos), cada valor concreto en el eje X tendría un único impacto fijo en el eje Y, formando una línea o curva perfectamente delgada.

2. ¿Qué significa la dispersión vertical en el eje Y?

Cuando observas una columna o franja vertical de puntos para un mismo valor en el eje X, significa que propiedades con idéntico valor en esa característica reciben atribuciones SHAP muy distintas.

Por ejemplo, imaginemos dos viviendas que tienen exactamente la misma superficie habitable (150 m²):

- Para la Vivienda A, el modelo atribuye a esos 150 m² un valor SHAP de +25.000.
- Para la Vivienda B, el modelo atribuye a esos mismos 150 m² un valor SHAP de + 5.000$.

Esa diferencia vertical de 20.000 $ entre puntos situados sobre la misma coordenada X ocurre porque el modelo (por ejemplo, *XGBoost*) no evalúa la superficie de forma aislada: su aporte marginal depende de los valores que toman otras características secundarias con las que interactúa a lo largo de las ramas del árbol.

3. Identificación de la variable de interacción mediante código de colores

Para revelar qué variable está provocando esa dispersión vertical, la librería SHAP incorpora un mapa de color secundario (habitualmente asignado de forma automática a la característica con mayor índice de interacción de *Shapley*):

1. Cada punto se colorea según el valor de una segunda característica (rojo para valores altos, azul para valores bajos).
2. Patrón de interacción revelado:
    - Si observas que en la parte superior de la dispersión vertical los puntos son predominantemente rojos y en la parte inferior son azules, has identificado la interacción directa.

- En el ejemplo inmobiliario: tener 150 m² en una casa con calidad constructiva alta (*OverallQual* = 9, puntos rojos) aporta +25.000 $, mientras que los mismos 150 m² en una vivienda con calidad modesta (*OverallQual* = 5, puntos azules) aportan únicamente +5.000 $.

4. Ventaja analítica frente a los *Partial Dependence Plots* (PDP)

Los métodos globales tradicionales (como los PDP o ALE) resumen el efecto de una característica calculando un promedio y aplanando la varianza.

El gráfico de dependencia *SHAP* muestra simultáneamente:

- La tendencia principal (la trayectoria global no lineal de la nube de puntos).
- La varianza contextual y la magnitud de las interacciones a través del ancho de la dispersión vertical en cada tramo del dominio.

Tarjeta 49

En el anverso pone: "¿Qué función cumple el paquete *shap* de *Python* en el ecosistema de ciencia de datos?" La respuesta del anverso es: "Es la librería estándar desarrollada para calcular y visualizar explicaciones *SHAP* en modelos compatibles con *scikit-learn*, *XGBoost*, *LightGBM*, *CatBoost* y redes neuronales."

¿Podrías explicarme este tema más detalladamente?

El paquete *shap* en *Python* (creado inicialmente por *Scott Lundberg* y mantenido activamente por la comunidad de código abierto) es la librería de referencia y estándar de la industria para implementar Inteligencia Artificial Explicable (XAI) basada en la teoría de juegos cooperativos.

Su propósito central es cerrar la brecha entre la alta precisión predictiva de los modelos complejos («cajas negras») y la necesidad de auditar, entender y comunicar sus decisiones.

1. La Arquitectura Modular de Explicadores (*Explainers*)

En lugar de aplicar un único algoritmo genérico lento para todo, el paquete *shap* organiza el cálculo mediante clases especializadas según la arquitectura del modelo de *machine learning*:

- *shap.TreeExplainer*:
    - Optimizado específicamente para ensambles de árboles de decisión: *XGBoost*, *LightGBM*, *CatBoost*, *Random Forest* y modelos de *scikit-learn*.

- Aprovecha la estructura de árbol mediante el algoritmo *TreeSHAP* (así como aceleraciones nativas en GPU con *GPUTreeShap* y *backends* C++ como *Quadrature-TreeSHAP*), reduciendo la complejidad exponencial a un tiempo polinomial y lineal capaz de procesar cientos de miles de registros en segundos.

- *shap.LinearExplainer*:
  - Diseñado para modelos lineales de *scikit-learn* (regresión lineal, logística, *Lasso*, *Ridge*), calculando contribuciones exactas directamente a partir de los coeficientes del modelo.

- *shap.DeepExplainer* y *shap.GradientExplainer*:
  - Diseñados para modelos de aprendizaje profundo y redes neuronales (compatibles con *PyTorch*, *TensorFlow* y *Keras*), conectando la teoría de *Shapley* con algoritmos como *DeepLIFT* y gradientes esperados.

- *shap.KernelExplainer*:
  - El explicador agnóstico al modelo (*model-agnostic*). Puede evaluar cualquier función o tubería arbitraria como una caja negra ajustando una regresión lineal ponderada según el núcleo *SHAP*, aunque a un costo computacional sensiblemente mayor.

2. Catálogo Nativo de Visualizaciones

Una de las mayores fortalezas del paquete es su conjunto integrado de herramientas gráficas para traducir matrices matemáticas de números en visualizaciones comprensibles tanto para ingenieros como para auditores y directivos:

Explicaciones Locales (Una predicción concreta)

1. *shap.waterfall_plot*: Desglosa en forma de cascada cómo cada atributo empuja la predicción desde la línea base o esperanza promedio $E[f(X)]$ hasta el valor final estimado.
2. *shap.force_plot*: Muestra las variables como «fuerzas contrapuestas» (rojo para incrementos, azul para disminuciones) que equilibran la predicción.

Explicaciones Globales (El modelo completo)

1. *shap.summary_plot* (o gráfico *Beeswarm* / Enjambre): Resume en una única vista la jerarquía de importancia de las variables, el sentido del impacto (positivo o negativo) y la distribución del valor de cada característica mediante mapas de color.

2. *shap.dependence_plot*: Muestra cómo cambia la atribución *SHAP* de una característica a lo largo de su rango continuo, revelando relaciones no lineales y coloreando automáticamente los puntos según las interacciones entre pares de variables.
3. *shap.plots.bar* (*SHAP Feature Importance*): Ordena las características calculando el promedio de sus valores absolutos (mean(|SHAP|)), ofreciendo una alternativa axiomáticamente consistente a las métricas nativas de ganancia (*Gain*).

3. El Flujo de Trabajo Típico en *Python*

El uso del paquete en un proyecto estándar de ciencia de datos sigue una estructura simple y unificada:

```
import xgboost as xgb
import shap

# 1. Entrenar el modelo (ej. XGBoost con scikit-learn API)
modelo = xgb.XGBRegressor(n_estimators=100, max_depth=6)
modelo.fit(X_train, y_train)

# 2. Inicializar el explicador adecuado
explainer = shap.TreeExplainer(modelo)

# 3. Calcular los valores SHAP sobre el conjunto de prueba
shap_values = explainer(X_test)  # Genera un objeto Explanation

# 4. Generar diagnósticos visuales globales y locales
shap.summary_plot(shap_values, X_test)       # Diagnóstico global del mercado
shap.waterfall_plot(shap_values)           # Auditoría de una vivienda específica
```

4. Integración Bidireccional en el Ecosistema

La influencia del paquete shap es tal que los propios motores de *gradiente boosting* han adoptado su lógica internamente:

- *XGBoost* y *LightGBM*: Integran rutinas de cálculo de contribuciones *SHAP* directamente en su núcleo de C++ y *CUDALearn* (por ejemplo, a través de parámetros como *pred_contribs=True en predict*), permitiendo que el paquete shap sirva como una interfaz de alto nivel que delega el cómputo pesado al motor optimizado del árbol.
- Cumplimiento y Despliegue: Facilita la incorporación de auditorías de transparencia en aplicaciones web interactivas (como tableros de *Streamlit* para tasación inmobiliaria o análisis de riesgo crediticio) para sustentar decisiones automatizadas ante auditores y clientes.

Tarjeta 50

En el anverso pone: "¿Qué ventaja clave ofrece la integración de modelos *XGBoost* con *SHAP* en aplicaciones web desplegadas (ej. *Streamlit*) para tasación de inmuebles?" La respuesta del anverso es: "Permite a usuarios no técnicos interactuar con el modelo en tiempo real y comprender de manera transparente los factores que aumentan o disminuyen el valor estimado de un inmueble."

¿Podrías explicarme este tema más detalladamente?

La integración de *XGBoost* con *SHAP* en aplicaciones web interactivas (desplegadas comúnmente con *frameworks* como *Streamlit* o *Dash*) aborda una de las mayores barreras en la adopción práctica de la inteligencia artificial: llevar la transparencia algorítmica desde el código técnico hasta el usuario final en tiempo real.

1. El problema tradicional del despliegue predictivo: La caja negra aislada

Tradicionalmente, cuando un modelo de valoración automatizada (*Automated Valuation Model* - AVM) se despliega en una interfaz web o interna para tasadores, compradores o directivos bancarios, la plataforma funciona como un oráculo opaco:

- El usuario introduce las características del inmueble (superficie, dormitorios, ubicación, antigüedad).
- La interfaz devuelve un número cerrado: «Valor estimado: 285.000 €».
- La fricción: Si el tasador o el cliente no entiende por qué ese inmueble vale 285.000 € y no 260.000 €, surge desconfianza, rechazo de la tasación o dificultades para fundamentar la decisión ante comités de riesgo crediticio o auditorías regulatorias.

2. ¿Qué transforma la integración de *SHAP* en la aplicación web?

Al incorporar *TreeExplainer* de *SHAP* en el *backend* de la aplicación web, cada vez que el usuario presiona el botón de predecir ocurre un doble cálculo instantáneo:

1. *XGBoost* predice el precio final.
2. *SHAP* descompone de forma aditiva y exacta esa predicción a partir del promedio del mercado (valor base $E[f(X)]$) y calcula la contribución en dinero ($\phi_j$) de cada característica del inmueble.

En lugar de ver solo el precio total, la pantalla muestra un gráfico interactivo (habitualmente un *Waterfall Plot* o un Force Plot) donde:

- El valor base establece el estándar promedio del mercado (ej. 210.000 €).
- Barras rojas o positivas: Muestran con claridad los atributos que incrementaron el precio (ej. +45.000 € por tener 150 m², +20.000 € por ubicación en código postal prémium, +10.000 € por garaje).
- Barras azules o negativas: Revelan qué factores penalizaron el valor (ej. -15.000 € por antigüedad de 35 años, -7.000 € por calidad constructiva clasificada en 5.

3. La ventaja interactiva en tiempo real: Análisis de sensibilidad (*What-If*)

Al estar integrado en una plataforma dinámica como *Streamlit*, el usuario no técnico puede modificar los deslizadores (*sliders*) o menús desplegables y observar inmediatamente cómo cambian tanto el precio como la atribución explicativa:

- Simulación de renovaciones e inversión: Un valuador o propietario puede simular: *«¿Qué ocurre si remodelamos integralmente la vivienda (OverallQual pasa de 6 a 8)?»*. Al mover el control, la aplicación recalcula la predicción y el gráfico SHAP muestra visualmente cómo la penalización de calidad se transforma en una prima sustancial en euros.
- Compensación de factores: Permite entender cómo una variable negativa (como la antigüedad o un vecindario de menor demanda) puede verse compensada por una gran superficie habitable o acabados de lujo.

4. Viabilidad técnica: La velocidad de *TreeSHAP*

Esta experiencia fluida en la web solo es posible gracias a la eficiencia de *TreeSHAP* (y aceleraciones en GPU o C++ como *Q-TreeSHAP*):

- Mientras que métodos agnósticos antiguos (como *KernelSHAP*) tardaban decenas de segundos o minutos en evaluar un solo caso —lo que congelaría la página web—, *TreeSHAP* evalúa los árboles de *XGBoost* en milisegundos.
- La respuesta del servidor es prácticamente instantánea, permitiendo una experiencia interactiva sin latencias perceptibles.

5. Beneficios directos para los distintos perfiles de usuario

- Para el Tasador / Valuador Profesional: Le proporciona un desglose cuantitativo instantáneo que puede exportar directamente a su informe pericial o justificación técnica, respaldando su juicio profesional con evidencia matemática.

- Para el Cliente o Comprador: Elimina la sensación de arbitrariedad del algoritmo, aumentando la confianza al ver reflejadas de manera transparente las fortalezas y debilidades de la vivienda.
- Para la Entidad Financiera: Cumple con los requerimientos regulatorios de explicabilidad y transparencia en la concesión de hipotecas (como el RGPD o las normativas de préstamos justos), permitiendo auditar en tiempo real si el sistema se basa en factores legítimos del mercado.

Referencia bibliográficas

1. Fundamentos Teóricos de XAI, SHAP y Teoría de Juegos


- *Lundberg, S. M., & Lee, S.-I*. (2017). A unified approach to interpreting model predictions. En *Advances in Neural Information Processing Systems 30 (NIPS 2017)* (pp. 4768–4777). Curran Associates, Inc. https://doi.org/10.48550/arXiv.1705.078741.
- *Molnar, C.* (2022). *Interpretable machine learning: A guide for making black box models explainable* (2.ª ed.). Independently published. https://christophm.github.io/interpretable-ml-book/
- *Ribeiro, M. T., Singh, S., & Guestrin, C.* (2016). "Why should I trust you?": Explaining the predictions of any classifier. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 1135–1144). *Association for Computing Machinery*. https://doi.org/10.1145/2939672.29397787
- *Shapley, L. S. (1953)*. A value for $n$-person games. En H. W. Kuhn & A. W. Tucker (Eds.), *Contributions to the Theory of Games* (Vol. 2, pp. 307–317). *Princeton University Press*.


2. Algoritmos de Árboles, *Boosting* y Variantes de *TreeSHAP*


- *Chen, T., & Guestrin, C.* (2016). XGBoost: A scalable tree boosting system. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). Association for Computing Machinery. https://doi.org/10.1145/2939672.293978512more_horiz
- *Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., & Lee, S.-I*. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, *2*(1), 56–67. https://doi.org/10.1038/s42256-019-0138-915more_horiz
- *Mitchell, R., Frank, E., & Holmes, G.* (2022). *GPUTreeShap*: Massively parallel exact calculation of SHAP scores for tree ensembles. *PeerJ Computer Science*, *8*, e880.
- *Wettenstein, R., Mitchell, R., & Yu, P.* (2026). *Quadrature-TreeSHAP*: *Depth-independent TreeSHAP* and Shapley interactions. *arXiv preprint arXiv:2605.0449*719
- *Yu, P., Bifet, A., Read, J., & Xu, C.* (2022). Linear *TreeShap*. En *Advances in Neural Information Processing Systems (NeurIPS 2022)* (Vol. 35, pp. 25818–25828). *Curran Associates*, Inc.2021

3. Valuación Inmobiliaria Automatizada (AVM) y Modelos Espaciales (GIS)

- *Ezil Sam Leni, A., Revathi, T., & Sridhar Devarajan* (2024). A robust ensemble-based framework for house price estimation: Integrating XG-Boost with SHAP and web deployment. *Proceedings of Science and Technology*.
- *İban, M. C.* (2022). An explainable model for the mass appraisal of residences: The application of tree-based Machine Learning algorithms and interpretation of value determinants. *Habitat International*, *128*, 102641.
- *Kee, T., & Ho, W. K. O.* (2025). eXplainable machine learning for real estate: XGBoost and Shapley values in price prediction. *Civil Engineering Journal*, *11*(5), 2118–213525.
- *Lenaers, I., & De Moor, L.* (2023). Exploring XAI techniques for enhancing model transparency and interpretability in real estate rent prediction: A comparative study. *Finance Research Letters*, *58*, 104306.
- *Staško, A., & Grundspeņķis, J.* (2024). Literature review of explainable machine learning in real estate. En *Proceedings of the Workshops of the 36th International Conference on Advanced Information Systems Engineering (CAiSE 2024)*, CEUR-WS.org, *3698*, 58–733031.

4. Estabilidad, Auditoría Ética y Aplicaciones Financieras

- *Chan, C.-P., Tsai, C.-H., Tang, F.-K., & Yang, J.-H.* (2025). A SHAP-based comparative analysis of machine learning model interpretability in financial classification tasks. *Journal of Applied Economic Sciences*, *20*(3), 385–400. https://doi.org/10.57017/jaes.v20.3(89).
- *Lin, L., & Wang, Y.* (2025). SHAP stability in credit risk management: A case study in credit card default model. *Risks*, *13*(12), 238. https://doi.org/10.3390/risks131202383334
- *Slack, D., Hilgard, S., Jia, E., Singh, S., & Lakkaraju, H.* (2020). Fooling LIME and SHAP: Adversarial attacks on post hoc explanation methods. En *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society (AIES '20)* (pp. 180–186). Association for Computing Machinery. https://doi.org/10.1145/3375627.337583035
- *Thanathamathee, P., Sawangarreerak, S., Chantamunee, S., & Mohd Nizam, D. N.* (2024). SHAP-instance weighted and anchor explainable AI: Enhancing XGBoost for financial fraud detection. *Emerging Science Journal*, *8*(6), 2404–2430. https://doi.org/10.28991/ESJ-2024-08-06-0163637